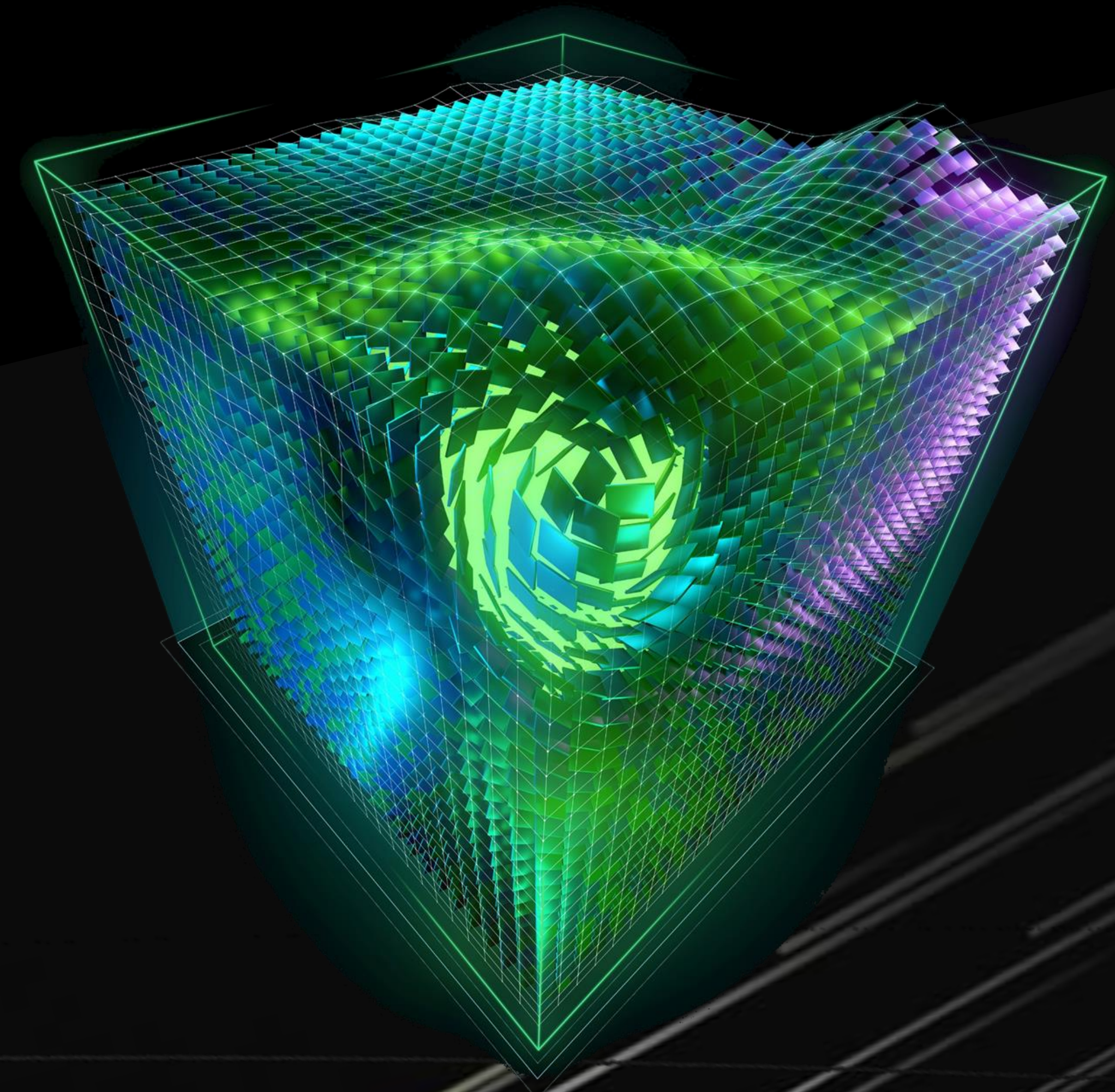




CUDA: New Features and Beyond

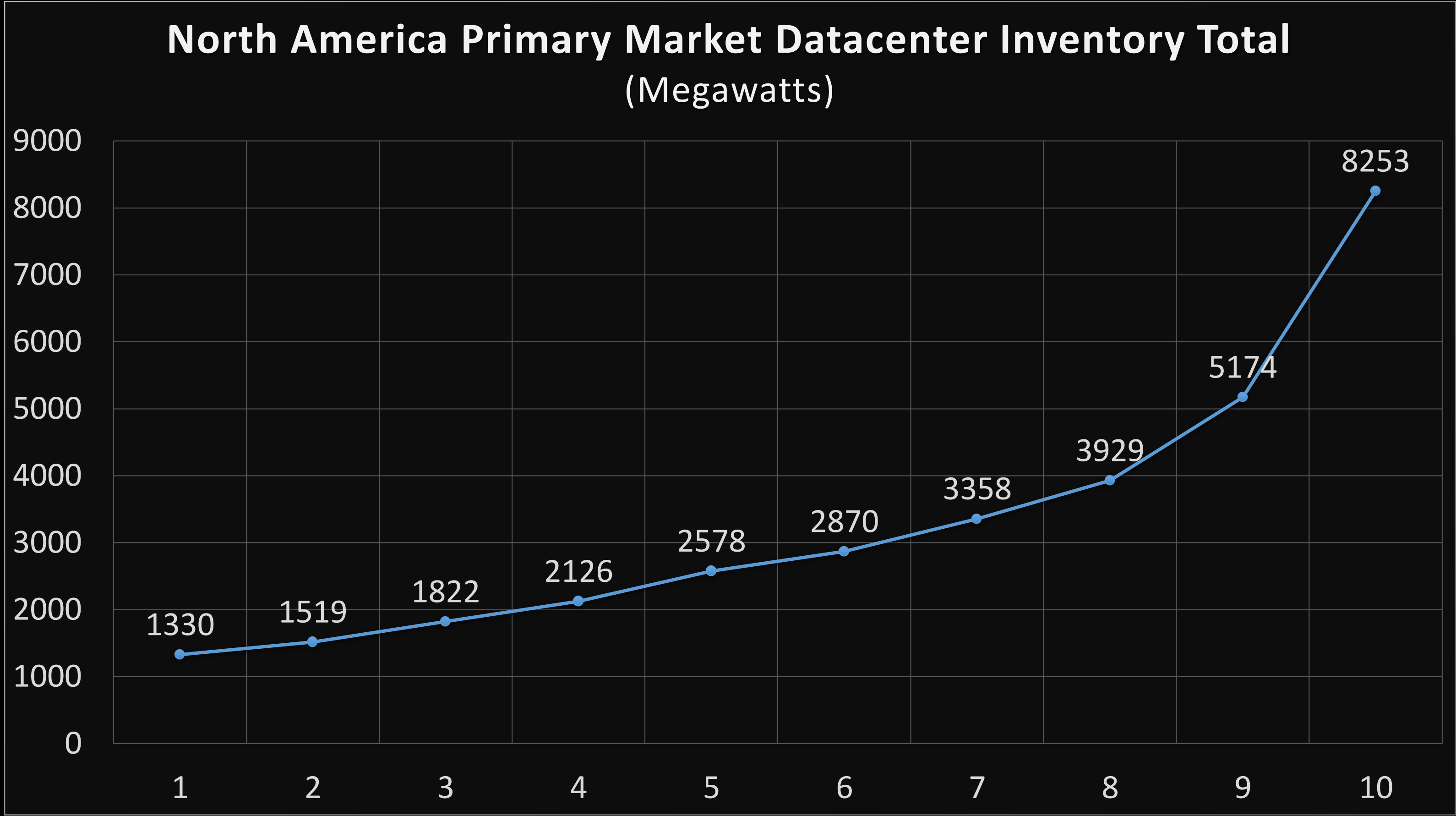
Stephen Jones, CUDA Architect | GTC 2024



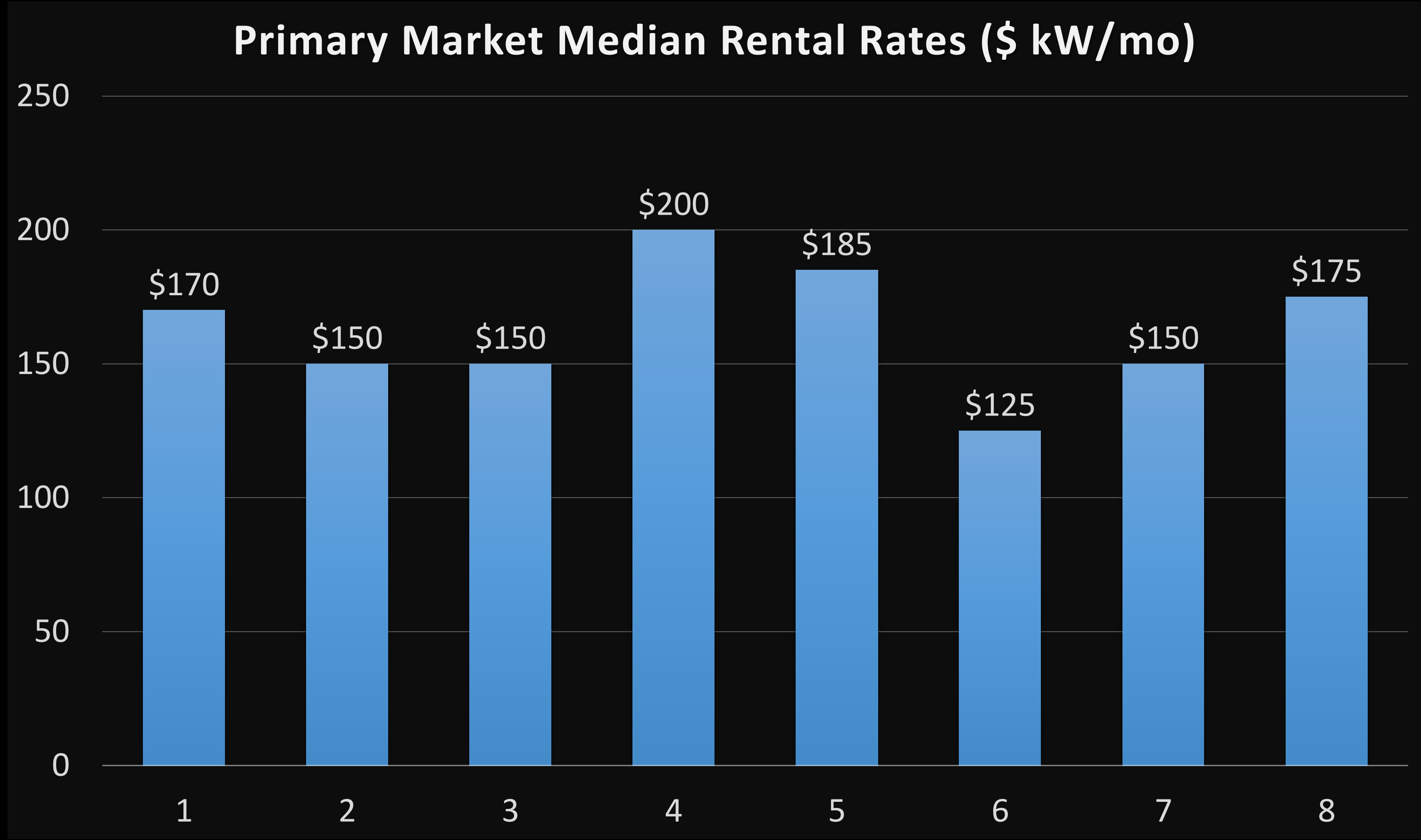
Accelerated Computing is not all about performance

*It's all about **ENERGY***

*It's about **Performance per Watt***



Source: CBRE - North America Data Center Trends H2 2023
(*) 2024 estimated: includes 3078MW currently under construction



Source: CBRE - North America Data Center Trends H2 2023



Typical Medium Sized Data Center

20,000 – 100,000 square feet

2,000 – 10,000 servers

5 – 20 MW

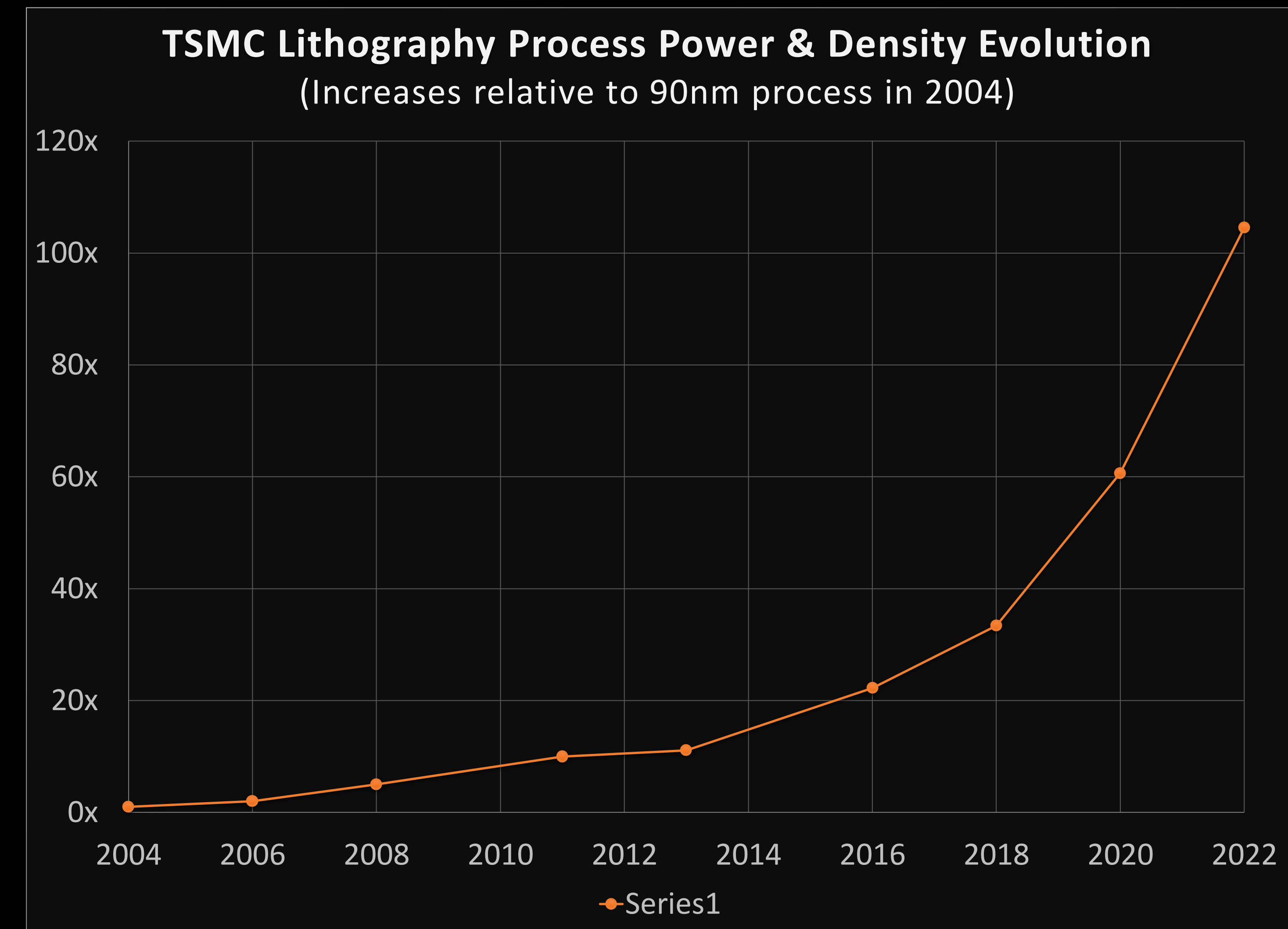
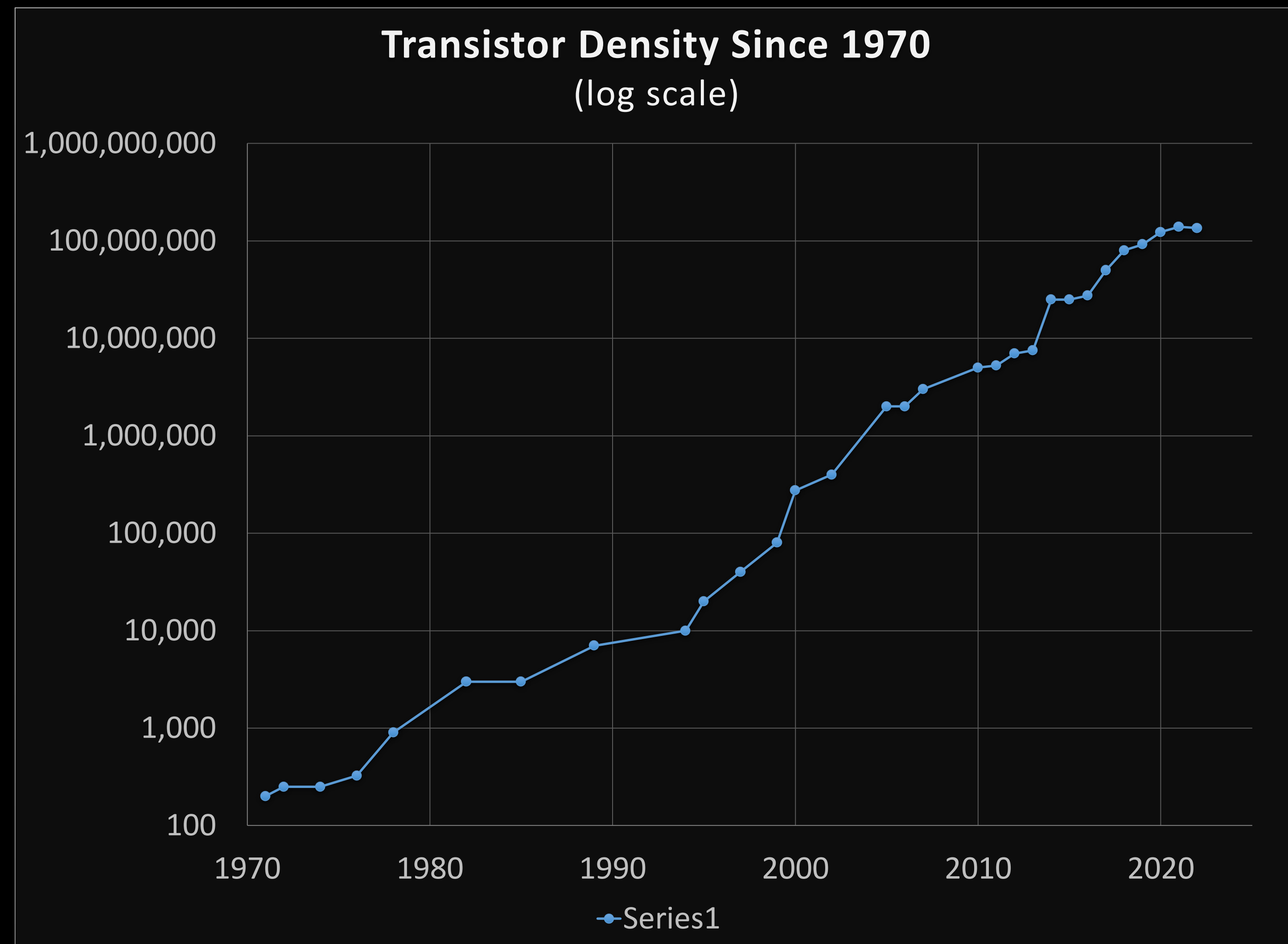


Max System Power: 1.5kW – 3kW



Max System Power: 250W

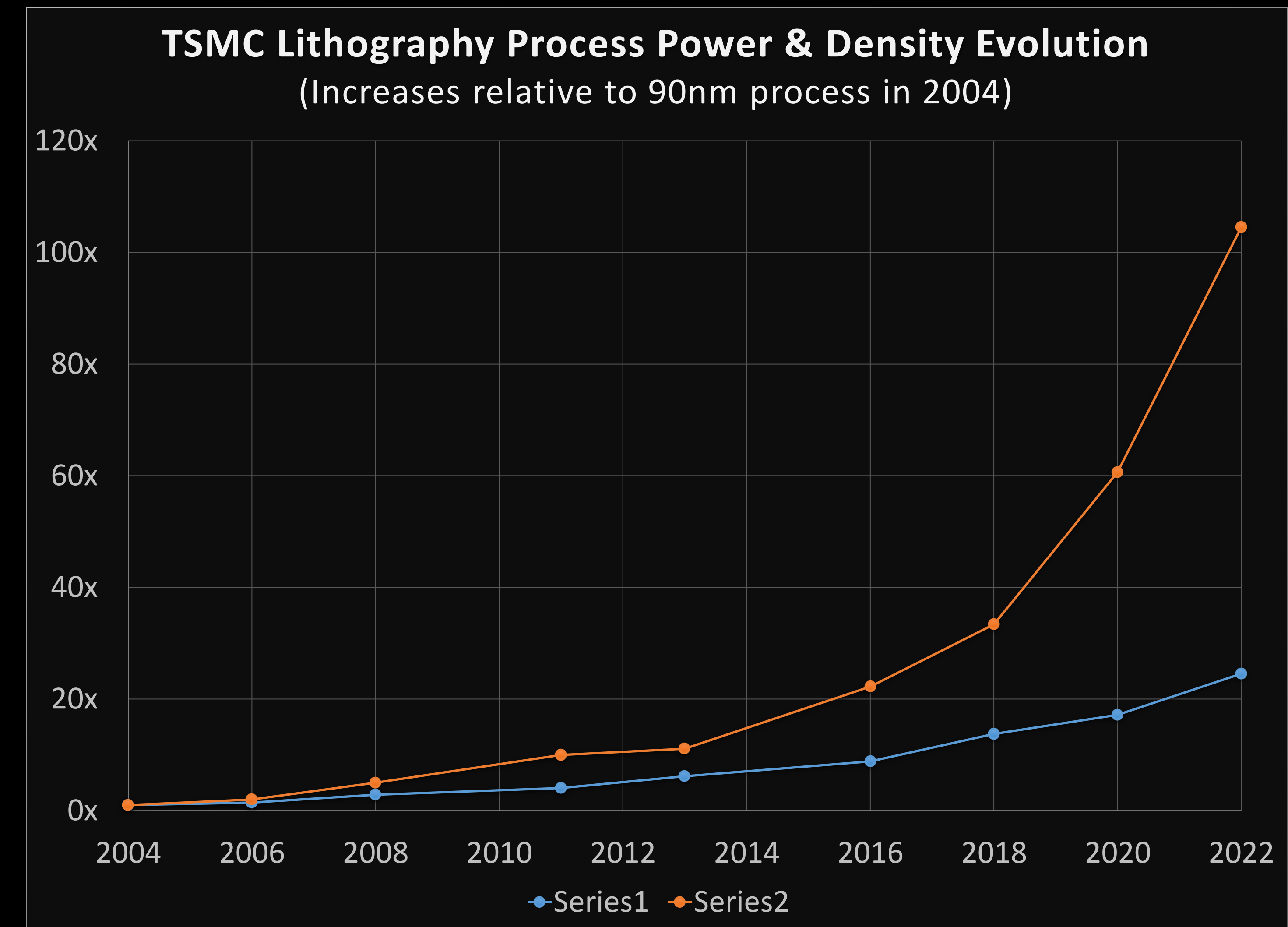
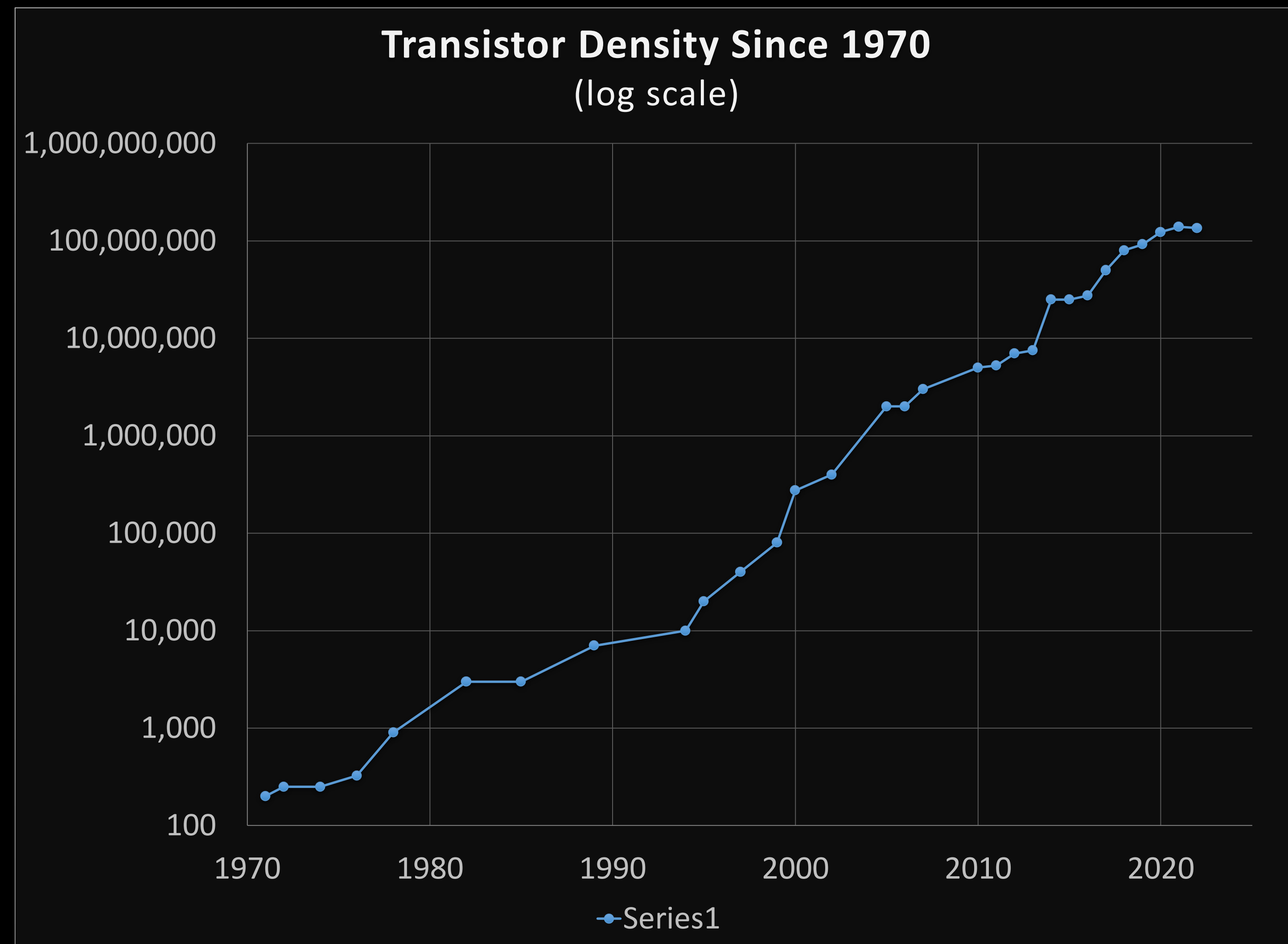
The Problem is That Transistor Density Keeps Increasing Exponentially



Sources: <https://spectrum.ieee.org/transistor-density>
IEEE International Roadmap for Devices and Systems, Stanford Nanoelectronics Lab, H.-S. P. Wong, et al., data accessed 17 October 2022

Source: TSMC

But Transistors Are Getting Less Power-Efficient



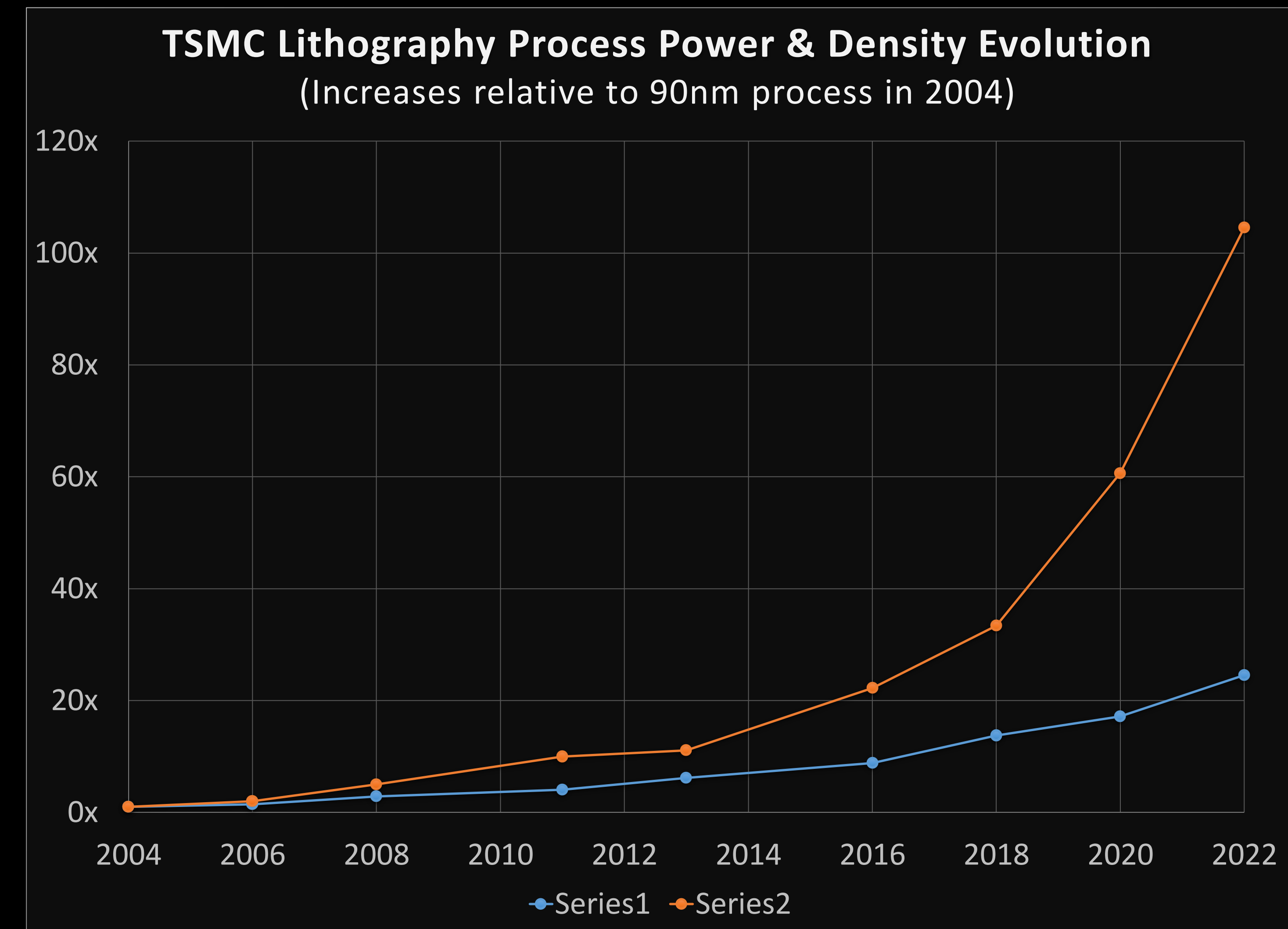
Sources: <https://spectrum.ieee.org/transistor-density>
IEEE International Roadmap for Devices and Systems, Stanford Nanoelectronics Lab, H.-S. P. Wong, et al., data accessed 17 October 2022

Source: TSMC

But Transistors Are Getting Less Power-Efficient

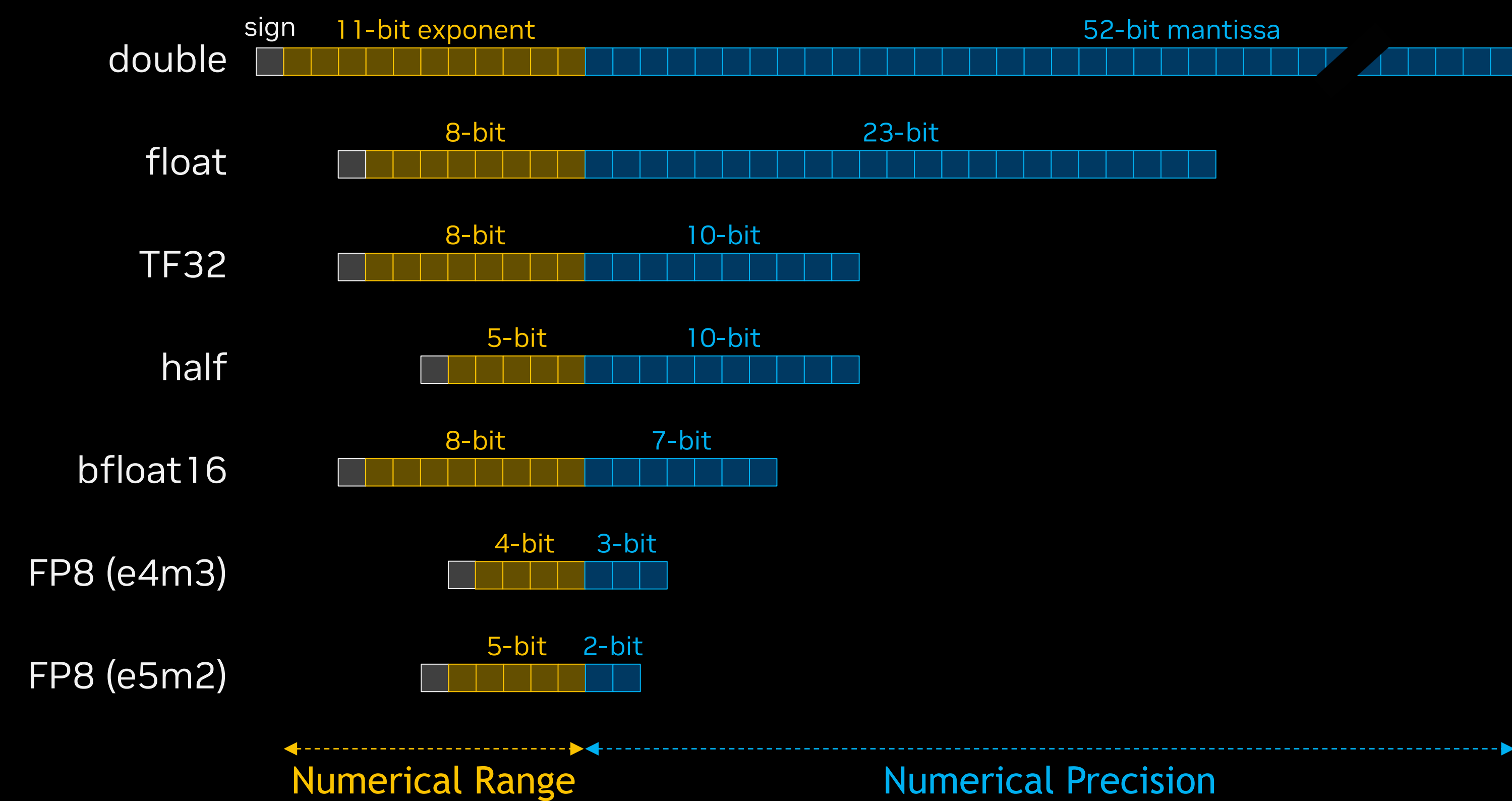
We spend power on two things

1. Data movement
2. Computation



Source: TSMC

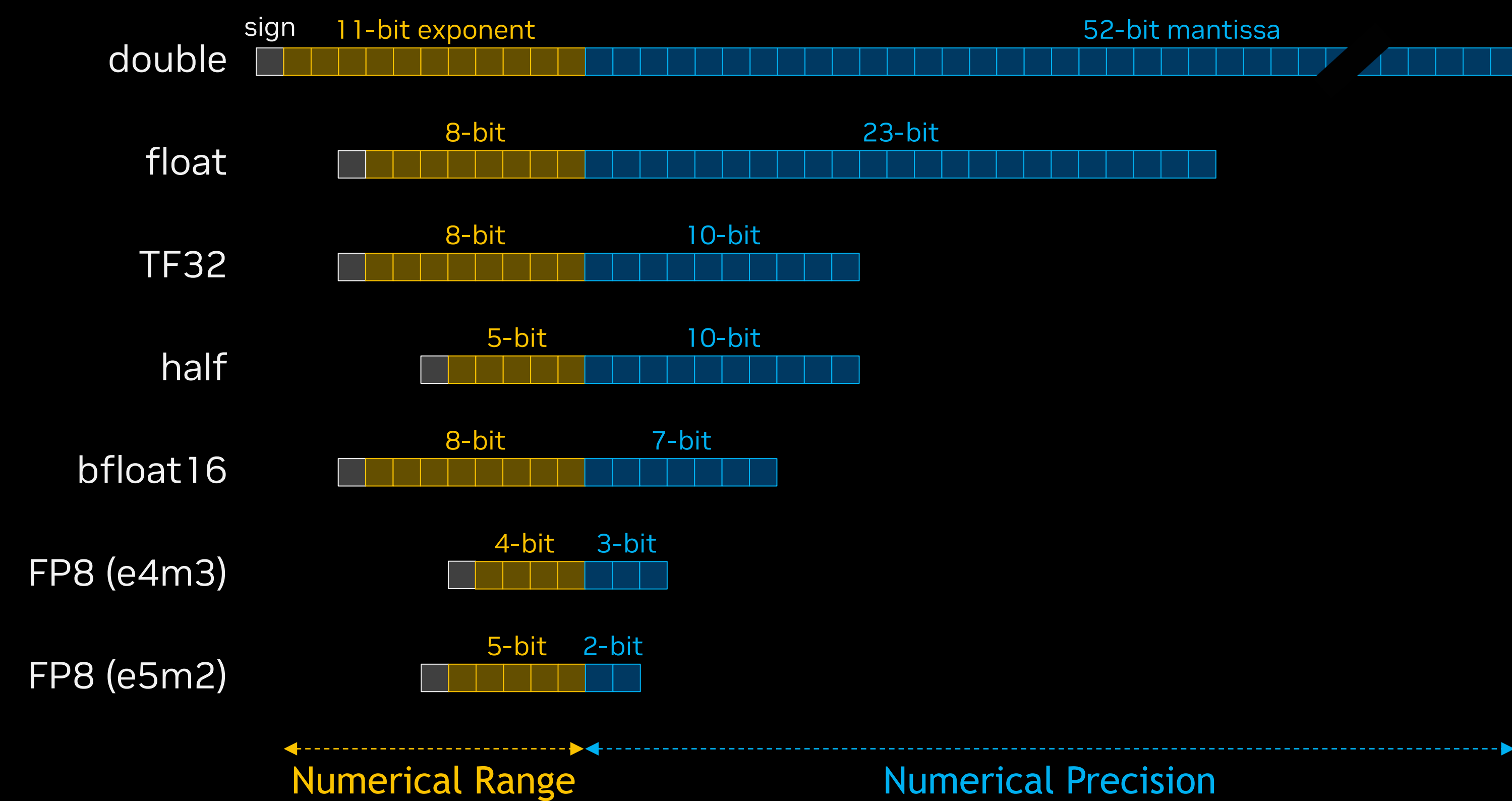
So Let's Talk About Floating Point Arithmetic



Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

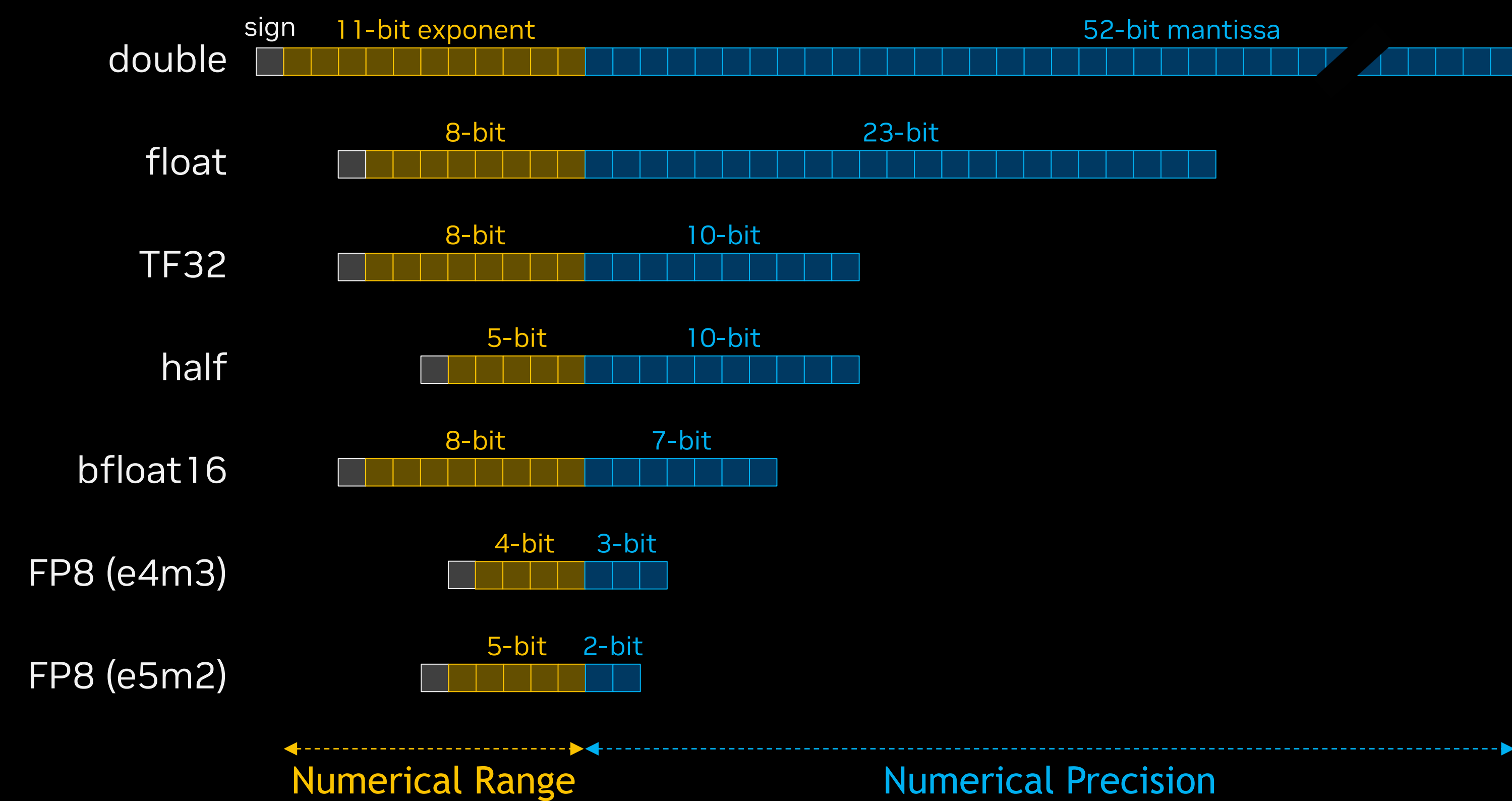
More to the point, the Power Cost of Floating Point Arithmetic



Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

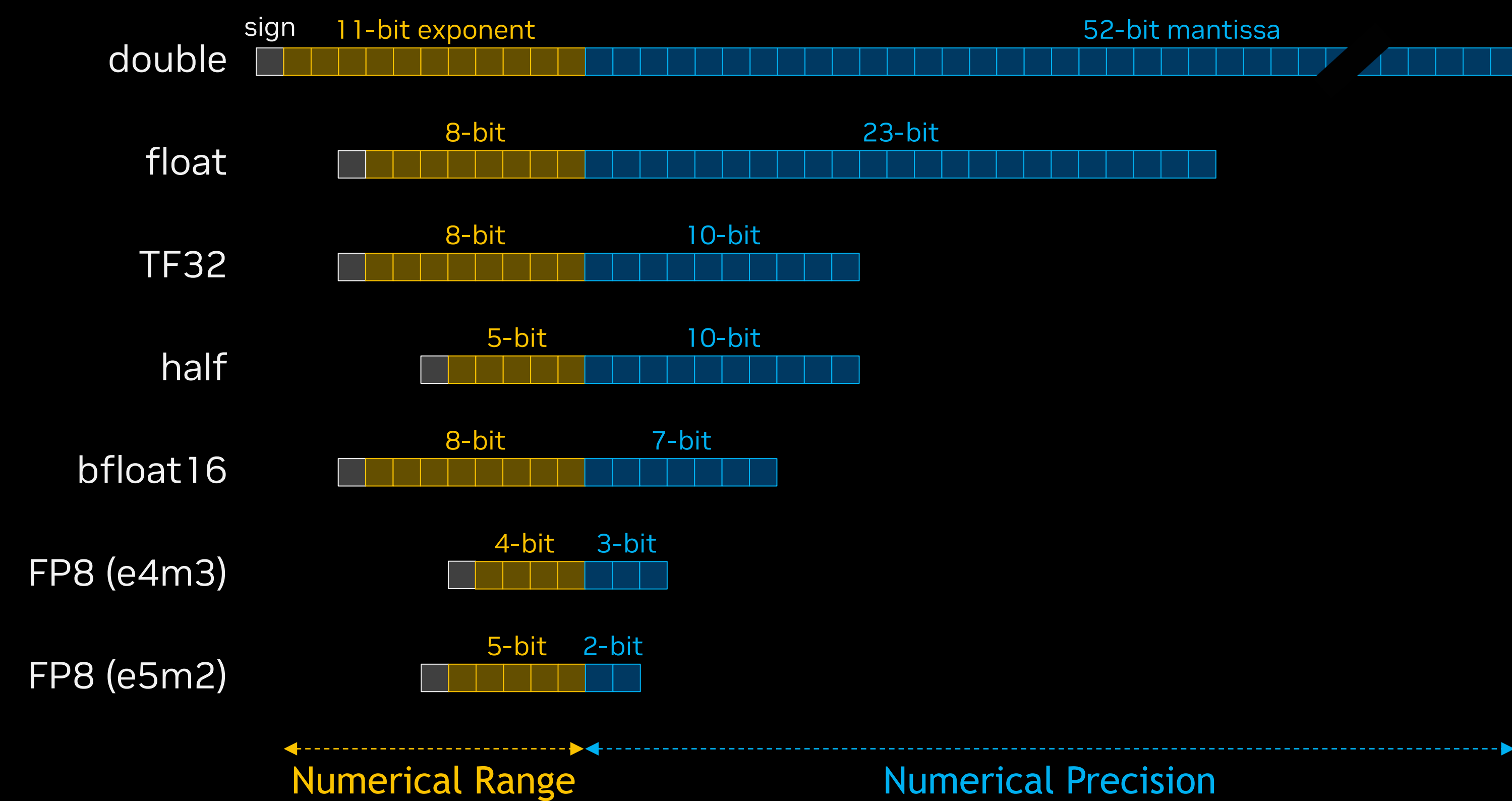
Tensor Cores are 1.5x-4x more power efficient than normal ops



Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

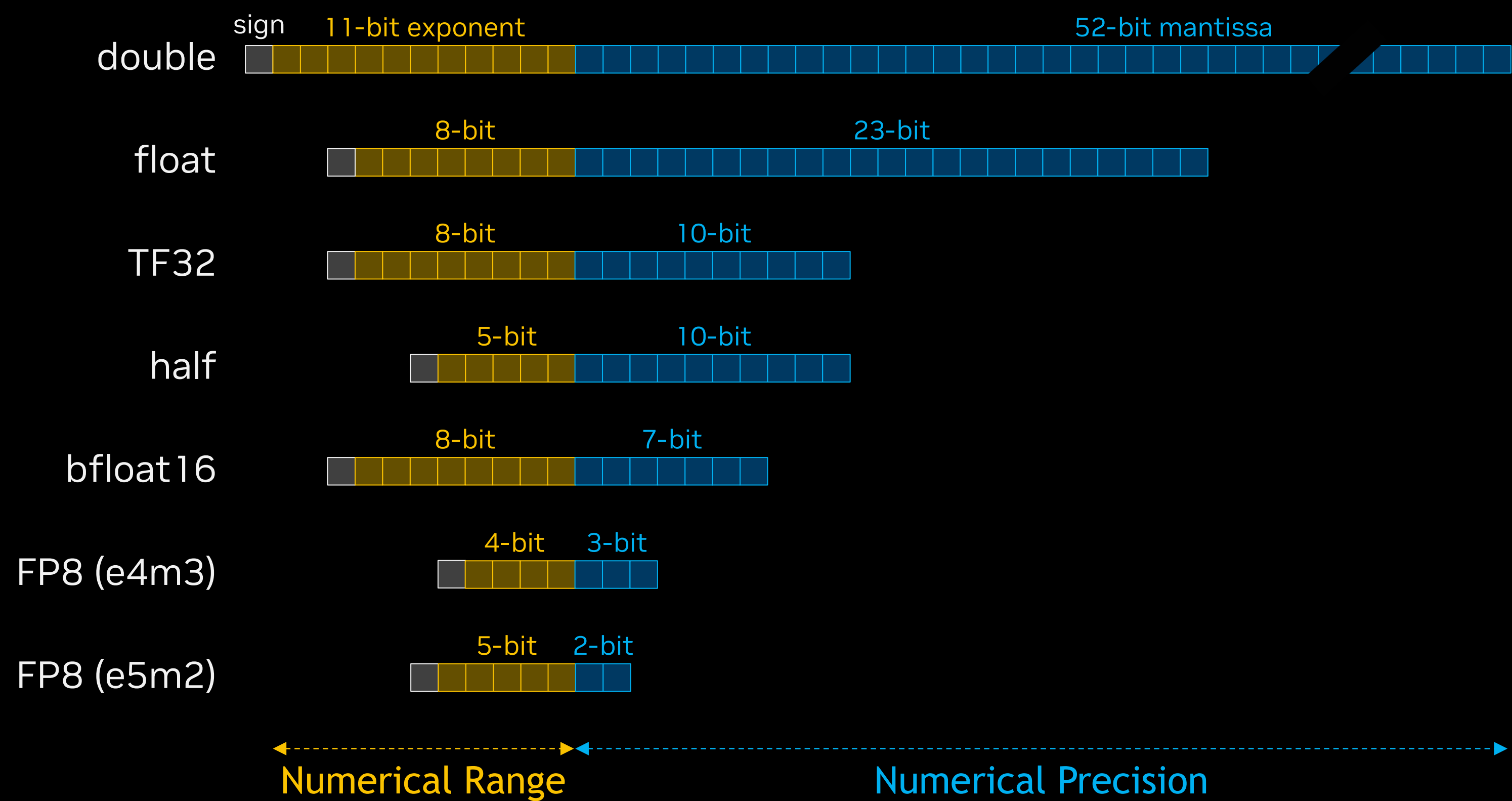
Tensor Cores are 1.5x-4x more power efficient than normal ops



Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

Tensor Cores are 1.5x-4x more power efficient than normal ops



Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

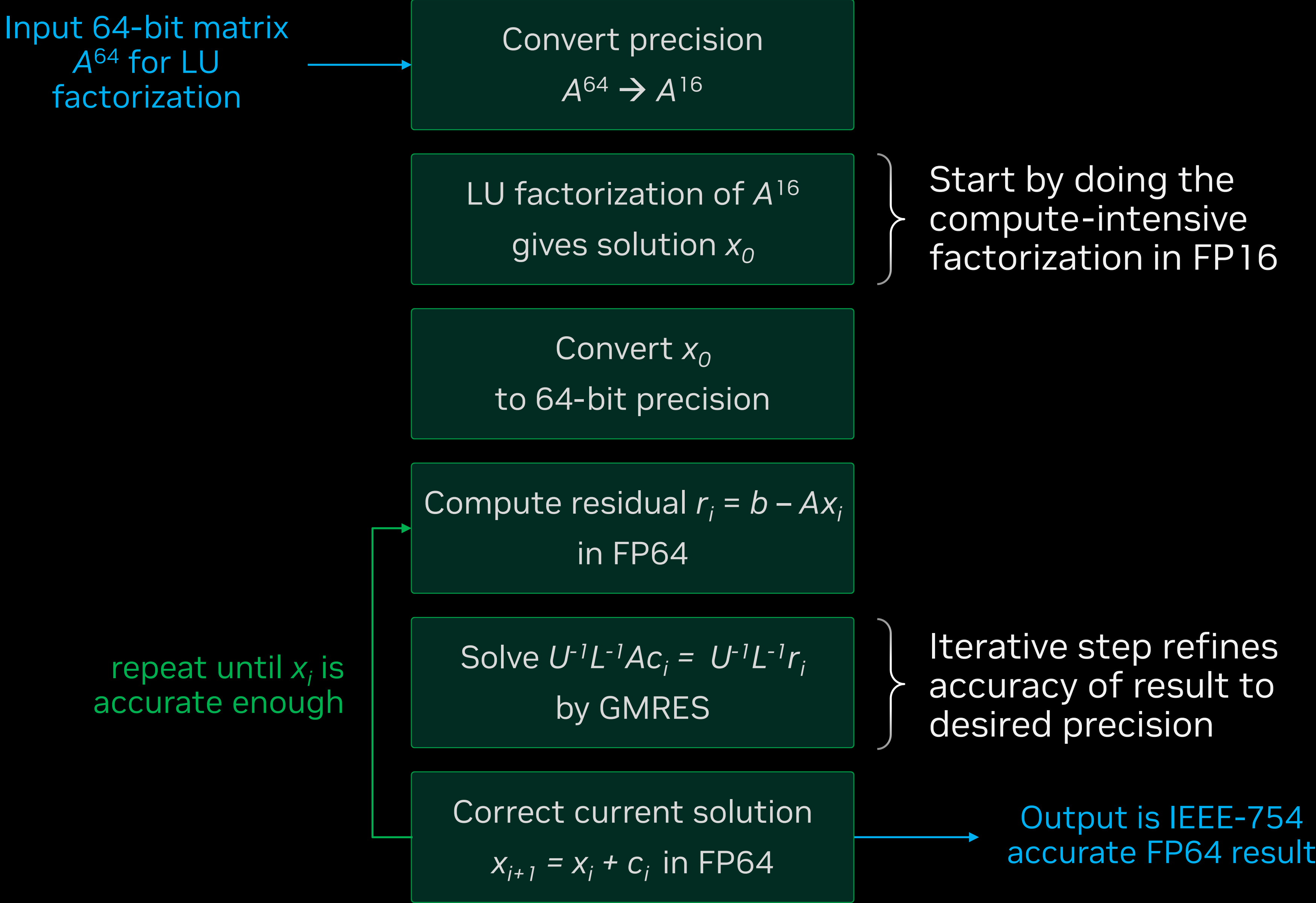
Factor of 20x in power efficiency

Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales
as the square of the mantissa length

Solving High Energy Problems With Low Energy Transistors

LU-factorization algorithm^[1] using low-precision tensor cores to give an IEEE-754 FP64 accurate result



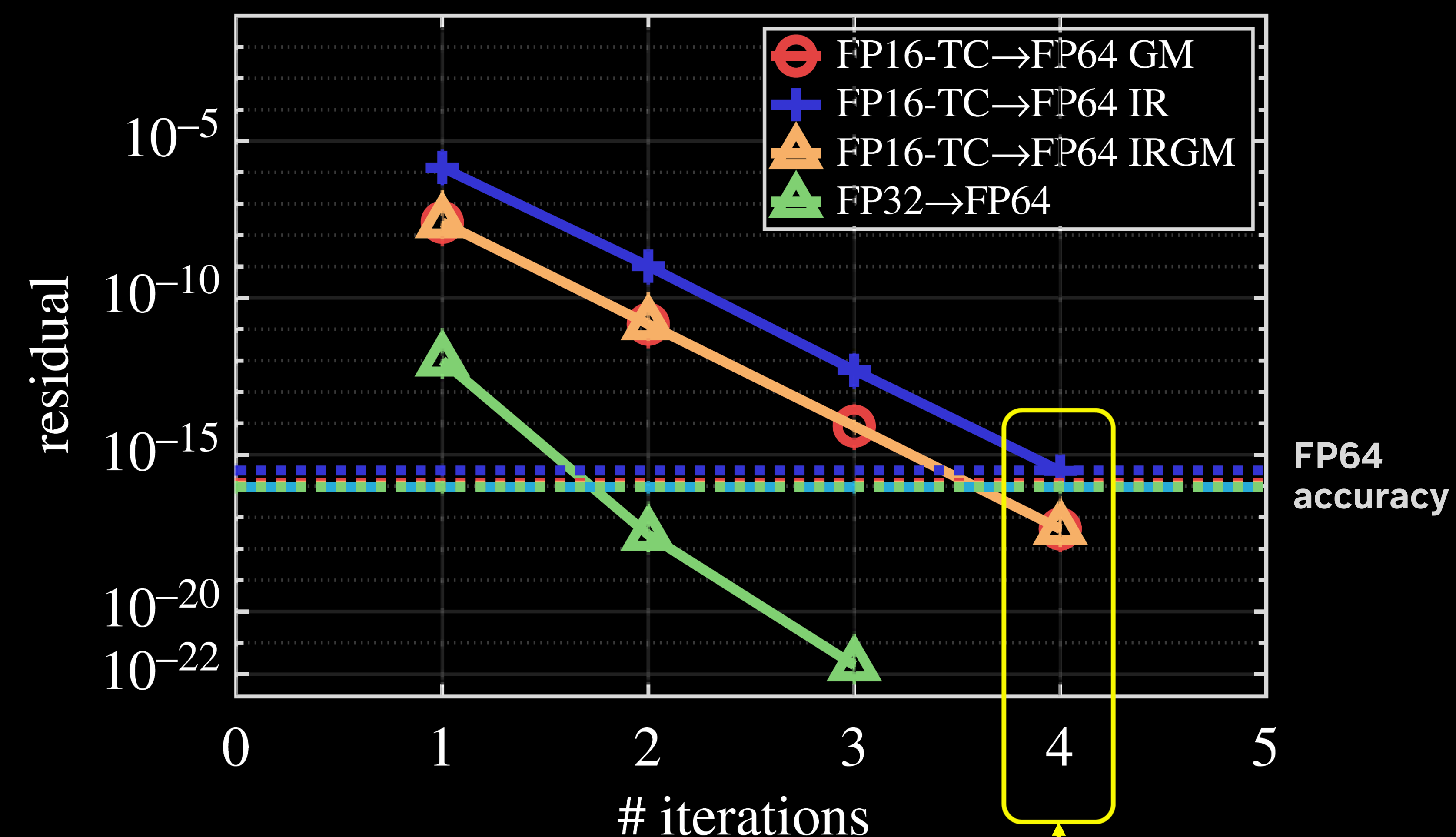
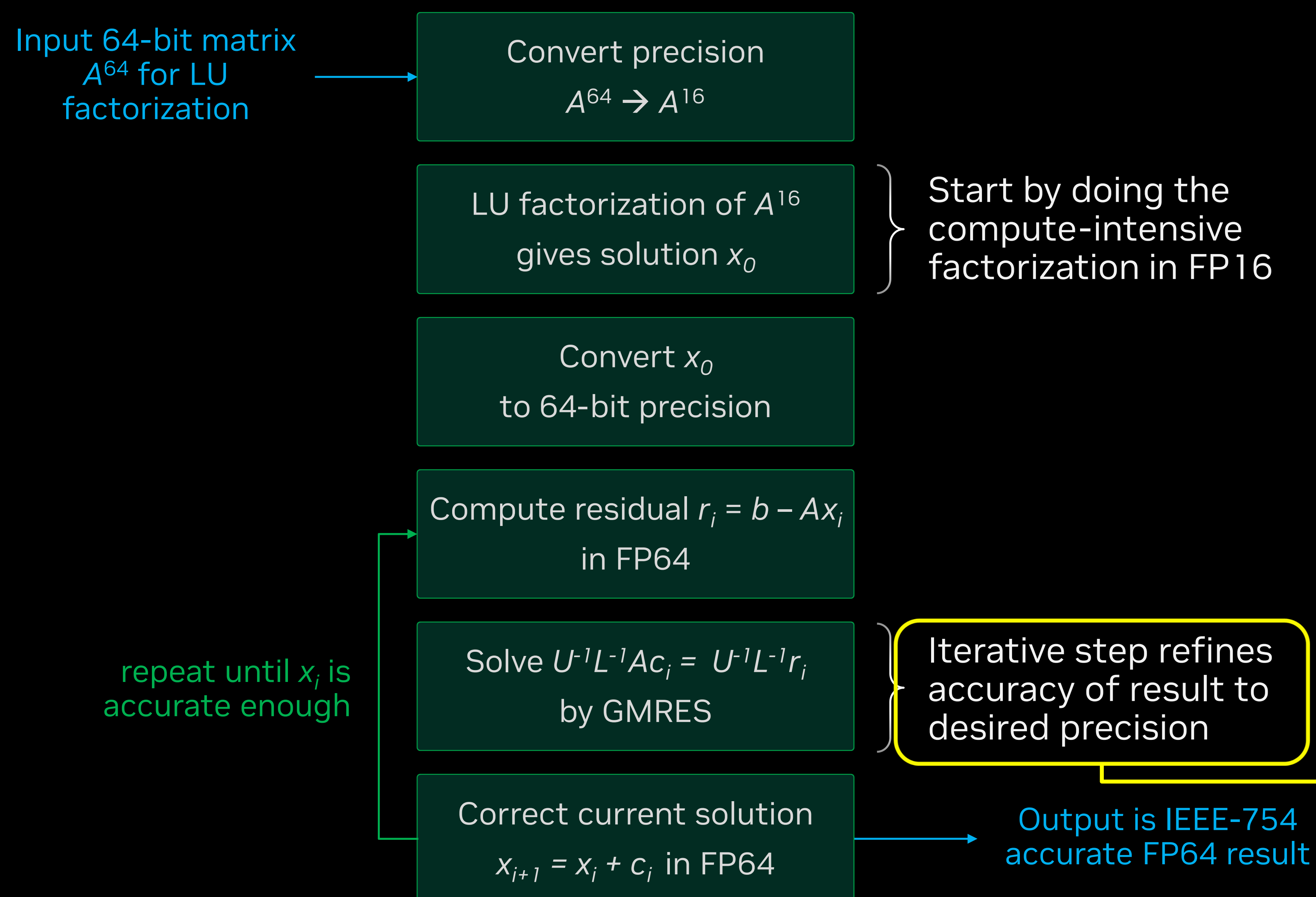
Precision	Operation	Energy per FLOP (Matrix Multiply)
FP64	FMA	2.5x
FP32	FMA	1.0x
FP16	FMA	0.5x
FP64	Tensor Core MMA	1.5x
FP16	Tensor Core MMA	0.12x
FP8	Tensor Core MMA	0.06x
Int8	Tensor Core MMA	0.04x

Power of floating point arithmetic generally scales as the square of the mantissa length

[1] Haidar Azzam, Bayraktar Harun, Tomov Stanimire, Dongarra Jack and Higham Nicholas J., 2020, Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems, Proc. R. Soc. A.4762020011020200110 <https://royalsocietypublishing.org/doi/10.1098/rspa.2020.0110>

Full FP64 Precision on FP16 Tensor Cores

LU-factorization algorithm^[1] using low-precision tensor cores to give an IEEE-754 FP64 accurate result

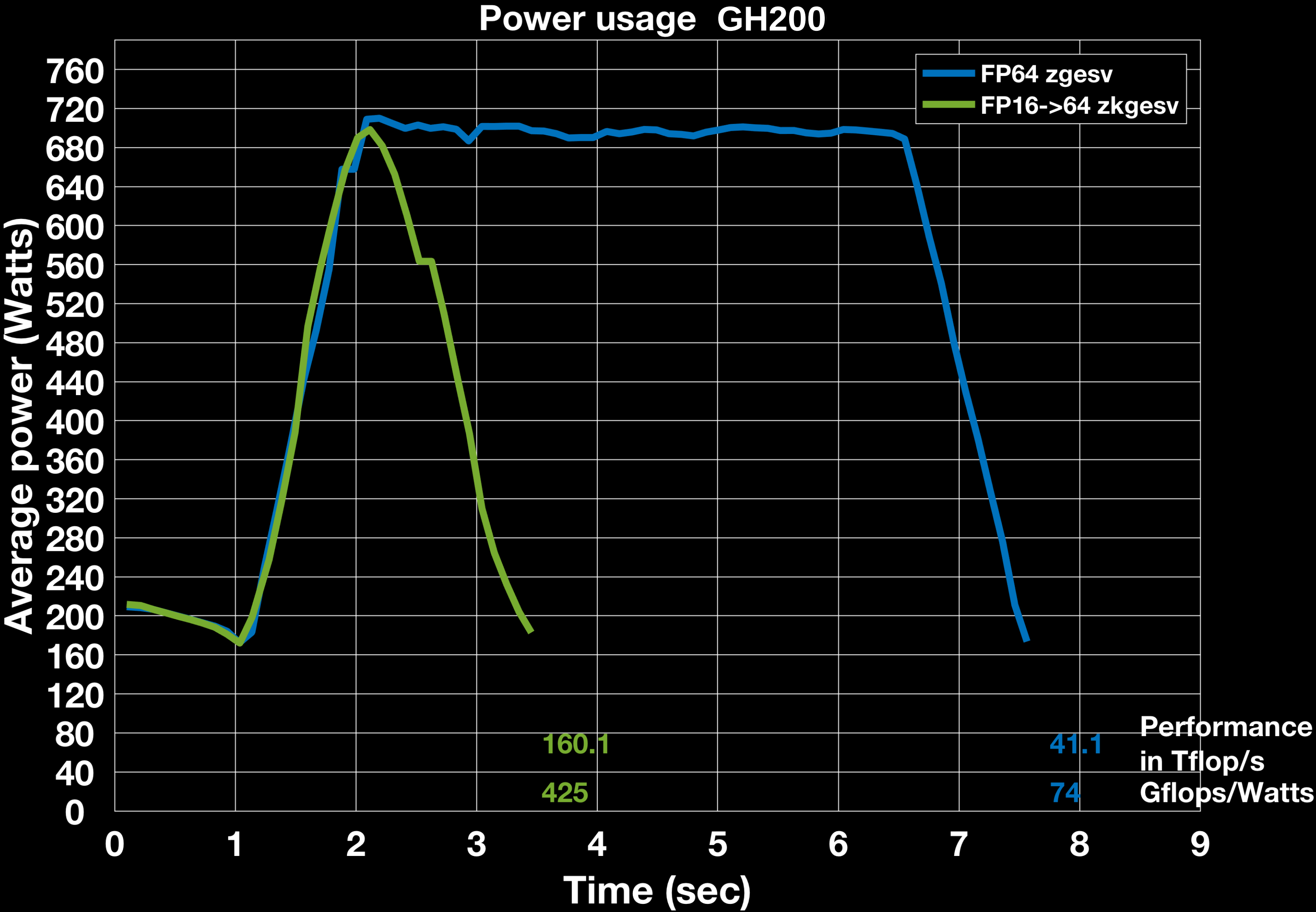


[1] Haidar Azzam, Bayraktar Harun, Tomov Stanimire, Dongarra Jack and Higham Nicholas J., 2020, Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems, Proc. R. Soc. A.4762020011020200110 <https://royalsocietypublishing.org/doi/10.1098/rspa.2020.0110>

Having Your Cake And Eating It: Both Power **and** Performance

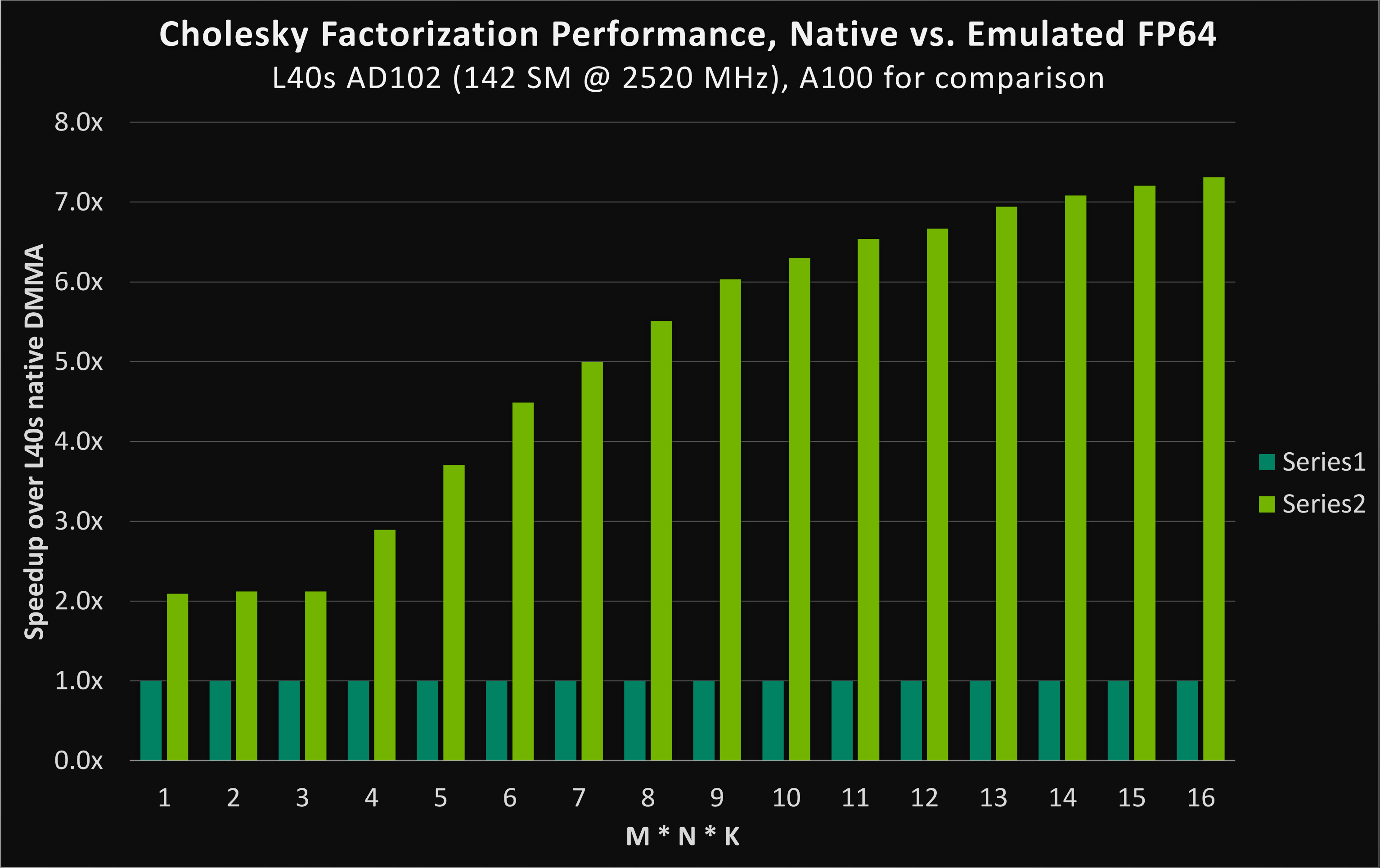
	FP64	FP16+FP64	Mixed Precision Benefit
Performance (TFLOP/s)	41.1	160.1	3.9x
Perf/Watt (GFLOPs/Watts)	74	425	5.7x

Performance and Perf/Watt improvement when using FP16 tensor cores with iterative refinement method



Complex matrix solve via LU decomposition
ZGESV on GH200, n=44000

High Performance FP64 GEMM On Non-Native Hardware



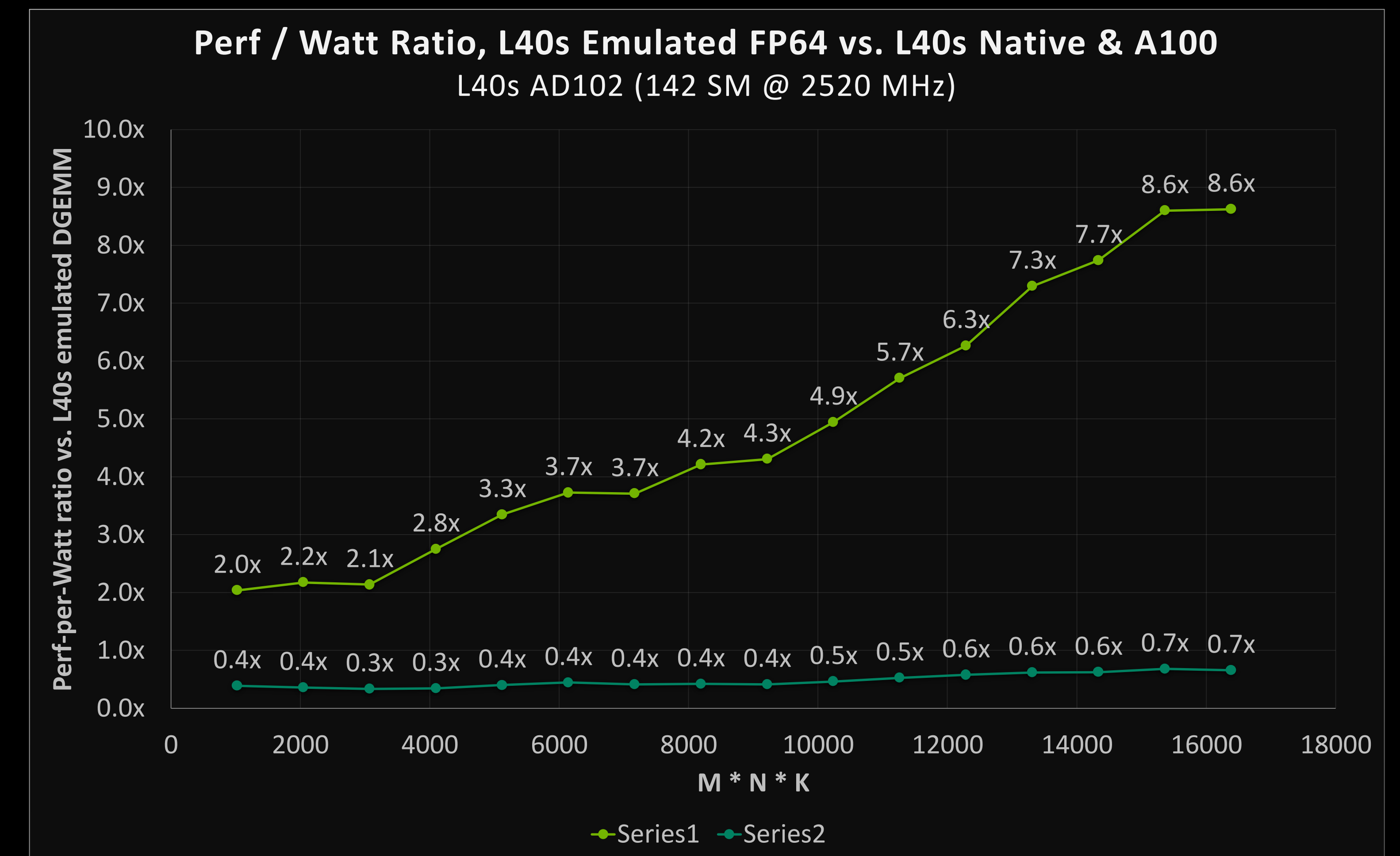
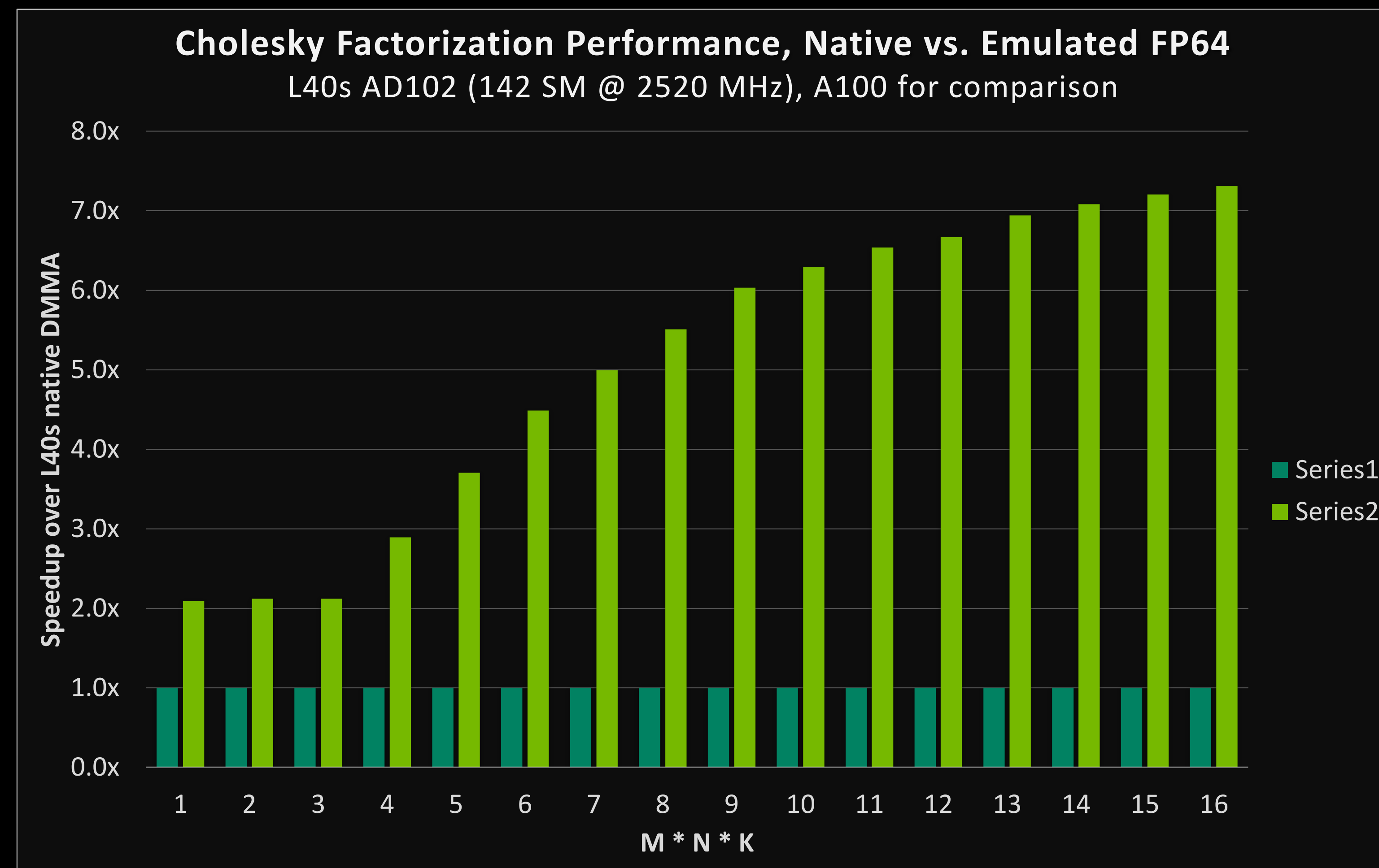
Based on the method of Ootomo et. al.^[2] to apply integer tensor cores to DGEMM, L40s achieves a 7x performance increase over native FP64 (and reaches 54% of A100 FP64 tensor core performance)

[2] Ootomo Hiroyuki, Ozaki Katsuhisa, Yokota Rio, 2023, DGEMM in Integer Matrix Multiplication Unit
arXiv:2306.11975v3 [cs.DC] 28 Dec 2023



NVIDIA L40S GPU	
GPU Architecture	NVIDIA Ada Lovelace
GPU Memory	48GB GDDR6 w/ ECC
Memory Bandwidth	864 GB/s
FP64 TFLOPS	1.4
FP32 TFLOPS	91.6
TF32 Tensor Core FLOPS	183
Bfloat16 Tensor Core TFLOPS	362
FP16 Tensor Core TFLOPS	362
FP8 Tensor Core TFLOPS	733

High Performance FP64 GEMM On Non-Native Hardware

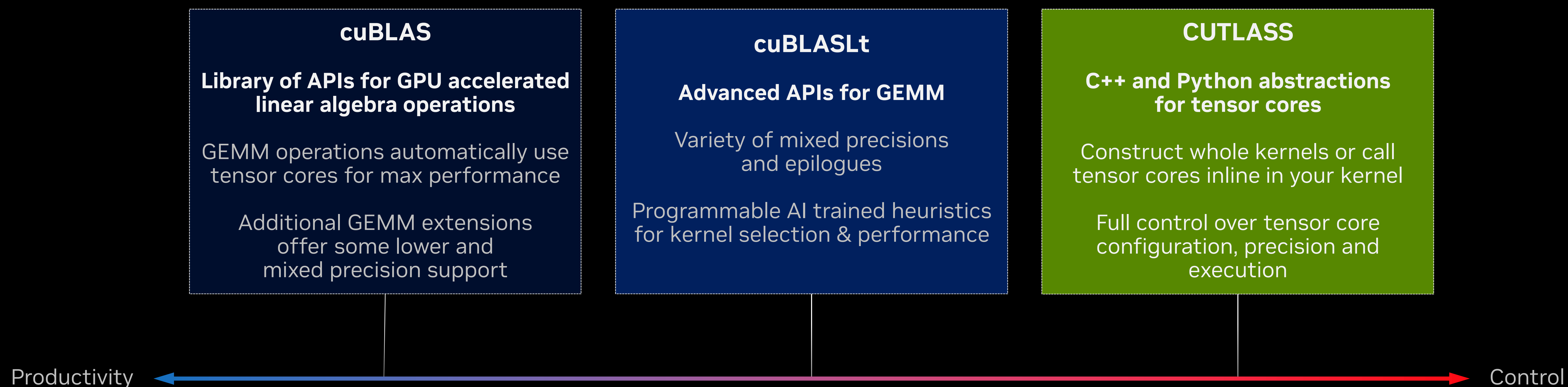


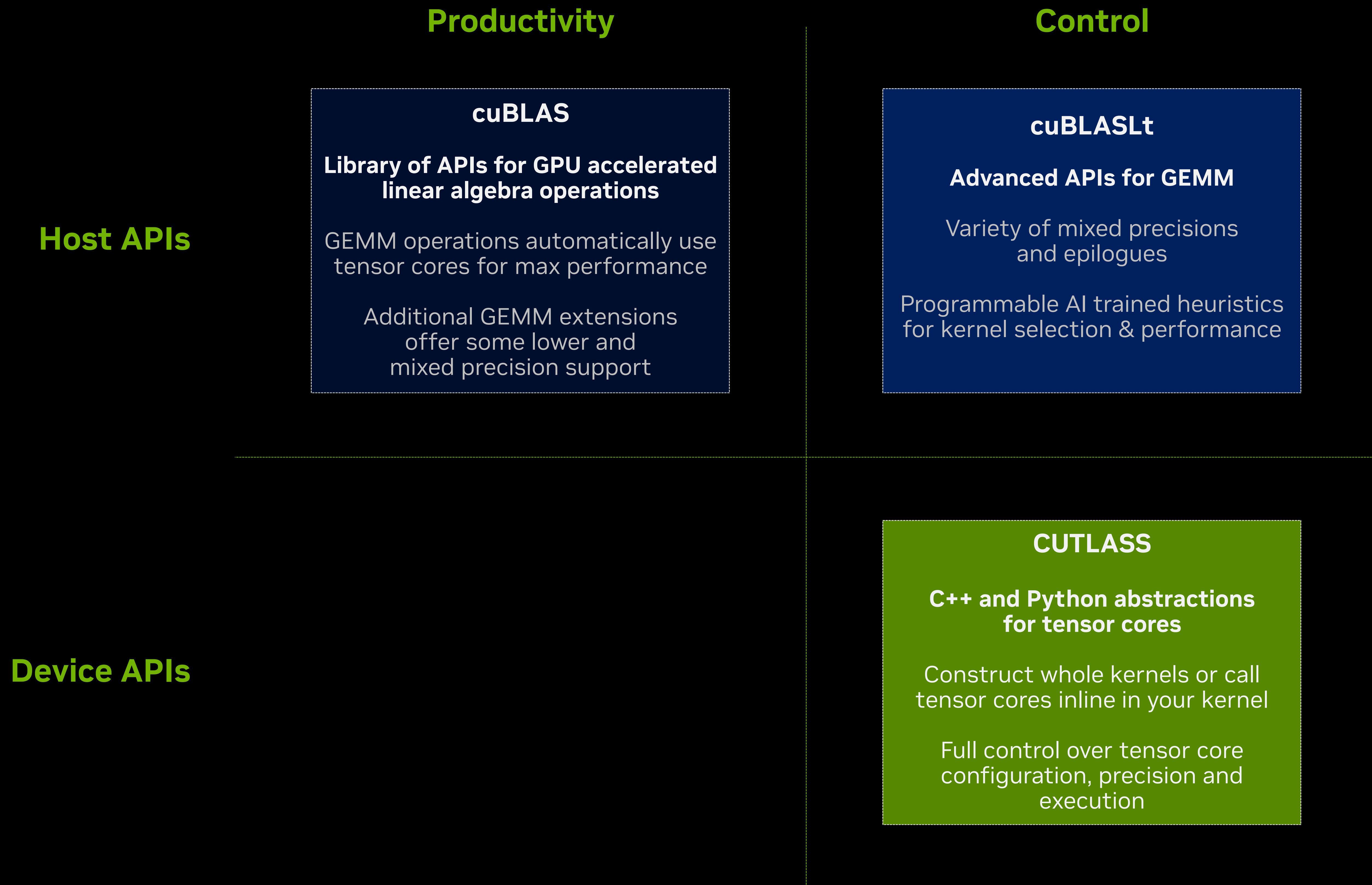
Based on the method of Ootomo et. al.^[2] to apply integer tensor cores to DGEMM, L40s achieves a 7x performance increase over native FP64 (and reaches 54% of A100 FP64 tensor core performance)

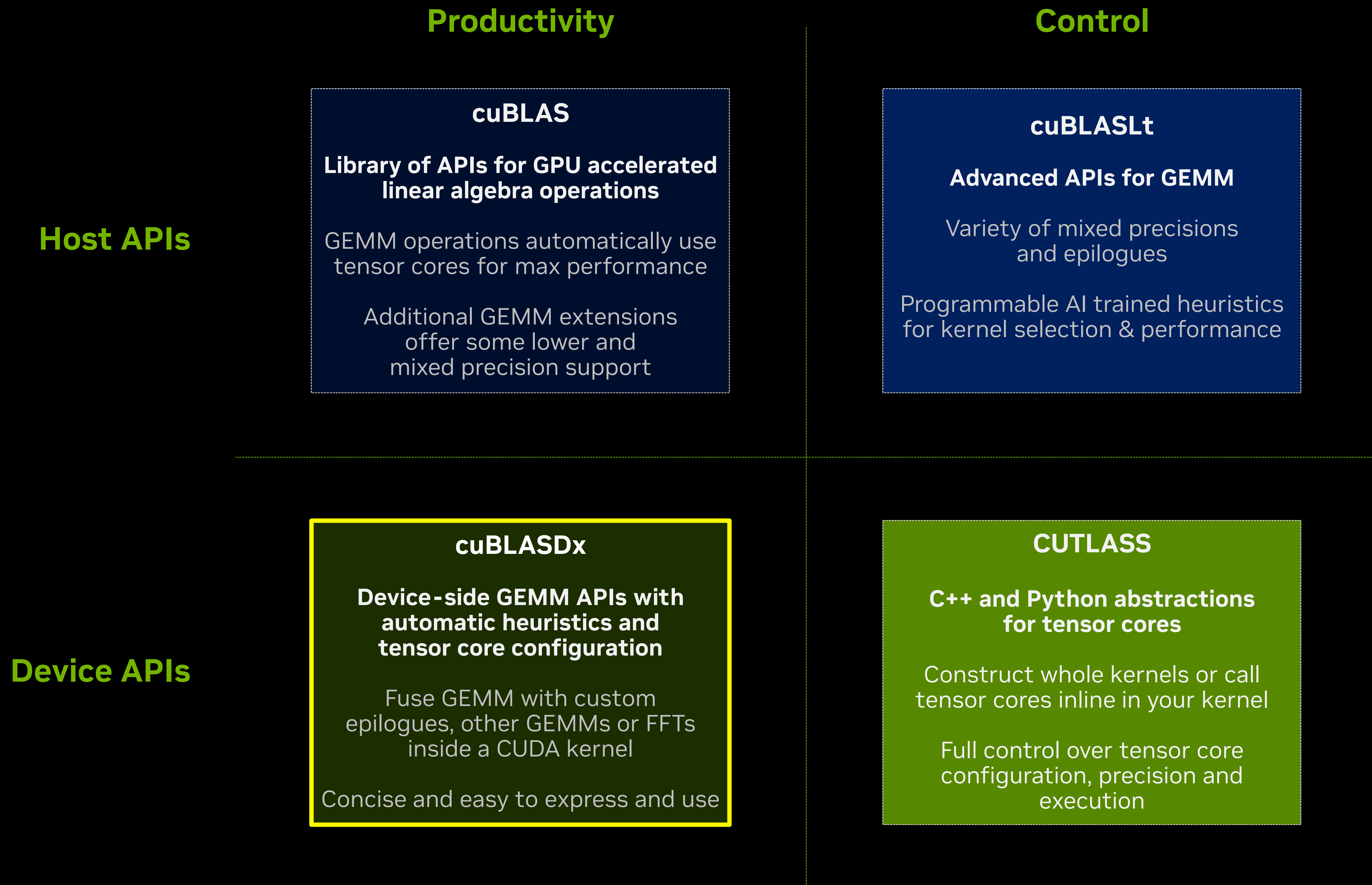
Power use is 8.6x more efficient than native FP64 MMA, although less efficient (0.7x) than A100's native tensor cores

[2] Ootomo Hiroyuki, Ozaki Katsuhisa, Yokota Rio, 2023, DGEMM in Integer Matrix Multiplication Unit
arXiv:2306.11975v3 [cs.DC] 28 Dec 2023

Tensor Core Programming Abstractions

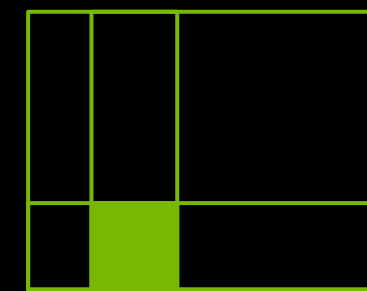






cuFFTDx, cuBLASDx: Math Device eXtensions for FFTs and MMA

Build-Your-Own FFT & MMA kernels



cuBLASDx

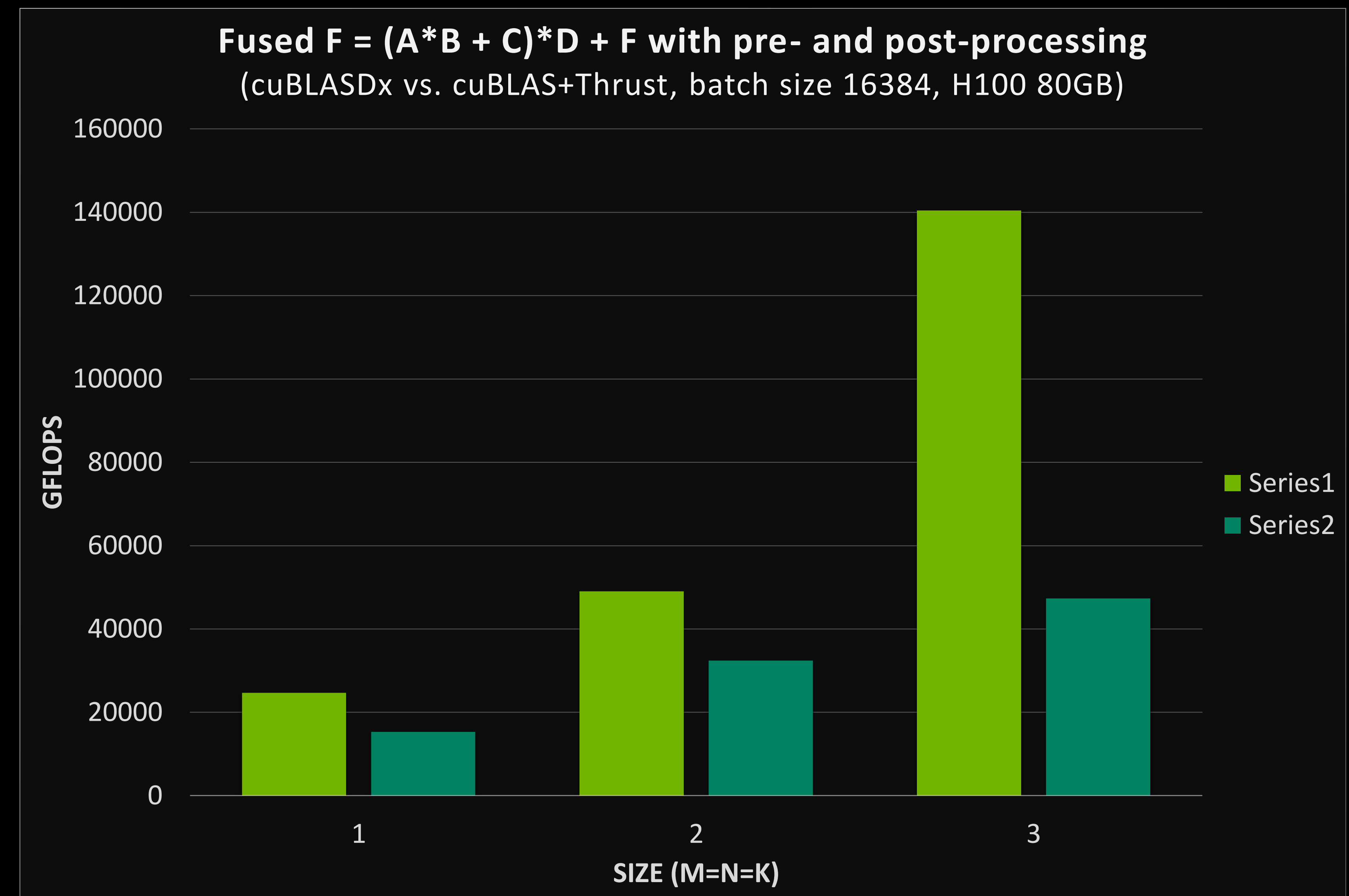
Tensor Core
accelerated
in-kernel GEMM

Device library of easy-to-use optimized, **tensor core accelerated matrix multiplication** (GEMM) inline in a kernel

Block-level GEMM operations

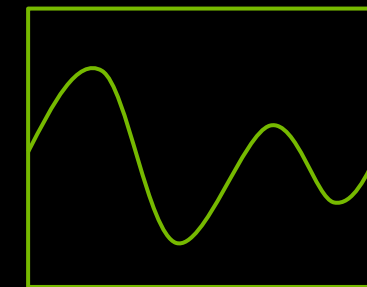
Automatic tensor core configuration: no knobs or heuristics, just specify GPU architecture

Inter-operates with other **Device Extension libraries**



cuFFTDx, cuBLASDx: Math Device eXtensions for FFTs and MMA

Build-Your-Own FFT & MMA kernels



cuFFTDx

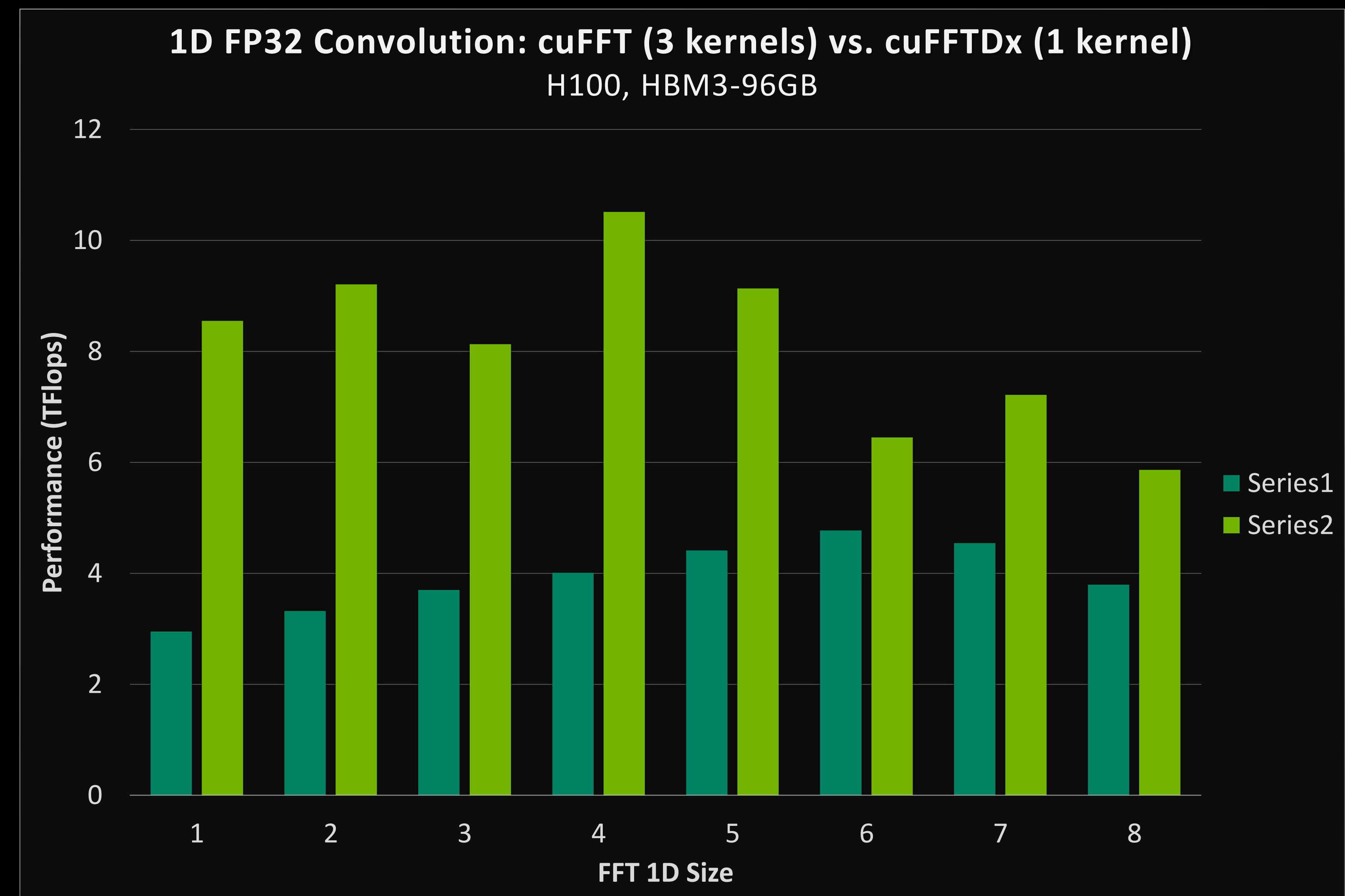
In-kernel
Fast Fourier
Transforms

C++, header-only library for **inlining performant FFT** computation into user kernels

Configure the FFT block to your kernel requirements

Best performance when data fits in shared memory / registers

Inter-operates with other **Device Extension libraries**

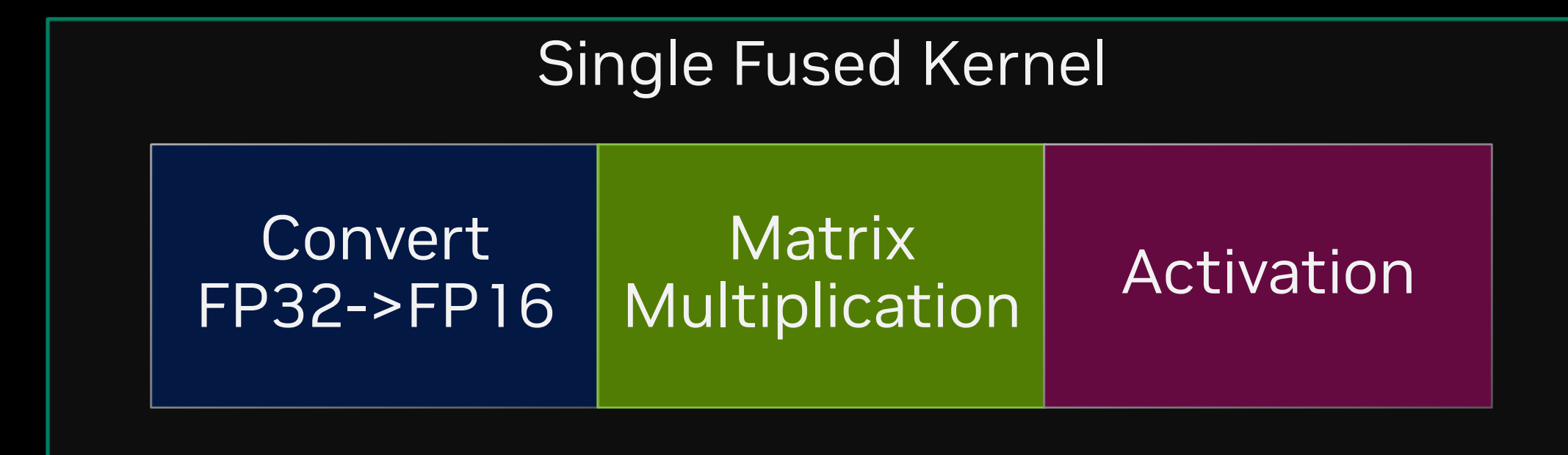


Basics of Kernel Fusion



End-to-end cost

3x kernel launch
3x loading of data from memory
1x { convert, MMA, activation }
3x storing of data to memory

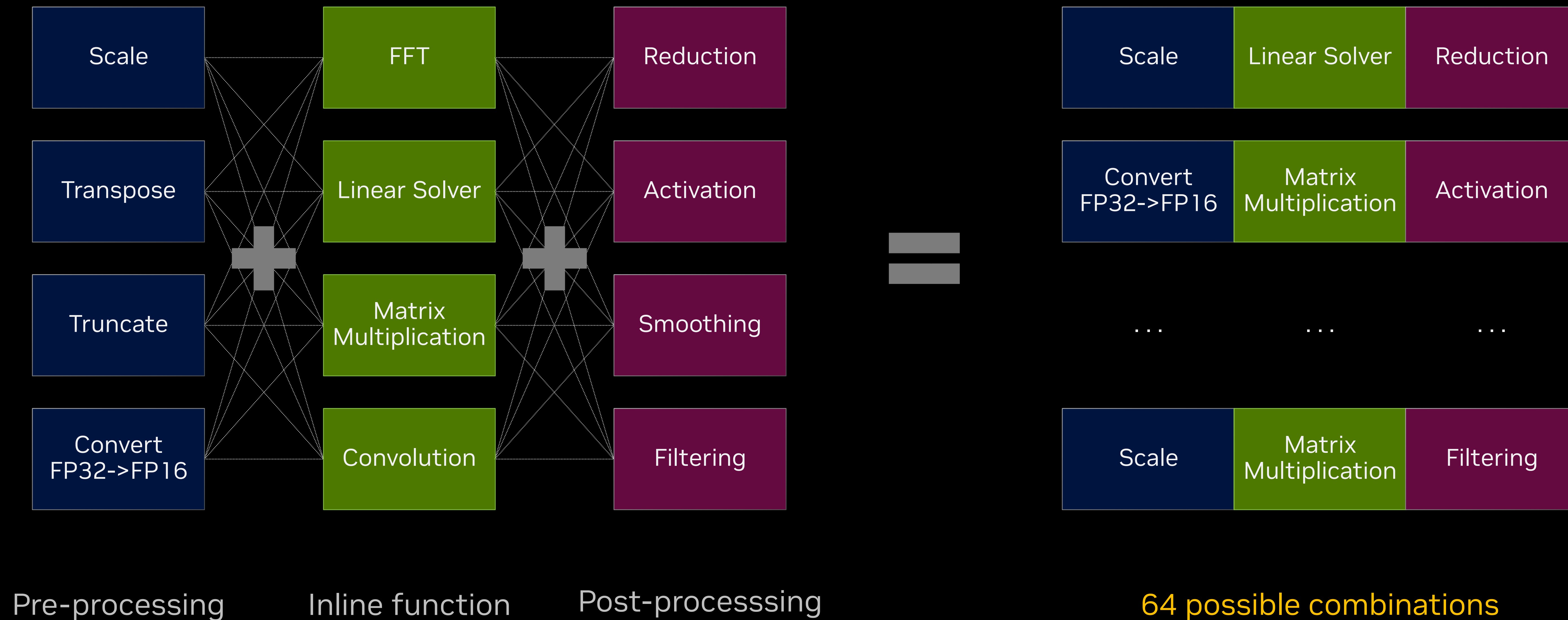


End-to-end cost

1x kernel launch
1x loading of data from memory
1x { convert, MMA, activation }
1x storing of data to memory

Kernel Fusion Can Lead To Specialization Explosion

Solution: Dynamically build fusions as required, using run-time kernel generation & JIT compilation

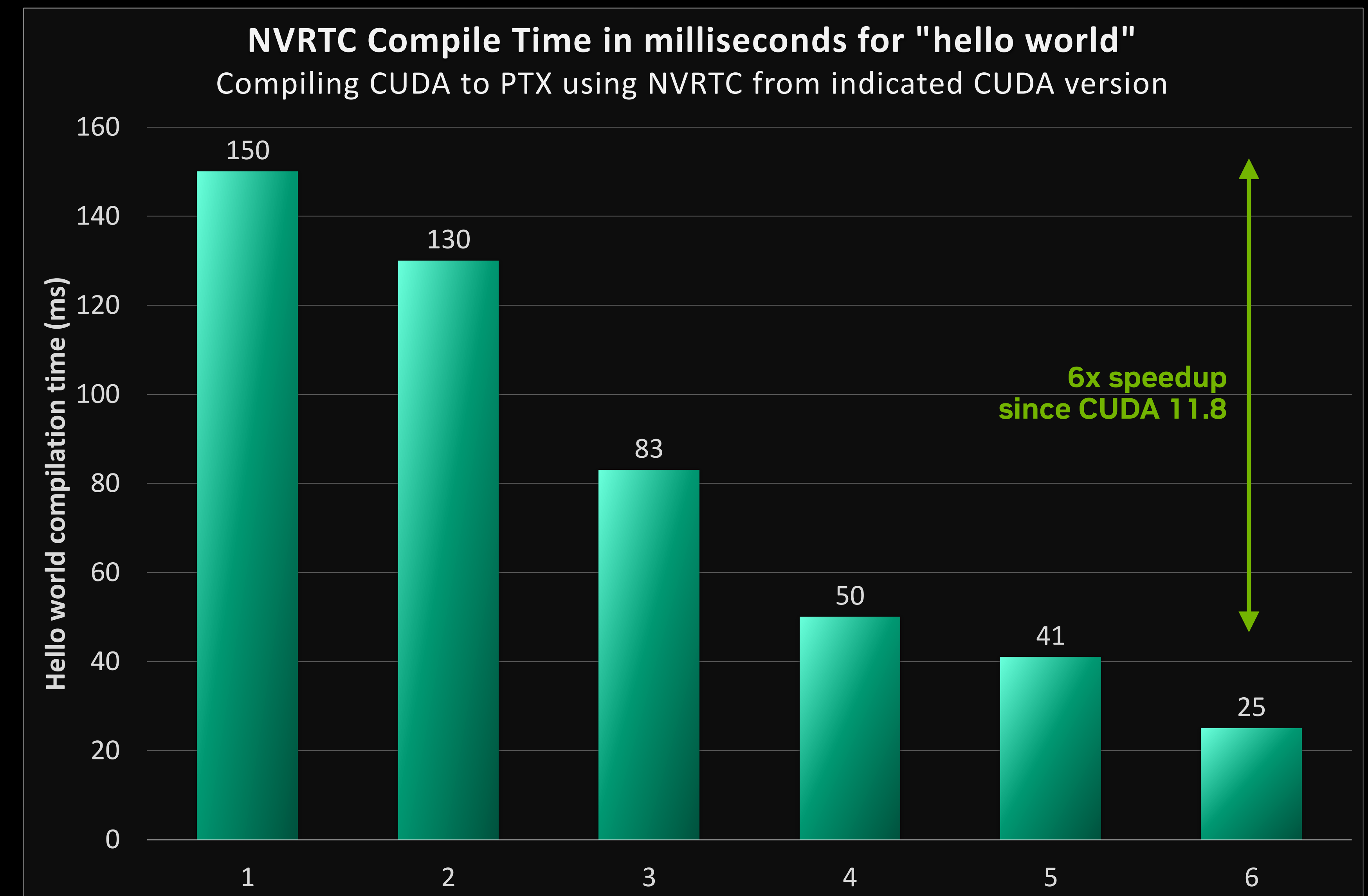
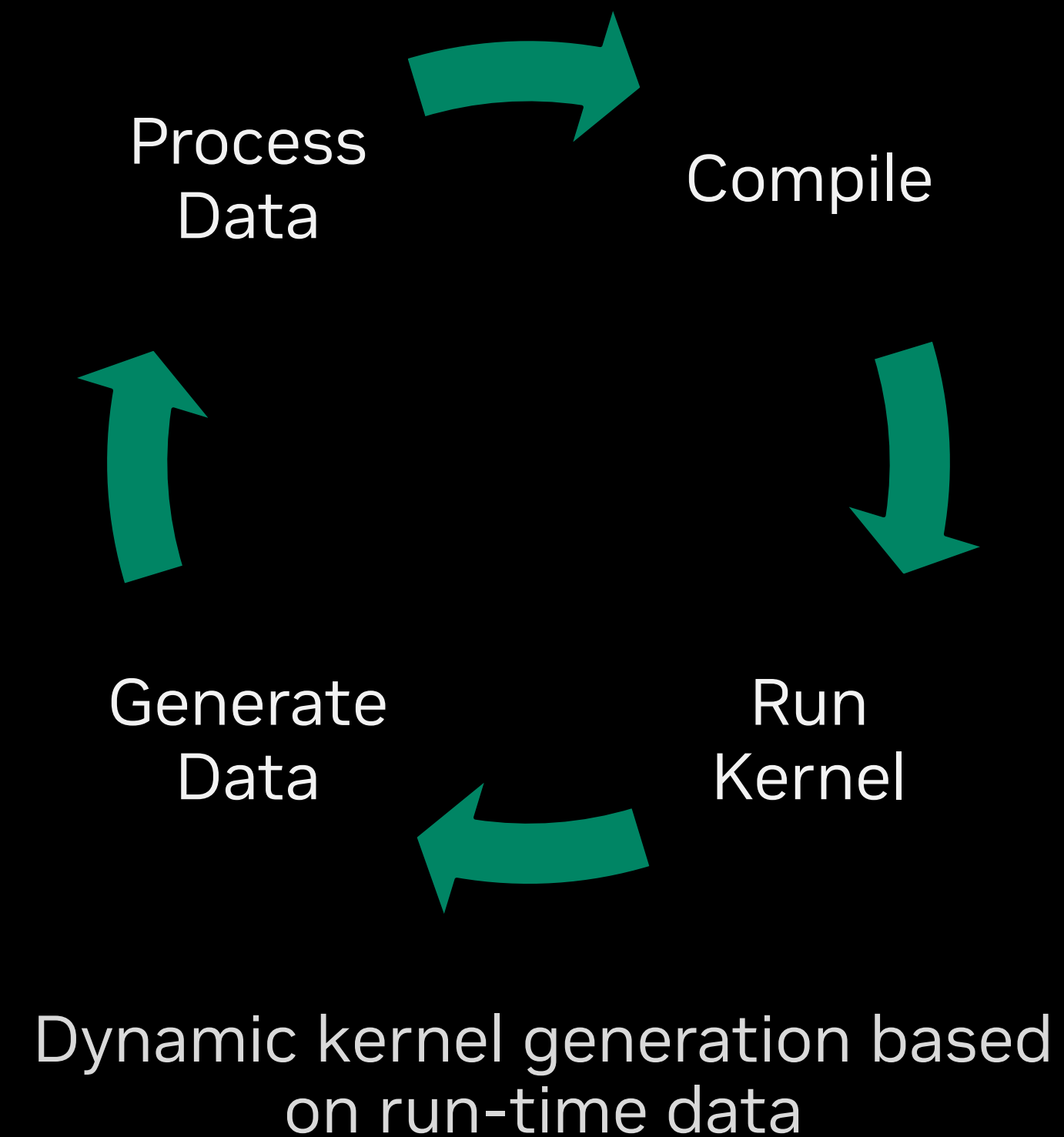


Runtime JIT Compilation Performance

Continuous investment in reducing compilation time, for both online and offline compilation

New “-minimal” flag reduces front-end compilation time by excluding rarely used compile-time-expensive features

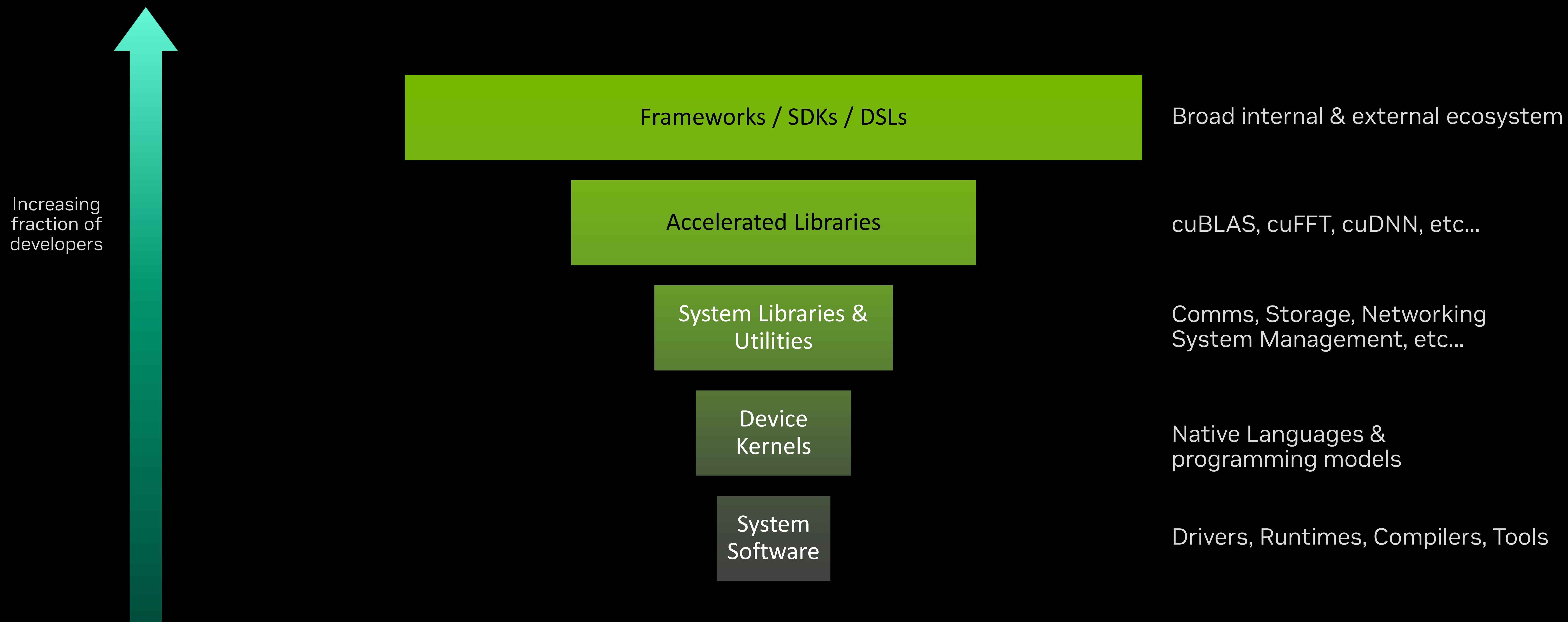
- Examples of such features include textures and surfaces
- 1.6x lower overheads than default “-O3”
- Ideal for programs which re-JIT kernels repeatedly (for example when targeting dynamic shapes in DNNs)



Benchmark system: Intel(R) Core(TM) i7 CPU 960 @ 3.20GHz, 16 GB RAM

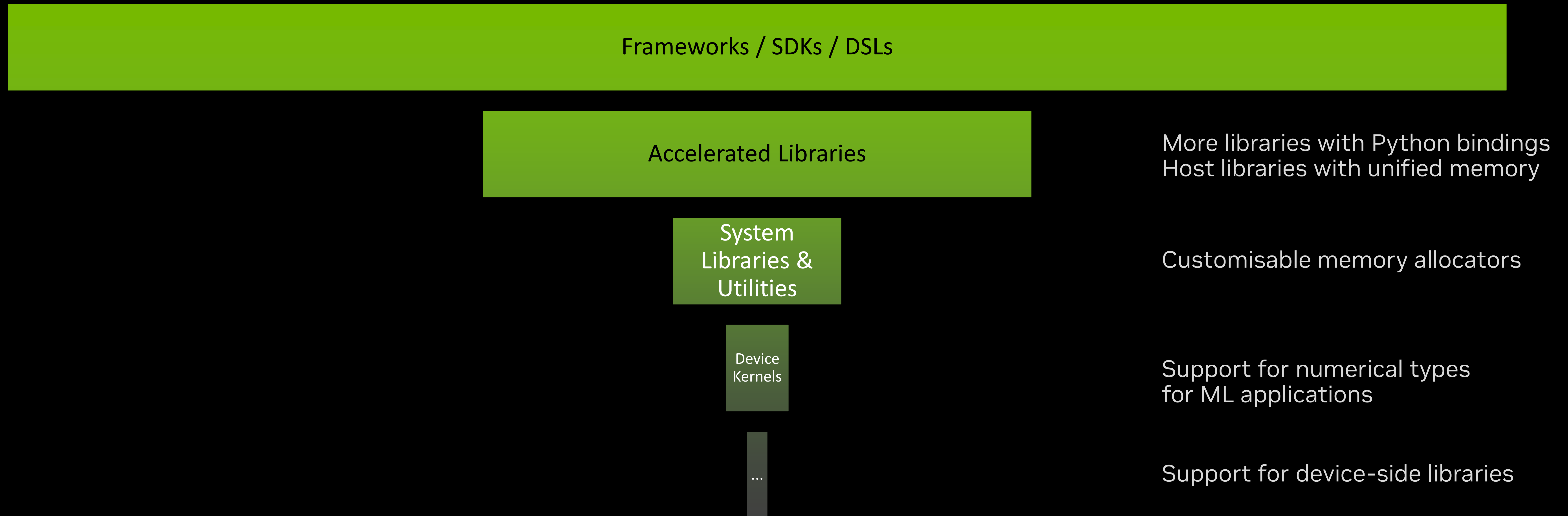
The Inverted CUDA Developer Pyramid

Low-level functions can have an outsized effect on higher-level development



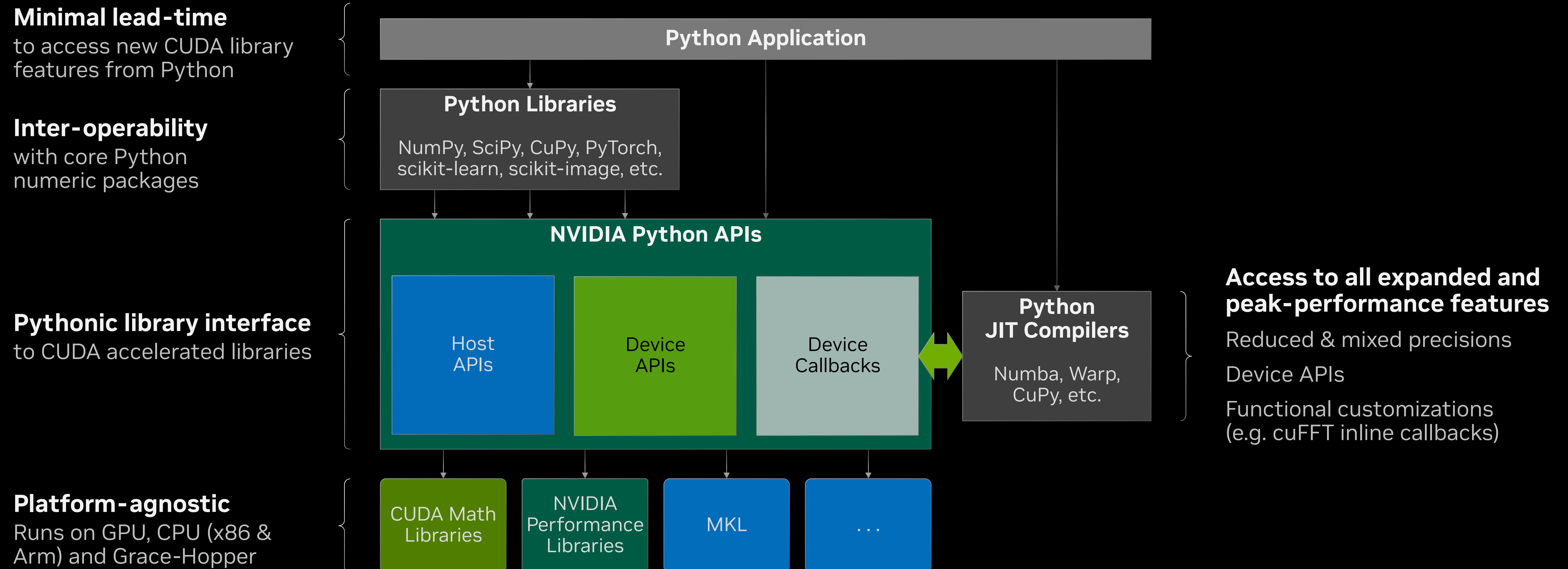
The (Much Wider) Inverted Python Developer Pyramid

Towards a complete NVIDIA experience for the Python developer to fully utilize the CUDA ecosystem



nvmath-python

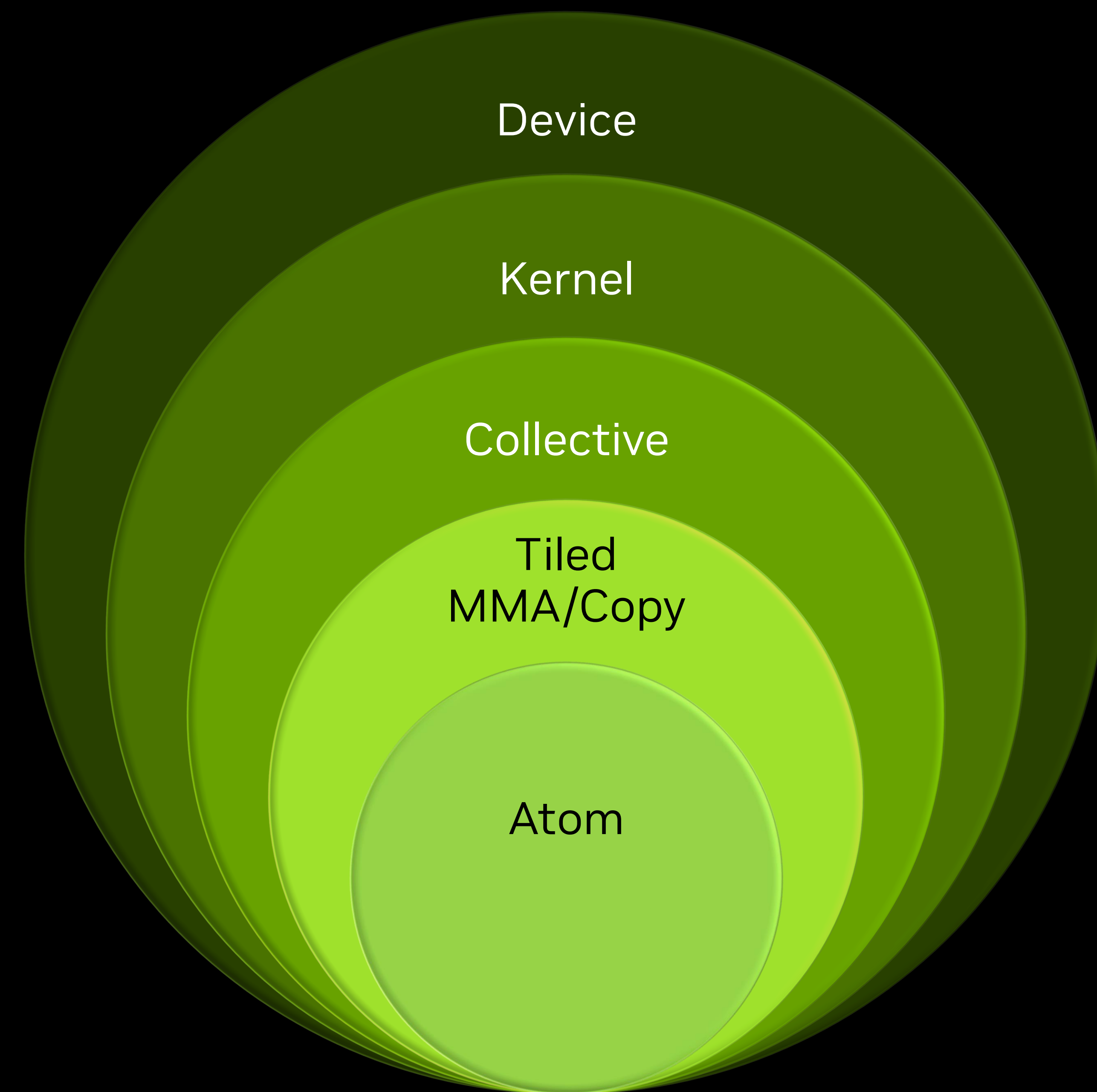
Easy Pythonic access to CUDA Math library functionality in a performant manner



[Deep Dive into Math Libraries \[S62162\]](#)

CUTLASS

Layered abstractions for programming Tensor Cores



Device Layer

Host side setup and interface

Kernel Layer

Launch API, grid planning logic, load balancing schedules, and kernel thread marshalling

Collective Layer

Temporal micro-kernels with arch specific synchronization that make up mainloops

Tiled MMA/Copy

Spatial micro-kernel interface for thread/value tilings and permutations.

Atom Layer

Architecture instructions associated meta-information

CUTLASS Python Interface

Easy integration into DL Frameworks (e.g., PyTorch)

C++ Interface

```
using Gemm =
typename gemm::kernel::DefaultGemmUniversal<
    half_t, layout::RowMajor,
    ComplexTransform::kNone, 8,
    half_t, layout::RowMajor,
    ComplexTransform::kNone, 8,
    half_t, layout::RowMajor,
    half_t, arch::OpClassTensorOp, arch::Sm80,
    gemm::GemmShape<256, 128, 64>,
    gemm::GemmShape<64, 64, 64>,
    gemm::GemmShape<16, 8, 16>,
    epilogue::thread::LinearCombination<
        half_t, 8, half_t, half_t>,
    gemm::threadblock::GemmIdentityThreadblockSwizzle<1>,
    3, arch::OpMultiplyAdd
>::GemmKernel;
```

Python Interface

```
// Create GEMM plan
plan = cutlass.op.Gemm(element=torch.float16,
                      layout=cutlass.LayoutType.RowMajor)

// Change swizzling functor
plan.swizzling_functor =
    cutlass.swizzle.ThreadblockSwizzleStreamK

// Add fused ReLU
plan.activation = cutlass.epilogue.relu
```

```
plan = cutlass.GroupedGemm(element=torch.float16,
                          layout=cutlass.LayoutType.RowMajor)

cutlass.emit.pytorch(plan.construct(), name='grouped_gemm',
                    cc=80, sourcedir='out')
```

out/setup.py

Python

out/grouped_gemm.cpp

C++

out/grouped_gemm.cu

CUDA

```
pip install nvidia-cutlass
```

```
# Import and run generated PyTorch extension
import grouped_gemm
Ds = grouped_gemm.run([A0, A1],
                    [B0, B1])
```


Python Developer Tools

Detailed inspection of Python-based GPU & CPU execution

Python Call Stacks Samples and CUDA API Backtrace

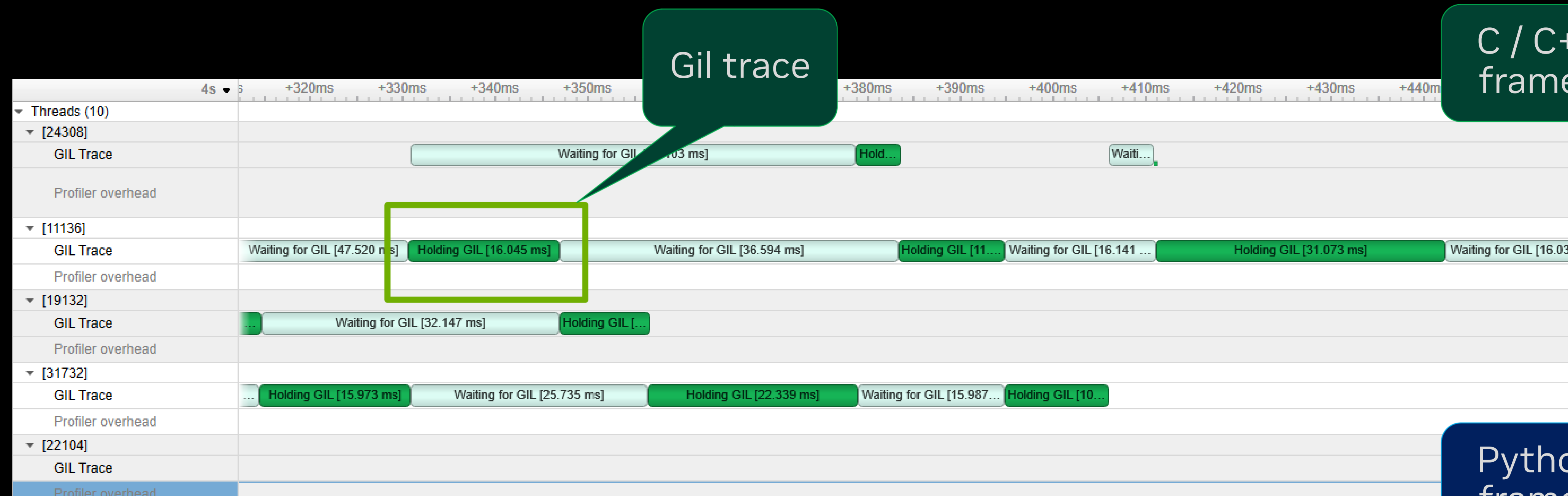
Identify where you are and how you got there

Global Interpreter Lock (GIL) trace

Common performance limiter in Python

Annotated dynamic code ranges

Built into in popular frameworks and libraries such as RAPIDS, Spark, CV-CUDA, and more...

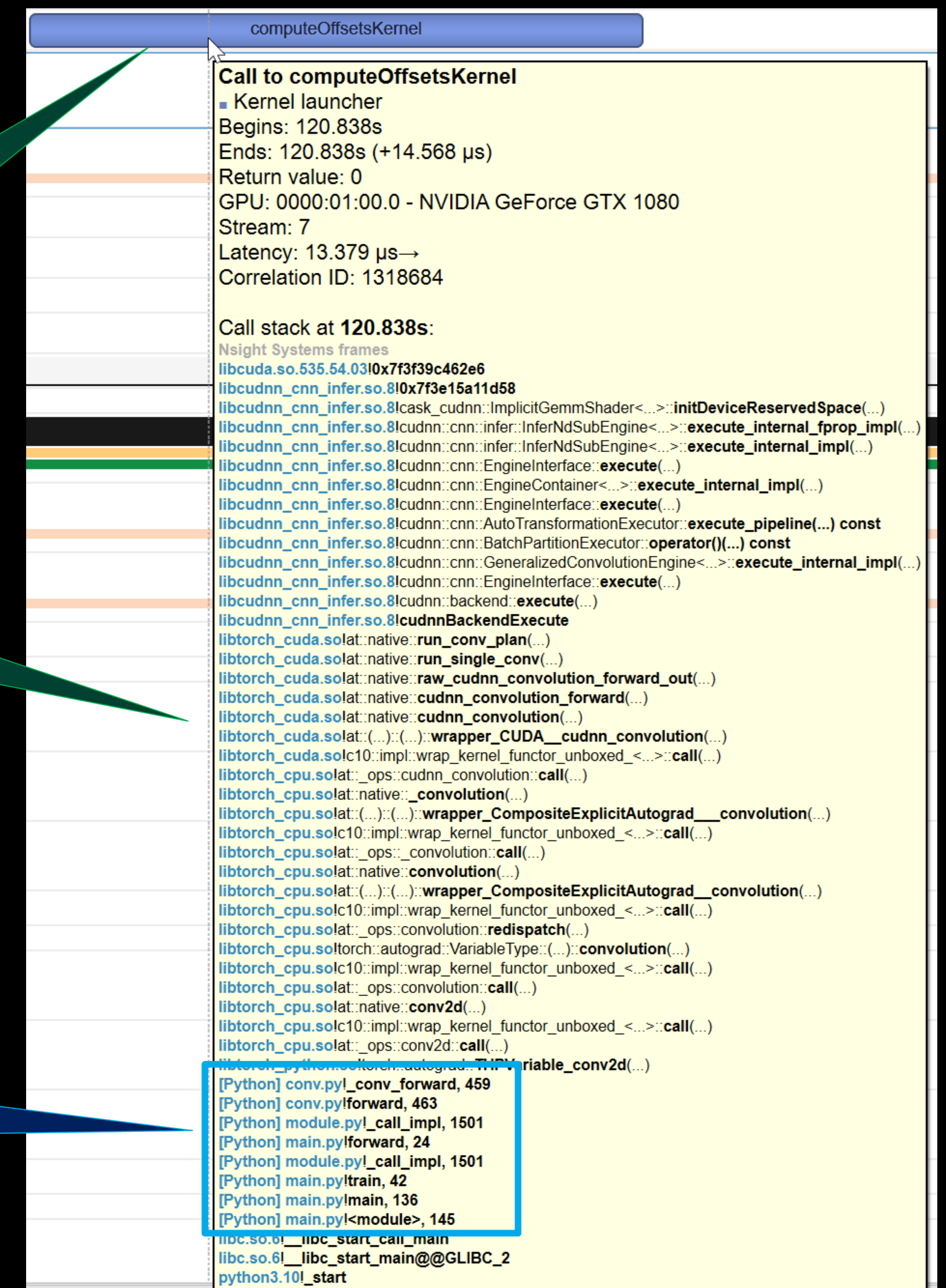


Gil trace

Timeline range for a CUDA API call

C / C++ frames

Python frames



Python Dynamic Profiling Annotations

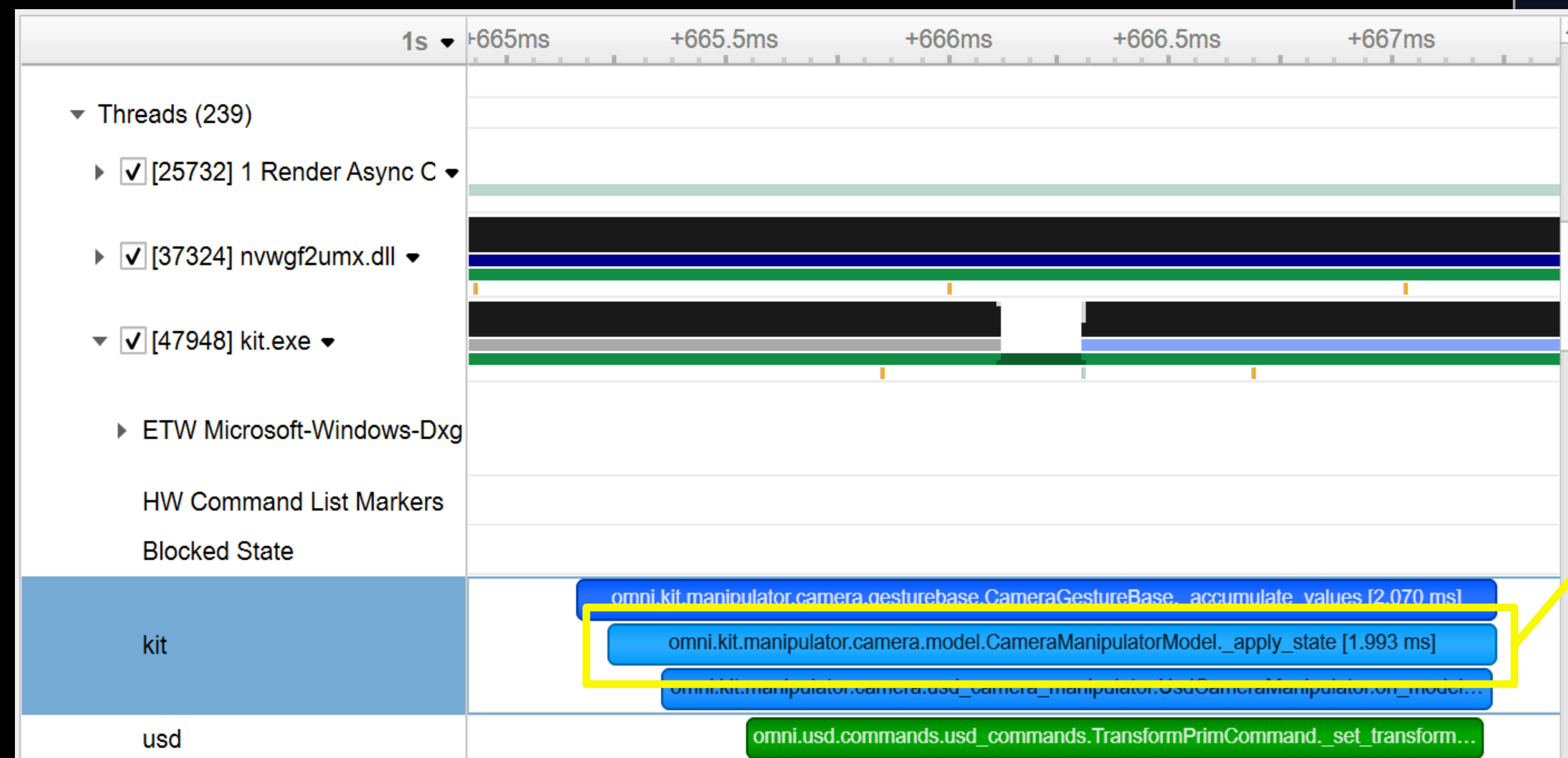
Target application must be launched by Nsight Systems

Requires no change to source code

Extremely low runtime overhead

Config file in JSON format details

- identify functions to annotate
- provide NVTX placement info



```
[  
  {  
    "domain": "usd",  
    "color": "0x008000",  
    "module": "omni.usd.commands.usd_commands",  
    "functions": ["TransformPrimCommand._set_transform_matrix"]  
  },  
  {  
    "domain": "kit",  
    "module": "omni.kit.manipulator.camera.usd_camera_manipulator",  
    "color": "0x0080FF",  
    "functions": ["UsdCameraManipulator.on_model_updated"]  
  },  
  {  
    "domain": "kit",  
    "module": "omni.kit.manipulator.camera.gesturebase",  
    "color": "0x0055FF",  
    "functions": ["CameraGestureBase._accumulate_values"]  
  },  
  {  
    "domain": "kit",  
    "module": "omni.kit.manipulator.camera.model",  
    "color": "0x009CFF",  
    "functions": ["CameraManipulatorModel._apply_state"]  
  }  
]
```

Configure function highlighting in Nsight Systems timeline

Warp: Differentiable Spatial Computing for Python

GPU Kernels in Python

Write CUDA kernels in 100% Python syntax
Runtime JIT compilation
Fast developer iteration

Spatial Computing

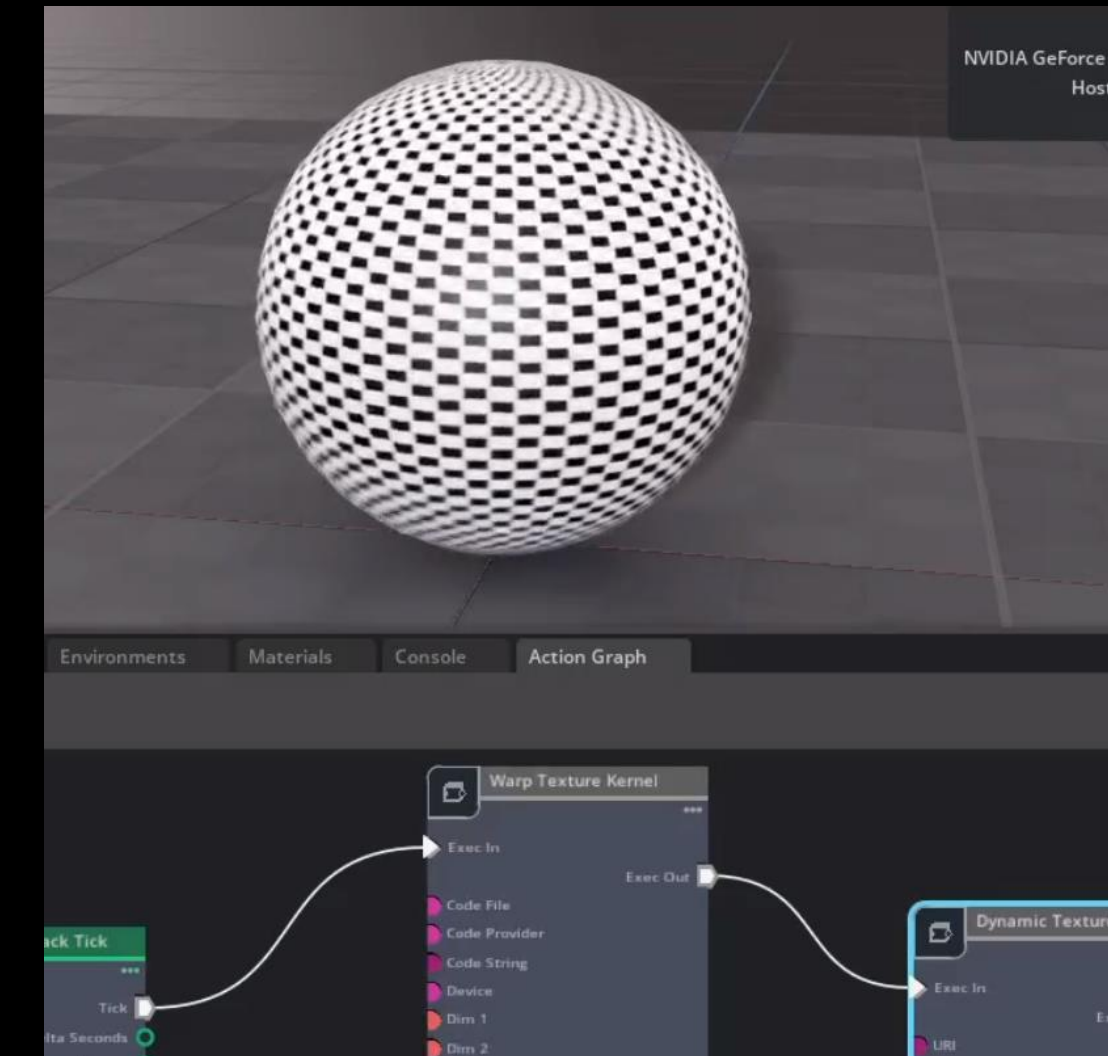
Rich vector math library
Mesh processing and queries
Sparse volumes (OpenVDB)
Hash grids

Differentiable programming

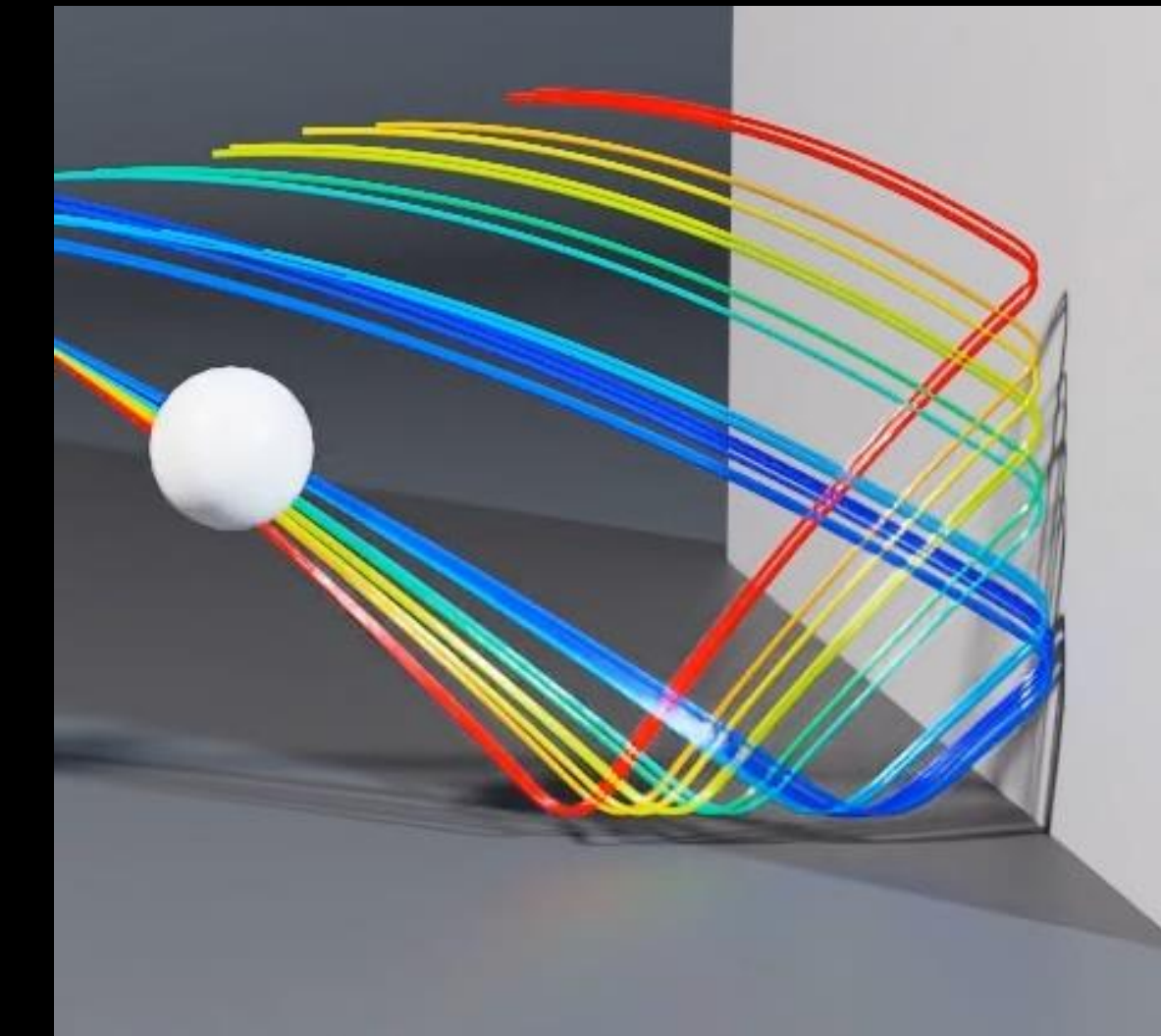
Auto-differentiation
Forward + backward kernel generation
Interop with DL frameworks (e.g.: Torch, JAX)

Omniverse integration

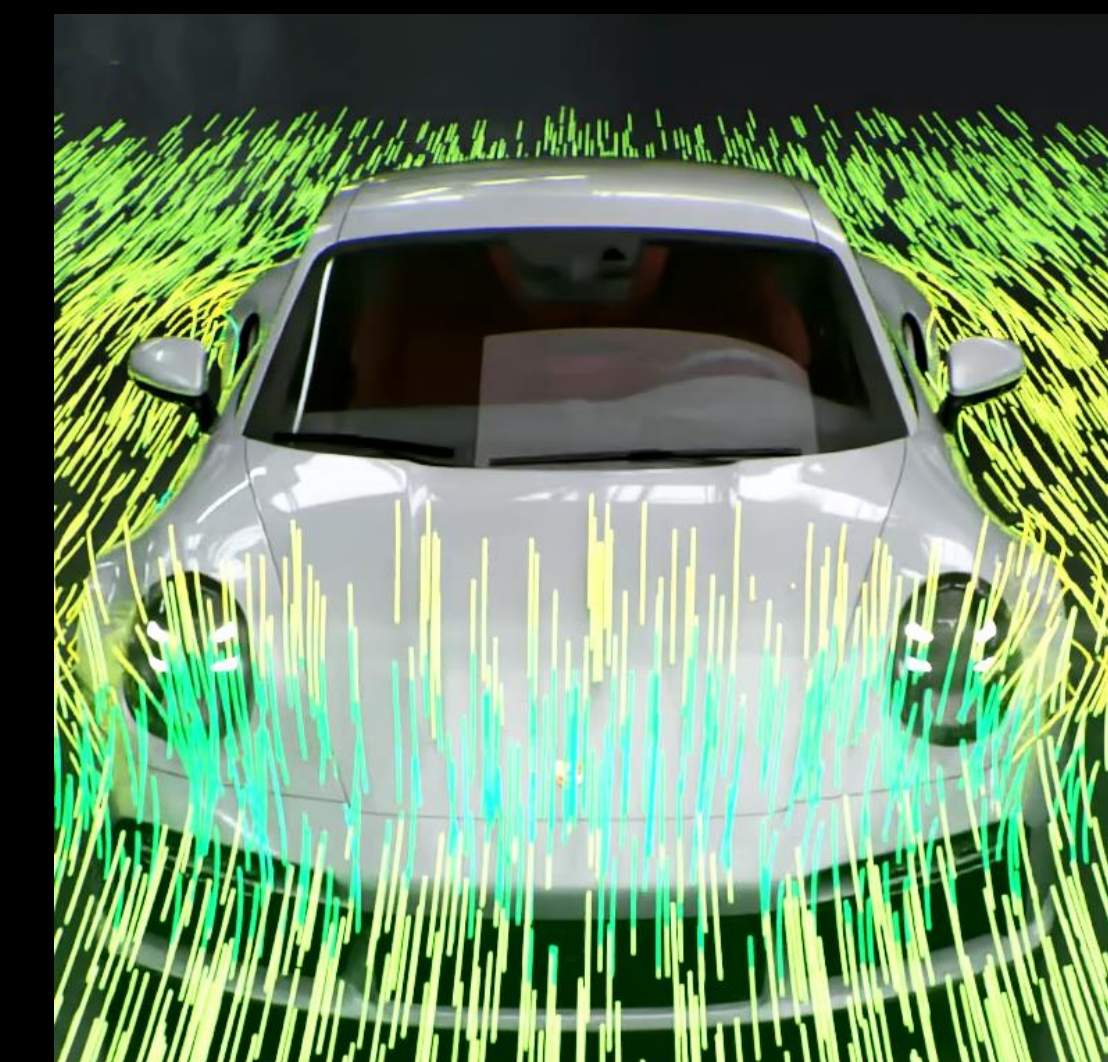
USD import/export
Runtime extensions for Isaac/USD Composer



Data Processing



System Training



Simulation

```
@wp.kernel
def initialize_particles(
    particle_x: wp.array(dtype=wp.vec3), sm
):
    tid = wp.tid()

    # grid size
    nr_x = wp.int32(width / 4.0 / smoothing)
    nr_y = wp.int32(height / smoothing)
    nr_z = wp.int32(length / 4.0 / smoothing)

    # calculate particle position
    z = wp.float(tid % nr_z)
    y = wp.float((tid // nr_z) % nr_y)
    x = wp.float((tid // (nr_z * nr_y)) % nr_x)
    pos = smoothing * wp.vec3(x, y, z)
```

Scripting

Fast JIT Compilation of Differentiable Python Kernels

Just-in-time reverse-mode automatic differentiation

```
import warp as wp

@wp.kernel
def integrate(p: wp.array(dtype=wp.vec3),
             v: wp.array(dtype=wp.vec3),
             f: wp.array(dtype=wp.vec3),
             m: wp.array(dtype=float)):

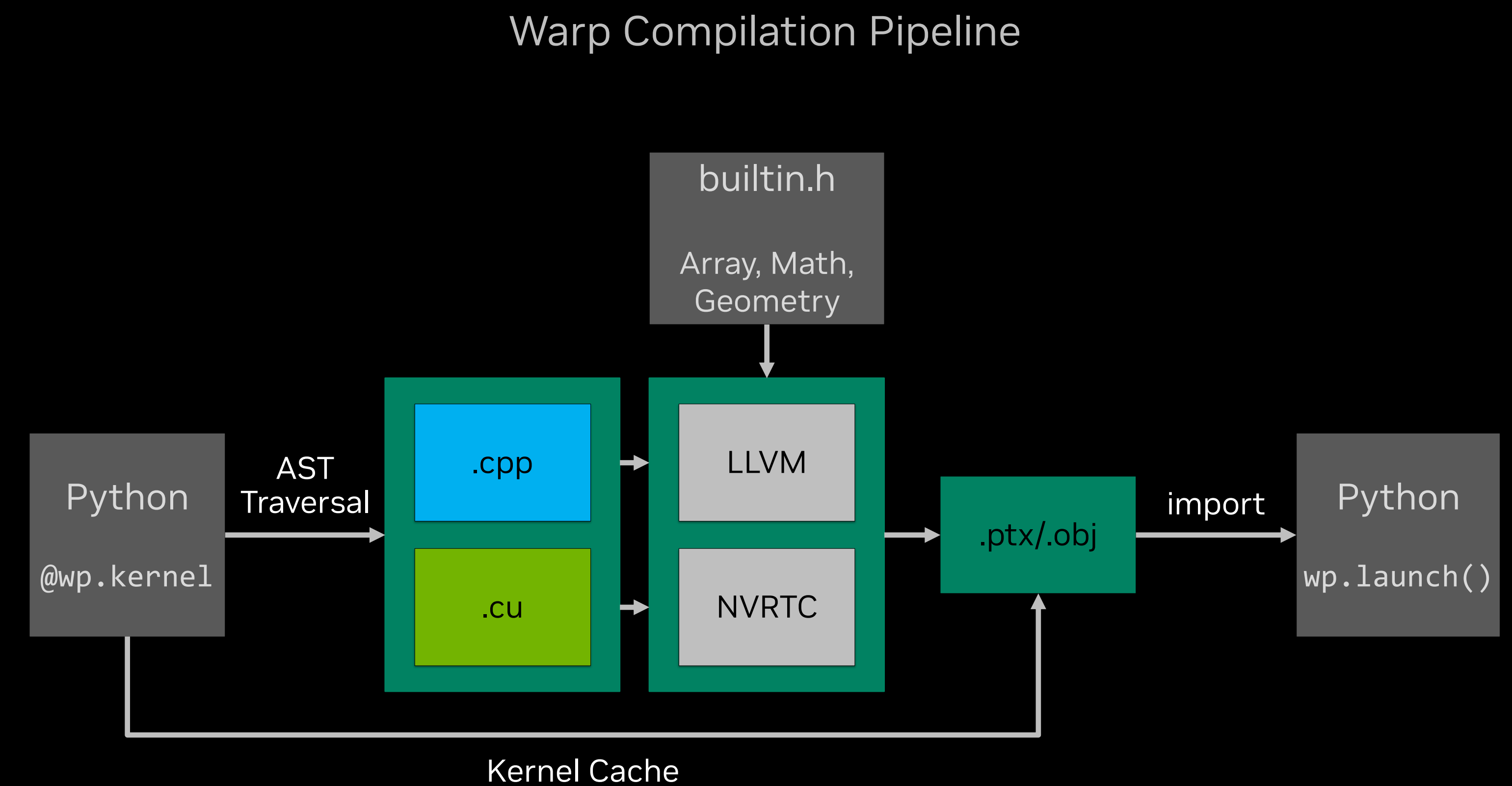
    # thread id
    tid = wp.tid()

    # Semi-implicit Euler step
    v[tid] = v[tid] + (f[tid] * m[tid] + wp.vec3(0.0, -9.8, 0.0)) * dt
    x[tid] = x[tid] + v[tid] * dt

# kernel launch
wp.launch(integrate, dim=1024, inputs=[x, v, f, ...], device="cuda:0")
```

Get Warp at: <https://github.com/nvidia/warp>

or: `pip install warp-lang`



Warp Example: Neural Stress Fields

Reduced-Order Elasto-plasticity and Fracture

Goal: Accelerated MPM solvers for elastoplastic simulation

Created **differentiable MPM solver** in **Warp**

Learn a reduced **neural basis** to represent stress fields

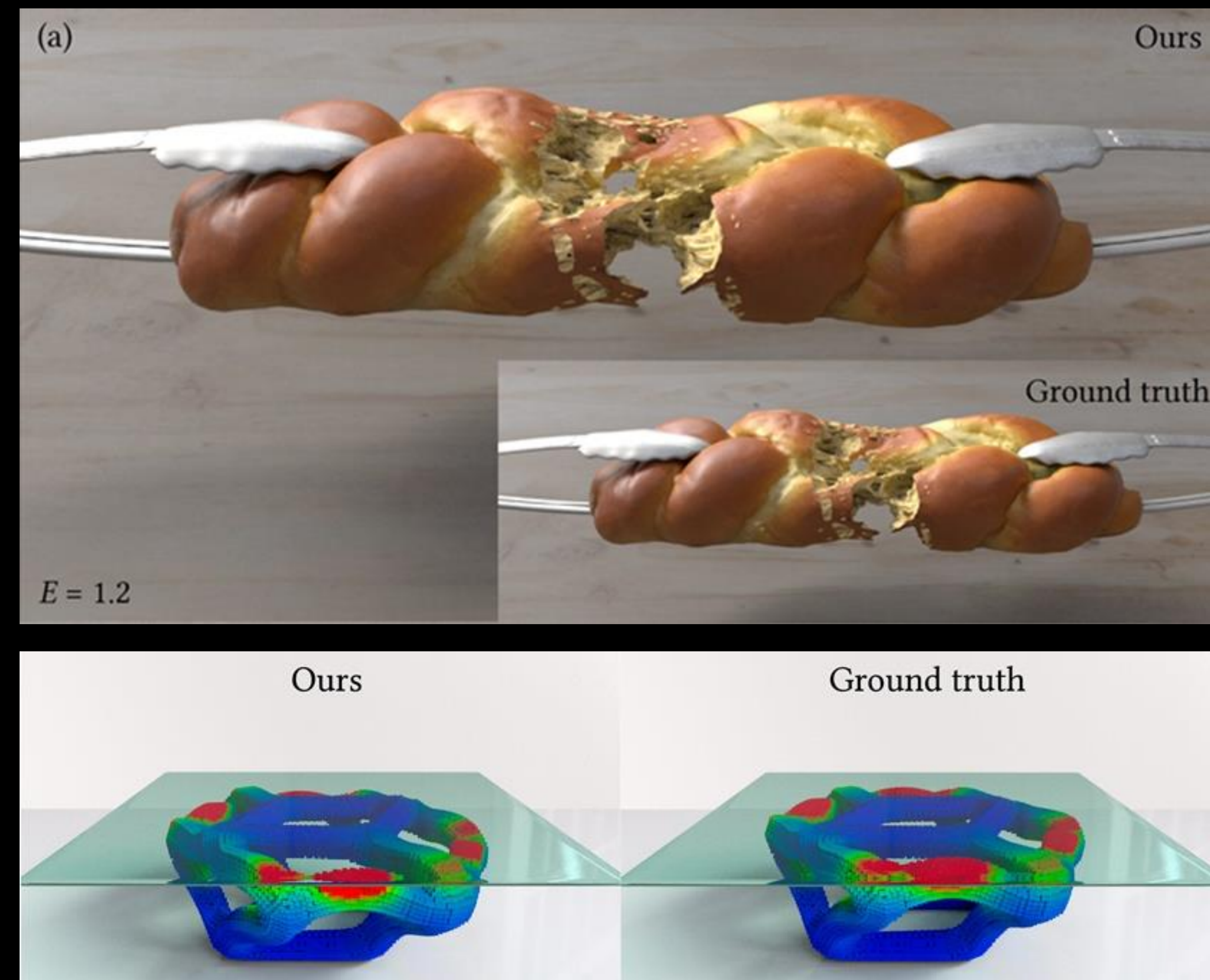
Dimension reduction of up to 100,000x

10x faster simulation

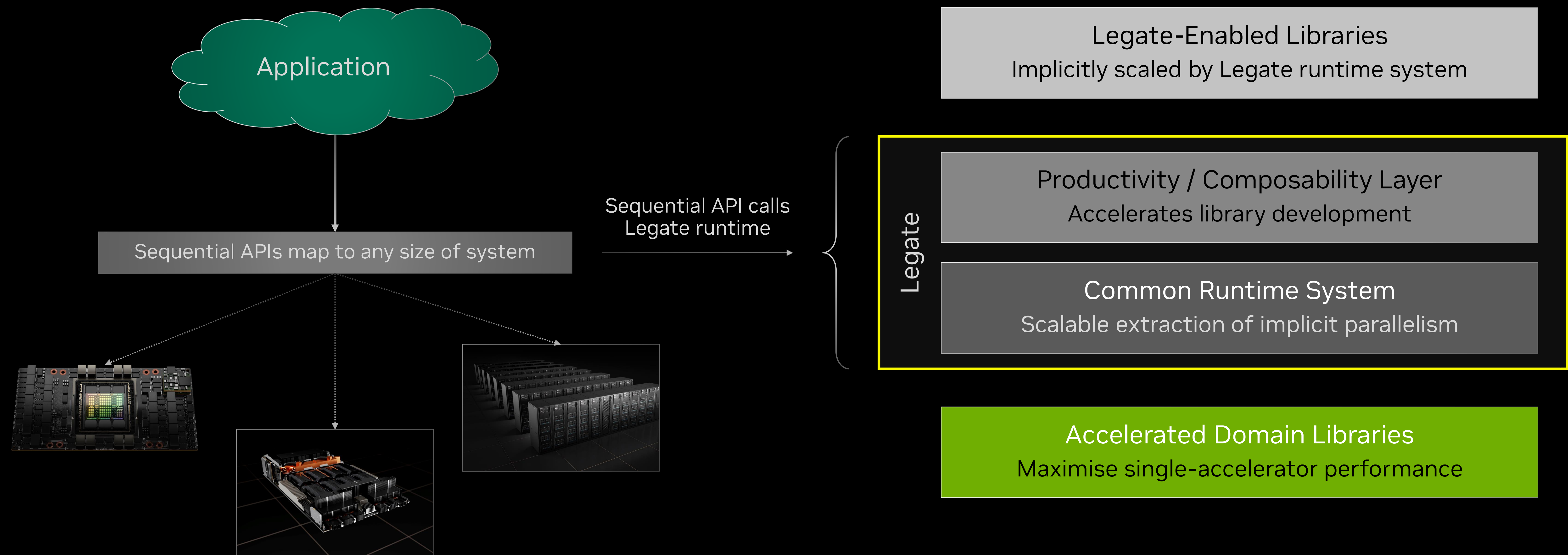
Generalizes to a broad range of materials

Joint work from UCLA, CMU, MIT, UToT

<https://zeshunzong.github.io/reduced-order-mpm/>



Legate: A Framework for Implicit Parallelism

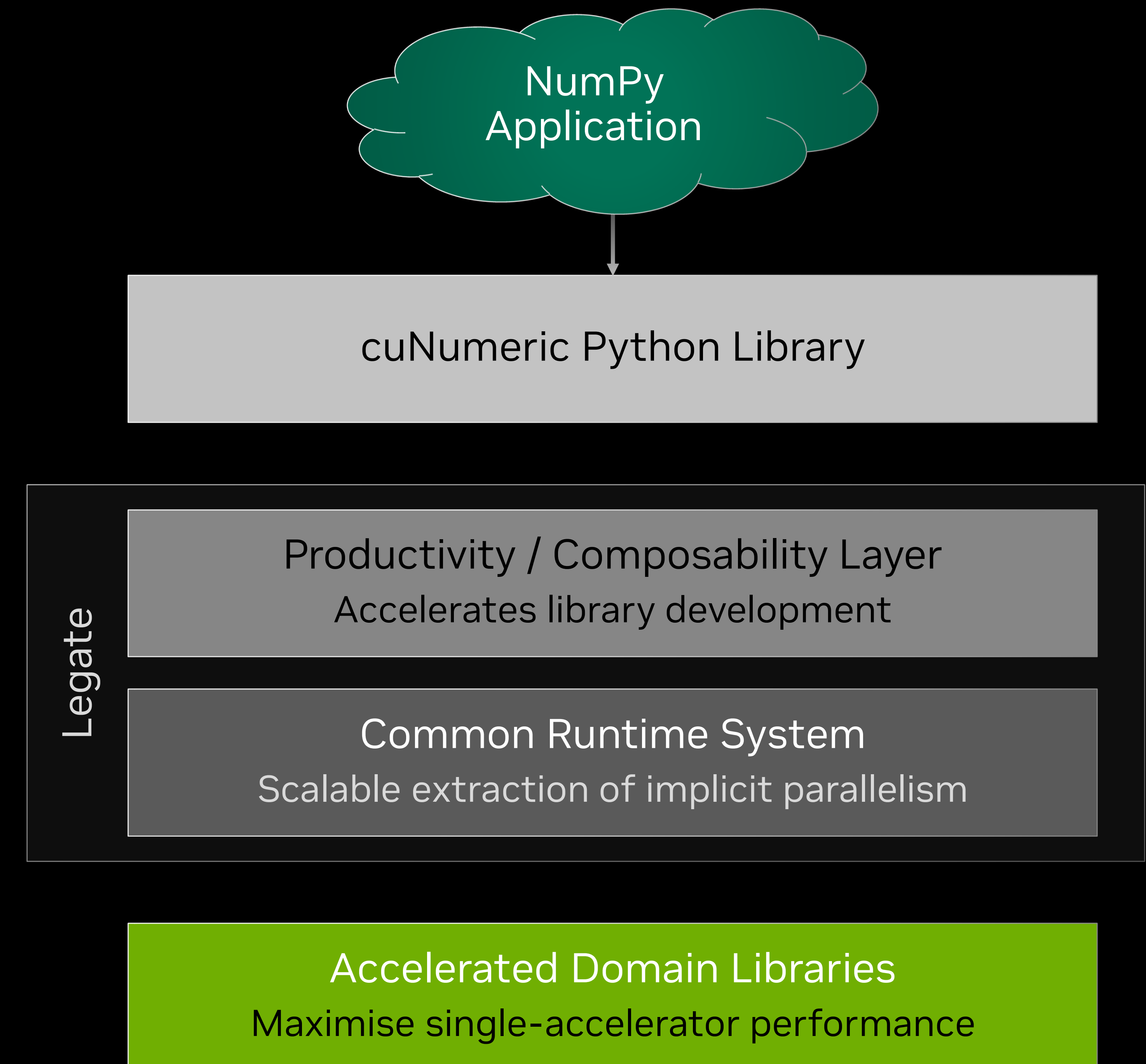


cuNumeric – Implicitly Parallel Implementations of NumPy APIs

Stencil Benchmark

No modifications required to scale to a thousand GPUs

```
32
33 def run_stencil(N, I, warmup, timing): # noqa: E741
34     grid = initialize(N)
35
36     print("Running Jacobi stencil...")
37     center = grid[1:-1, 1:-1]
38     north = grid[0:-2, 1:-1]
39     east = grid[1:-1, 2:]
40     west = grid[1:-1, 0:-2]
41     south = grid[2:, 1:-1]
42
43     timer.start()
44     for i in range(I + warmup):
45         if i == warmup:
46             timer.start()
47             average = center + north + east + west + south
48             work = 0.2 * average
49             center[:] = work
50     total = timer.stop()
51
52     if timing:
53         print(f"Elapsed Time: {total} ms")
54     return total
```



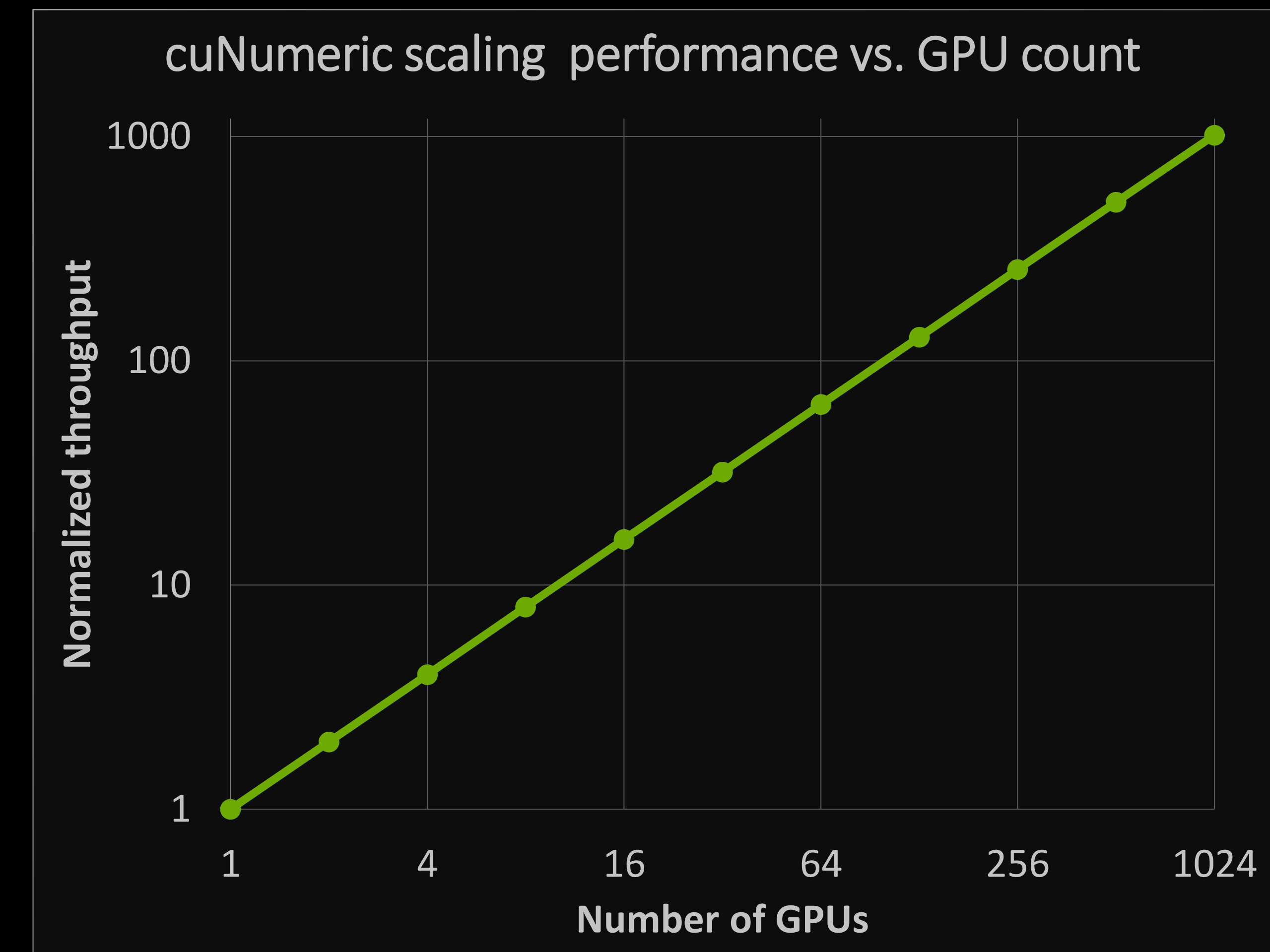
cuNumeric – Implicitly Parallel Implementations of NumPy APIs

Stencil Benchmark

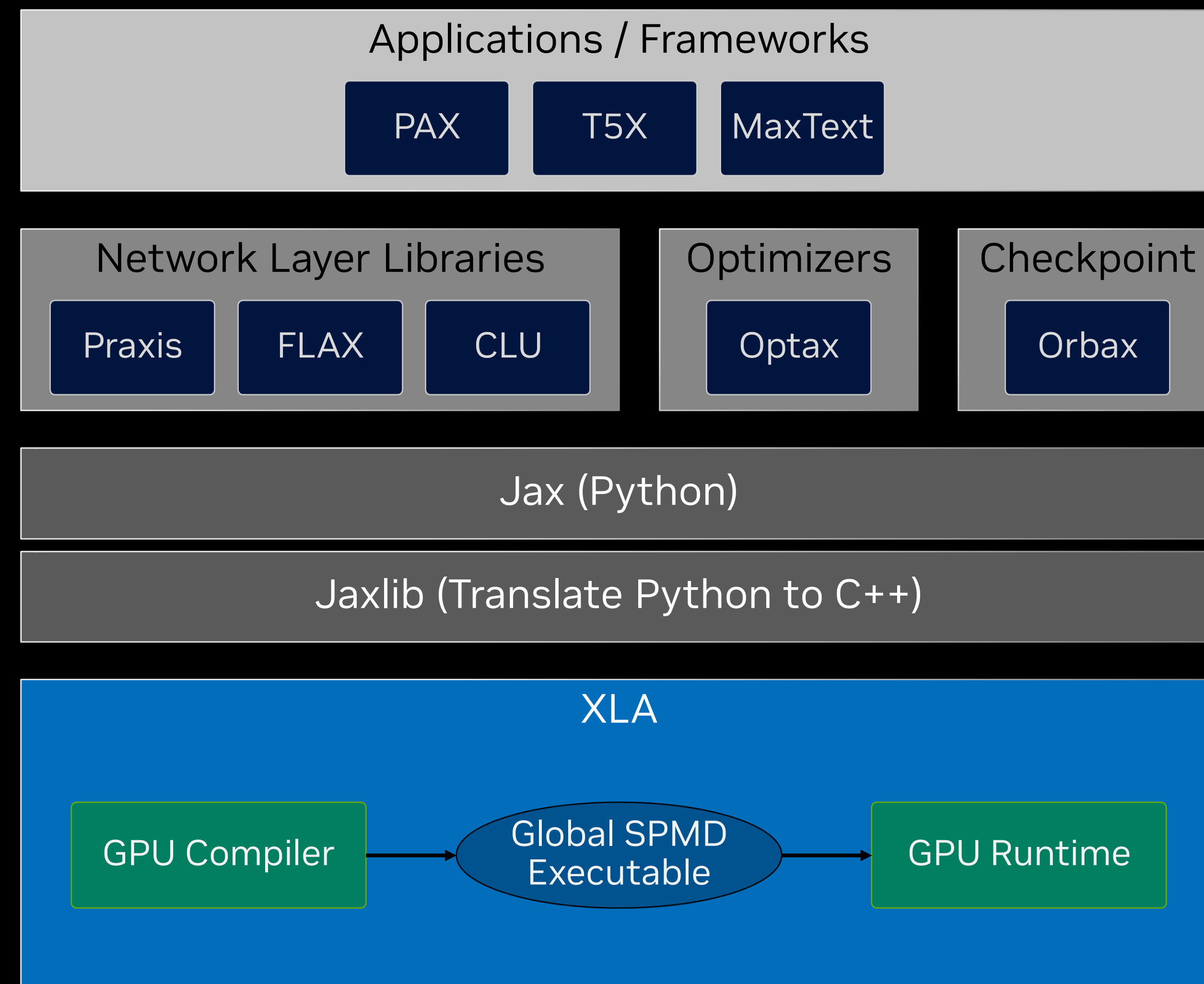
No modifications required to scale to a thousand GPUs

```
32
33 def run_stencil(N, I, warmup, timing): # noqa: E741
34     grid = initialize(N)
35
36     print("Running Jacobi stencil...")
37     center = grid[1:-1, 1:-1]
38     north = grid[0:-2, 1:-1]
39     east = grid[1:-1, 2:]
40     west = grid[1:-1, 0:-2]
41     south = grid[2:, 1:-1]
42
43     timer.start()
44     for i in range(I + warmup):
45         if i == warmup:
46             timer.start()
47             average = center + north + east + west + south
48             work = 0.2 * average
49             center[:] = work
50     total = timer.stop()
51
52     if timing:
53         print(f"Elapsed Time: {total} ms")
54     return total
```

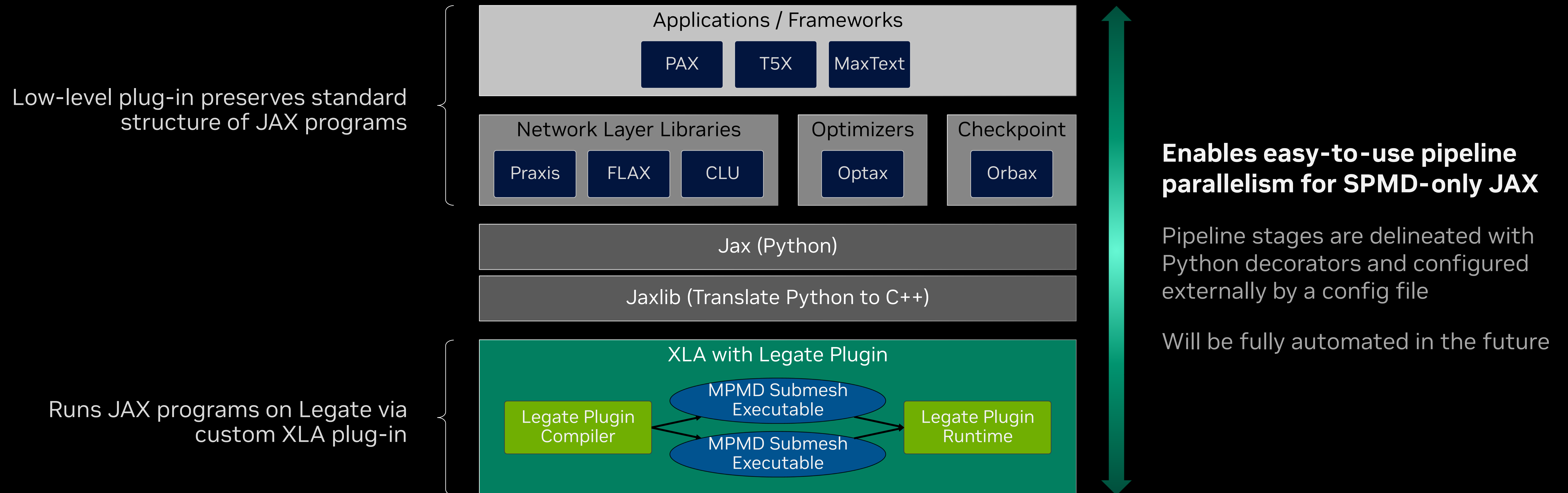
NumPy
Application



The Jax Framework for Differentiable Computing

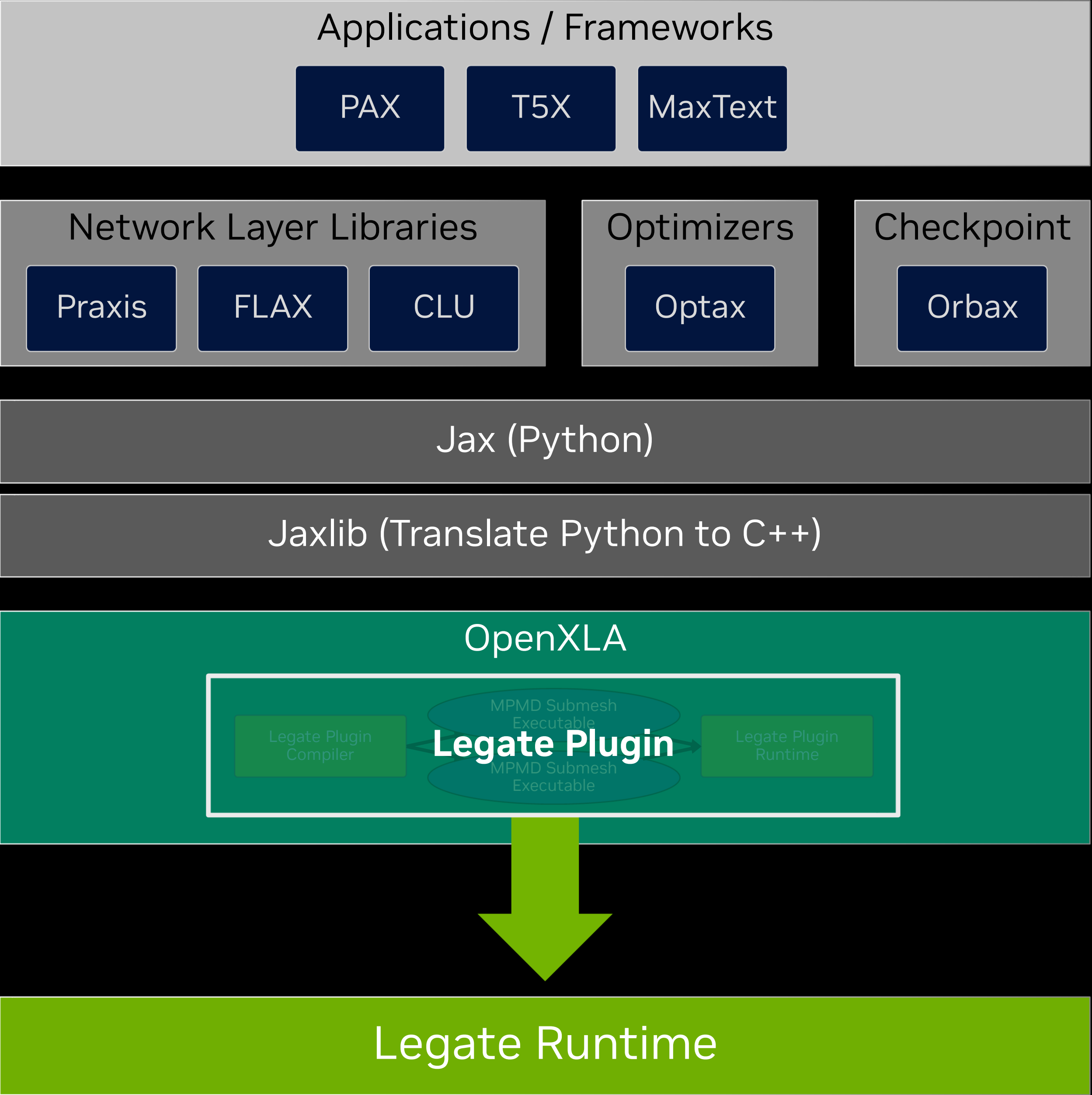


Legate Integration With Jax



Legate JAX: Flexible Parallelization for JAX Programs

Augmenting JAX with easy-to-use pipeline parallelism



Preliminary performance results
OPT 175B with BFloat16 on A100 GPUs (numbers in per-GPU TFLOPS)

	Legate	PAXML (PP)	ALPA
128 GPUs	146.8	133.6	128.6
256 GPUs	131.3	132.0	134.6
512 GPUs	130.8	129.7	122.1

Implemented as a plug-in to JAX

Allows Legate JAX users to stay within the JAX ecosystem[*]

Competitive performance over current state-of-the-art

Memory-friendly interleaved pipeline schedules easily expressible in Legate JAX

[*] All Legate-specific changes for JAX and OpenXLA are planned to be upstreamed

Nsight Systems Multi-Node Analysis

Literally tools for your tools

Recipes to analyze existing reports

Collect reports from multiple sources/nodes

Reduces “getting started” time

Analyze thousands of reports

Scale-out processing

Cluster-wide conclusions

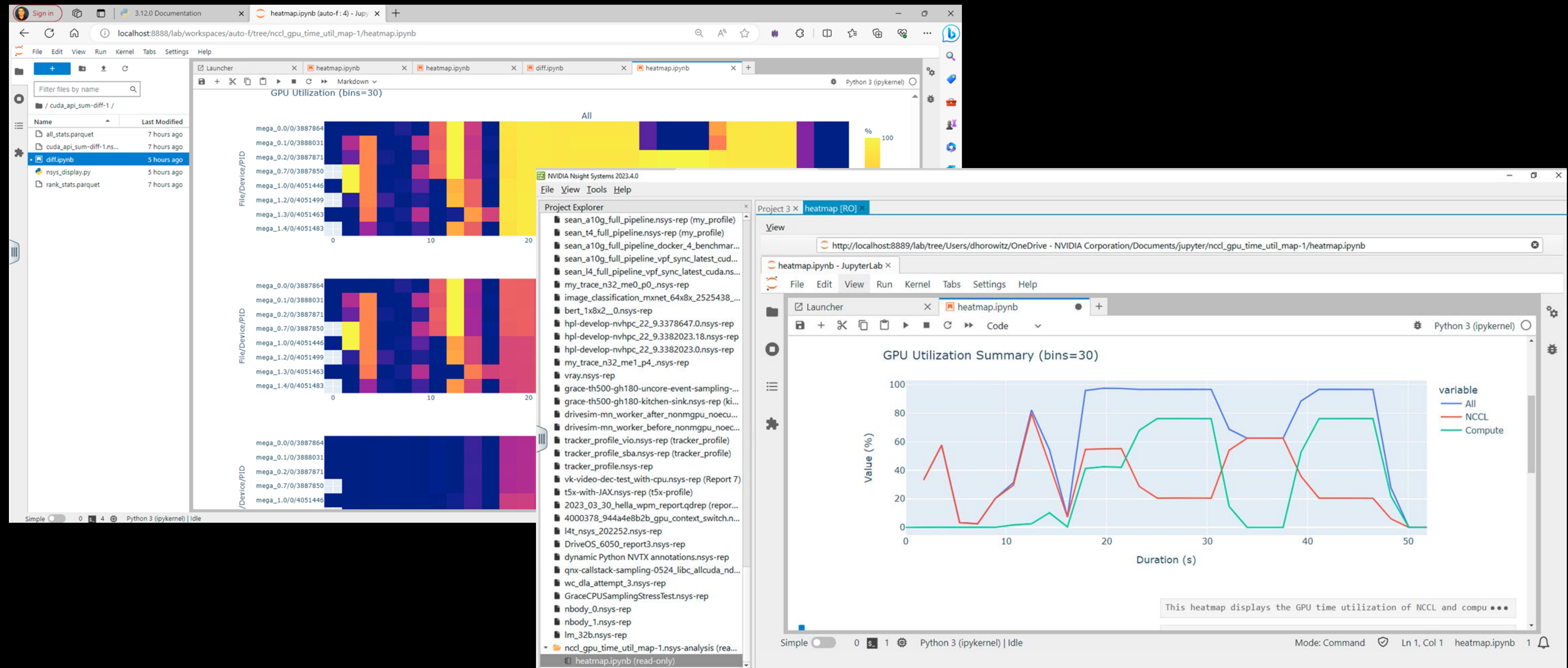
Recipes generate results various formats

Jupyter Notebook

Parquet, CSV, ...



Tools Notebook Loadable in Nsight Systems or Jupyter Lab



[Achieving Higher Performance From Your Data Center and Cloud Application \[S62388\]](#)

Multi-Node Heatmap Tools

Example: Analysing Compute-Only Zones

GPU Utilization

Communications

Compute-Only

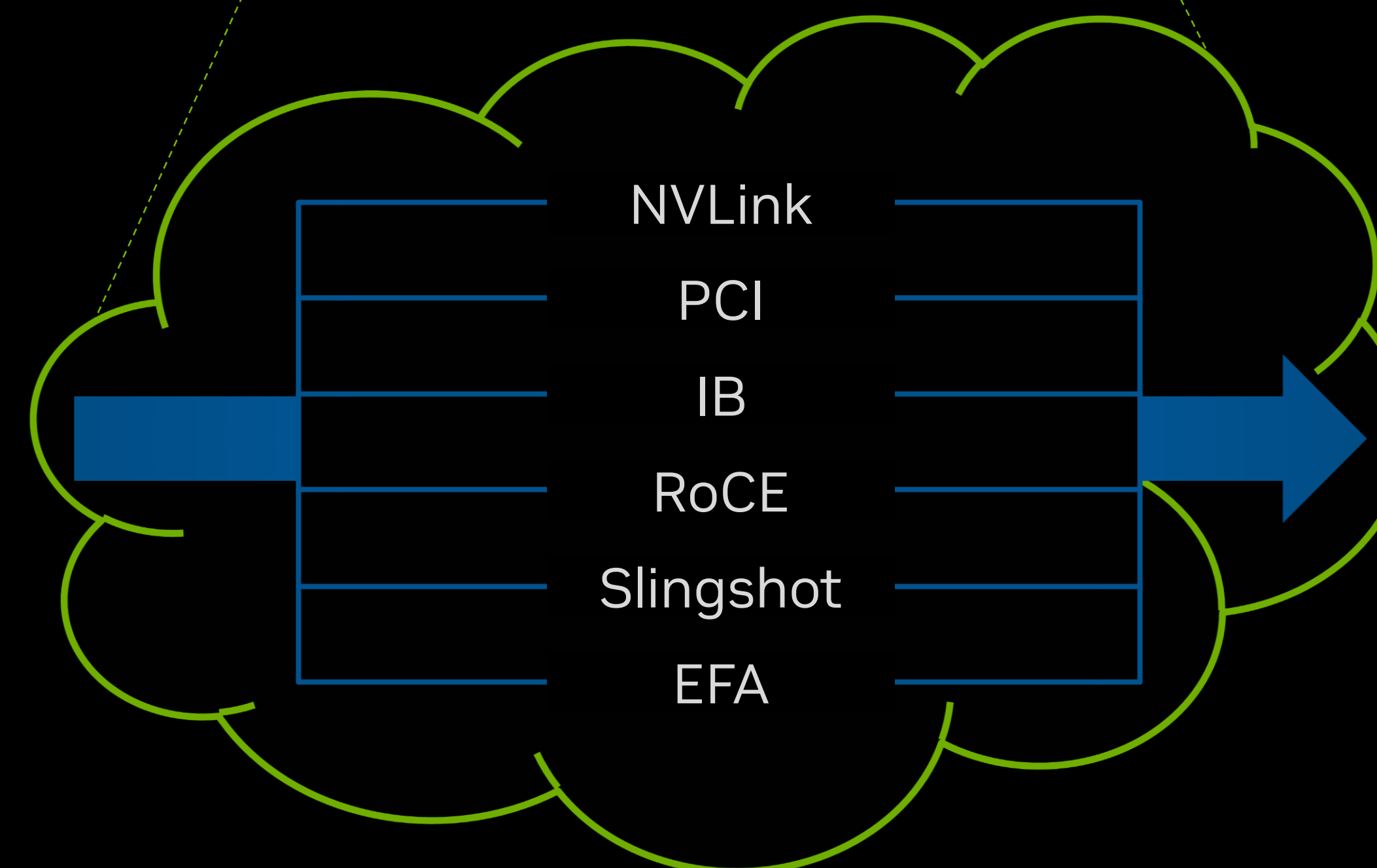


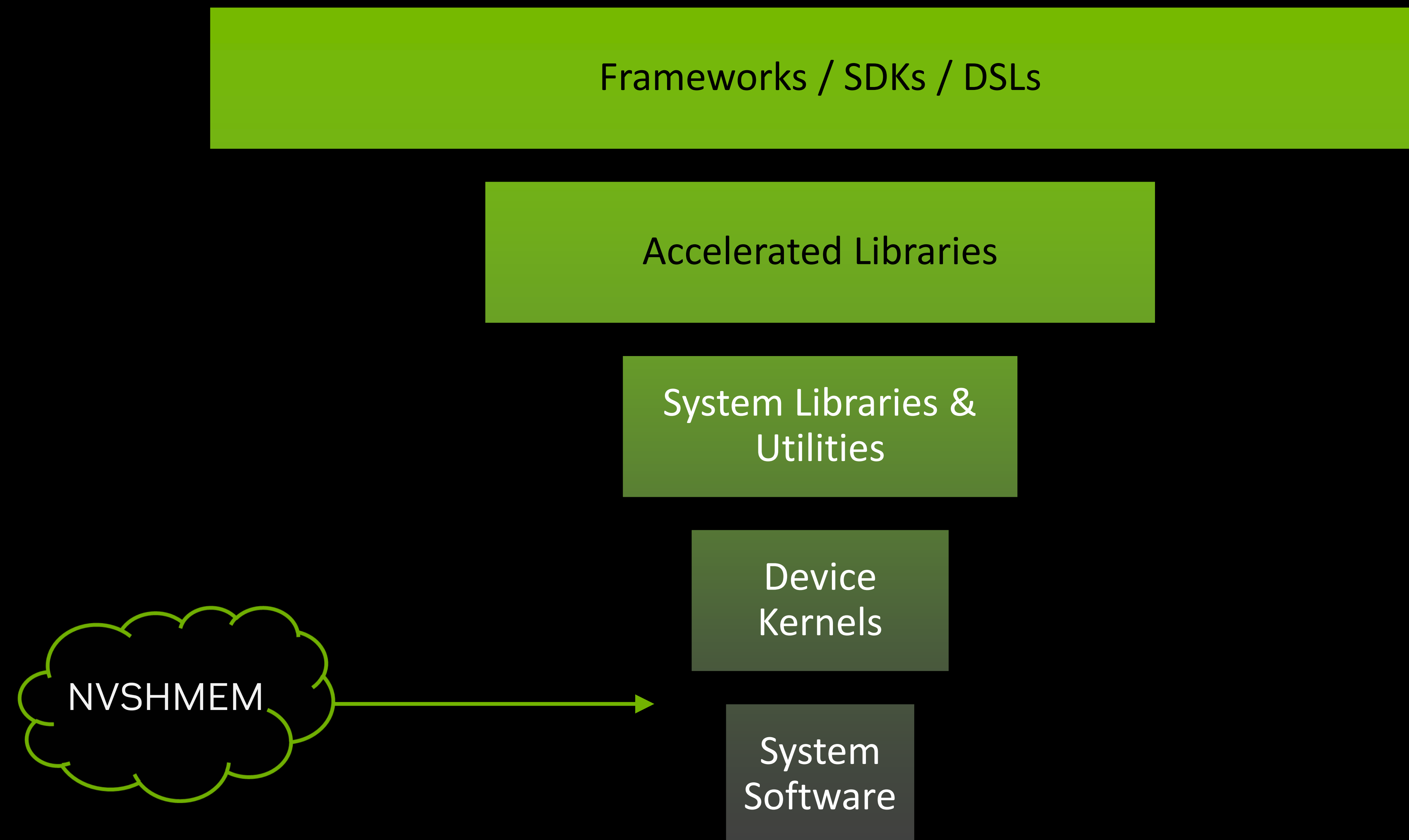
NVSHMEM: Fine-Grained Comms + Compute From Within GPU Kernels

Underneath NVSHMEM lies the GPUDirect technology from the MagnumIO family



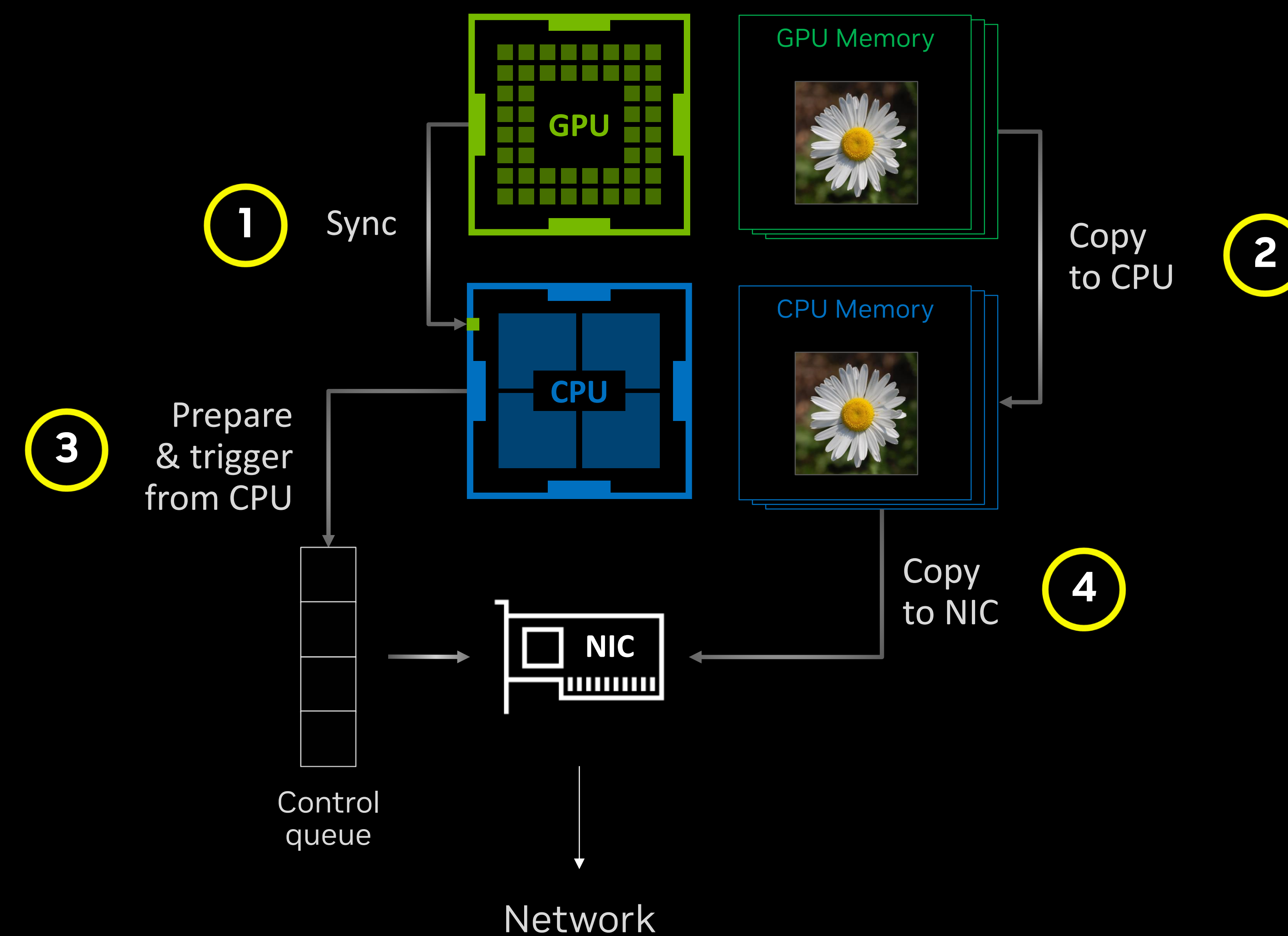
NVSHMEM abstracts any transport for GPU<>GPU communications





Sending GPU Data Over a Network Before GPUDirect

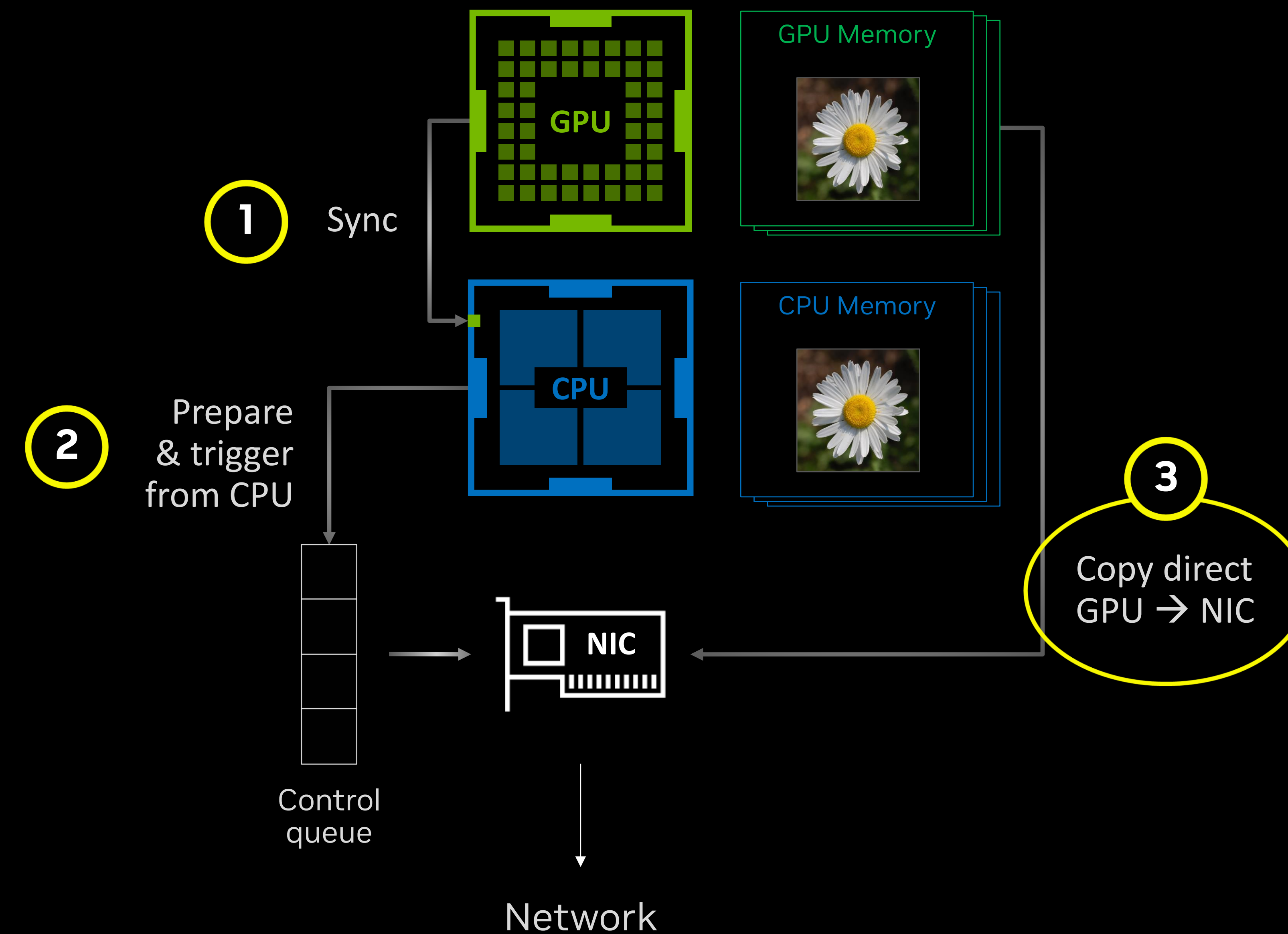
All data & control flows through CPU



Data: GPU → CPU → NIC
Control: GPU → CPU → NIC } 4 steps

GPUDirect

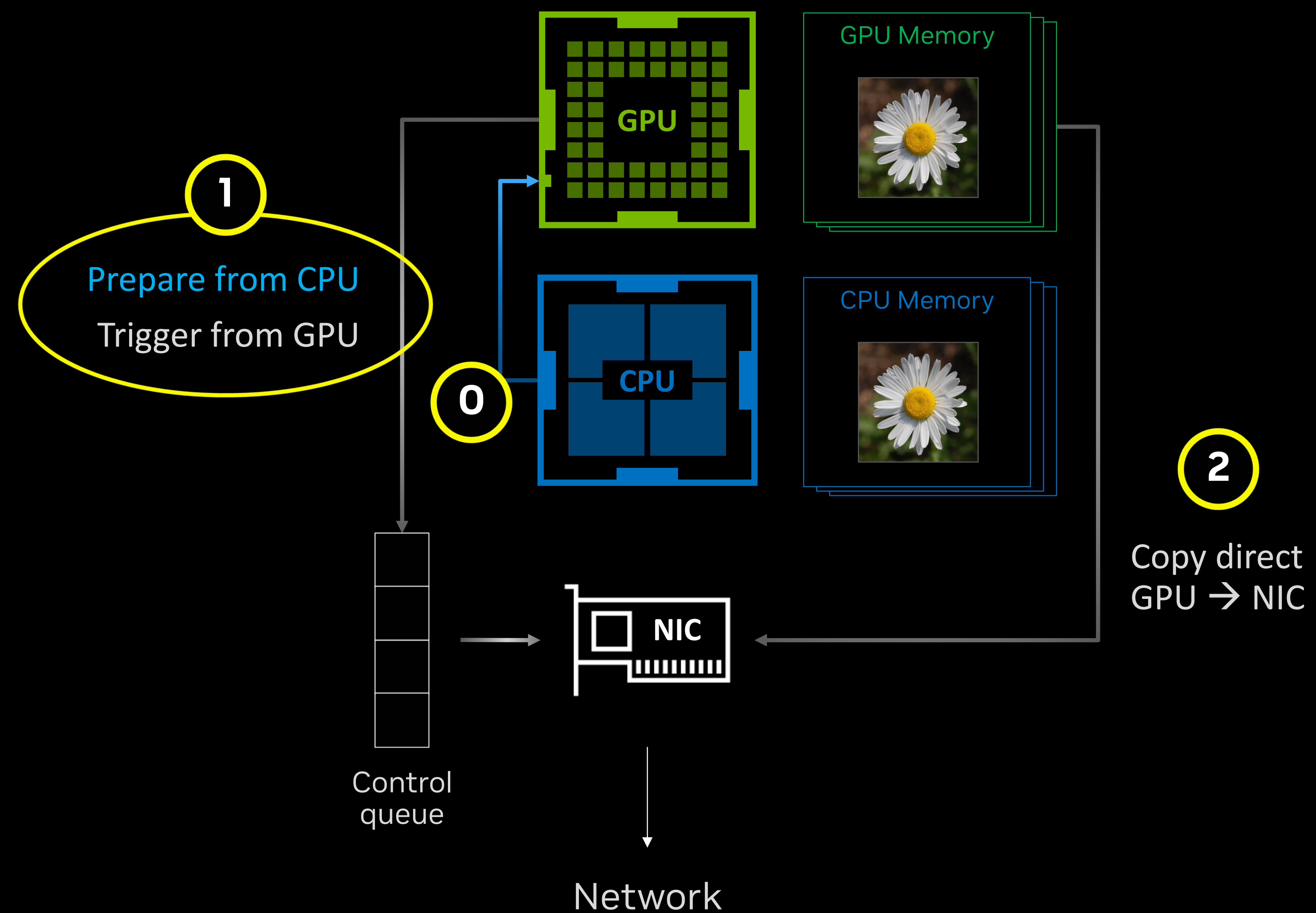
Data flows direct from GPU, control signals still hop through CPU



Data: GPU → NIC
Control: GPU → CPU → NIC } 3 steps

GPUDirect-Async

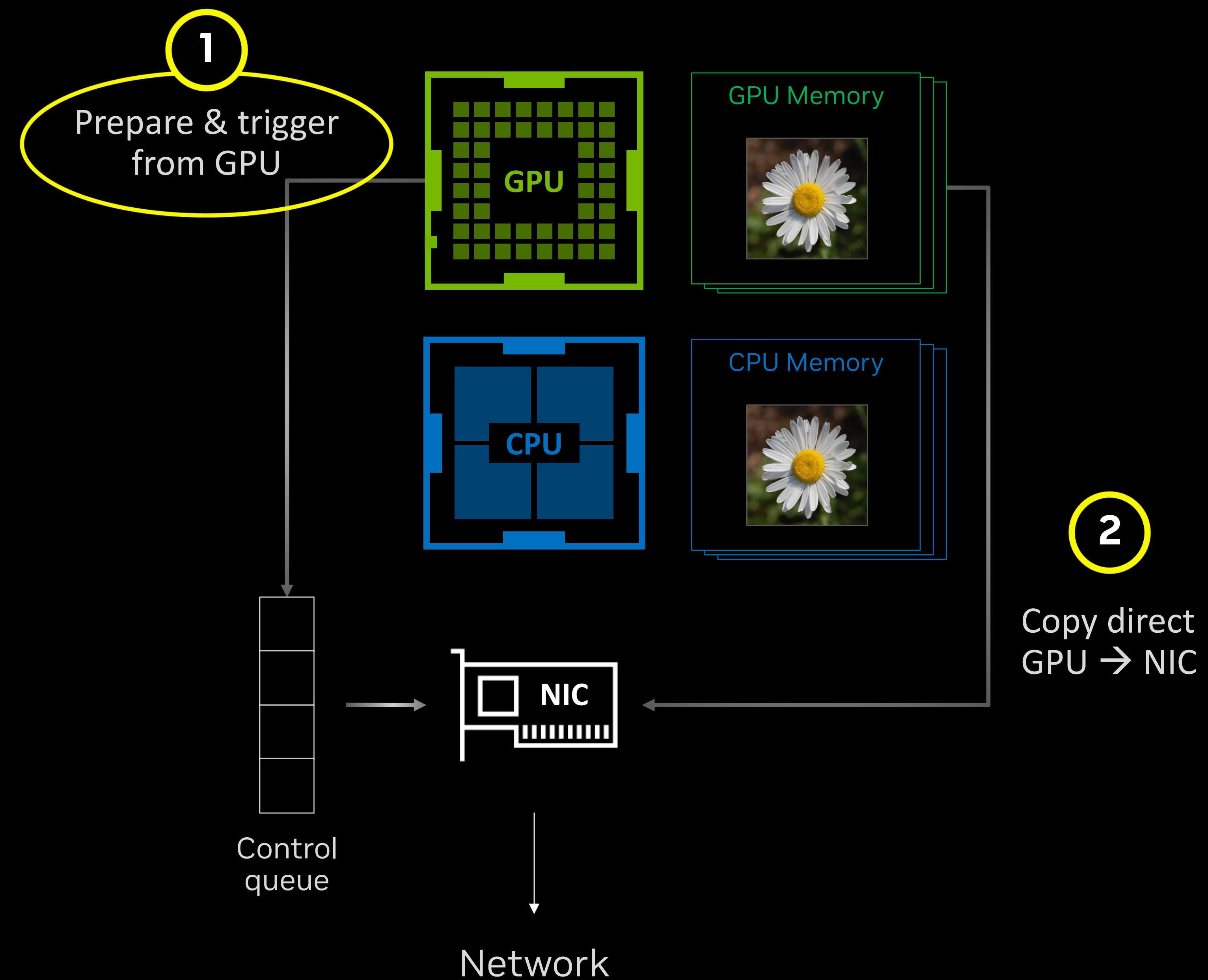
CPU prepares send packets, but GPU signals ready-to-send via CPU proxy



Data: GPU → NIC
Control: [CPU] → GPU → NIC } 2½ steps

GPUDirect-KI

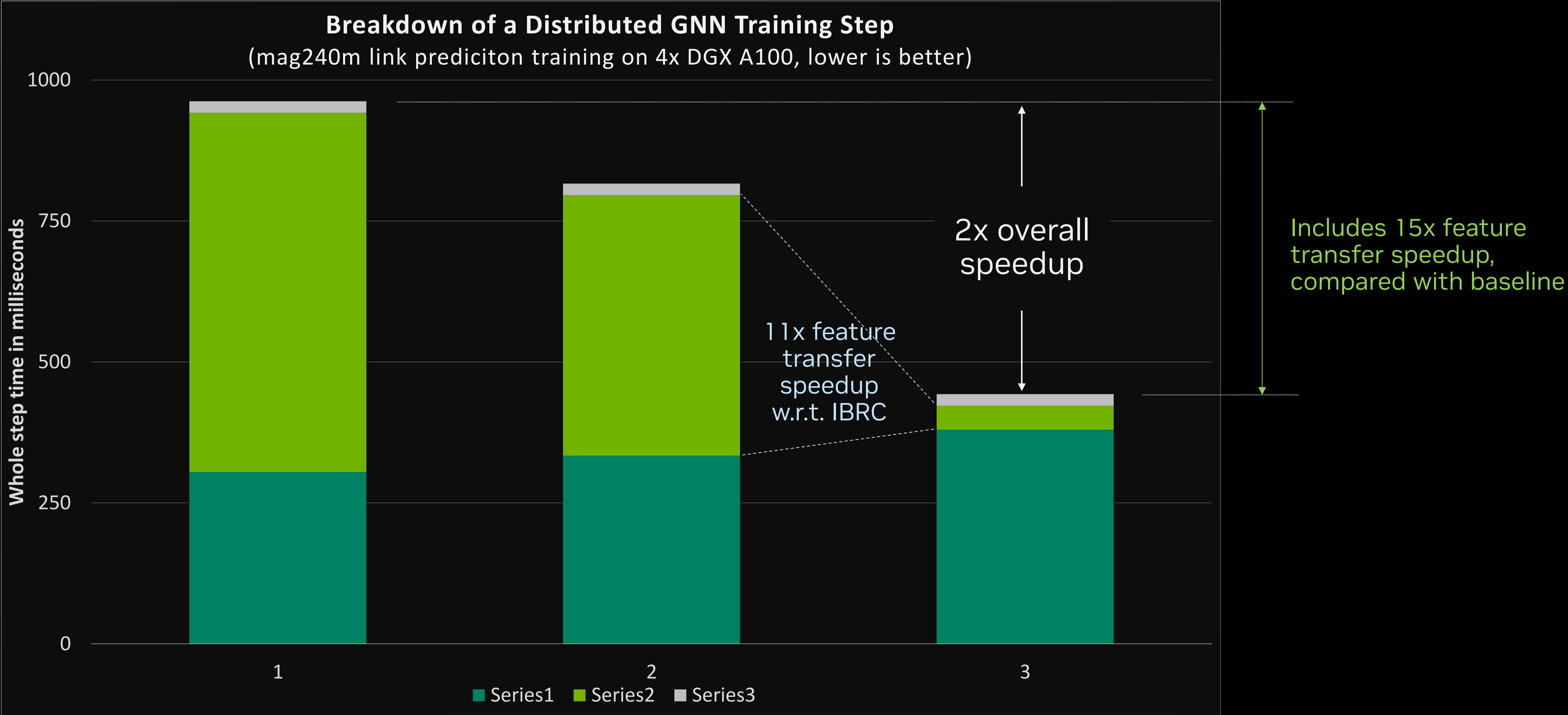
GPU autonomously directs **both** data and control flows to NIC



Data: GPU → NIC
Control: GPU → NIC

} 2 steps

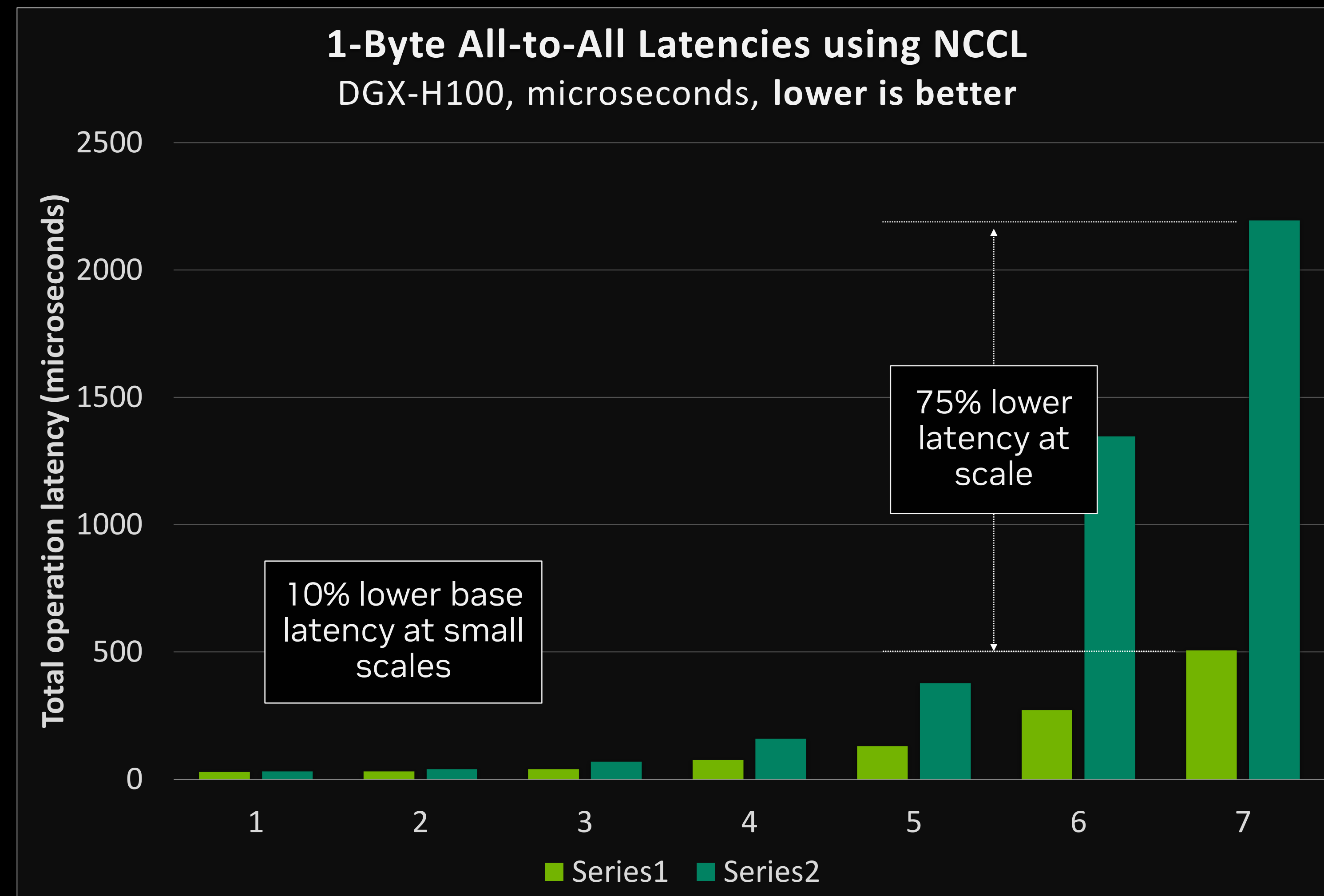
Benefit of GPUDirect-KI for NVSHMEM



mag240m link prediction training on 4x DGX A100 with 8x 200Gb/s IB each.
Feature (message) size is 1.5KB with 125K features fetched by each GPU from remote SYSMEMs per training step.

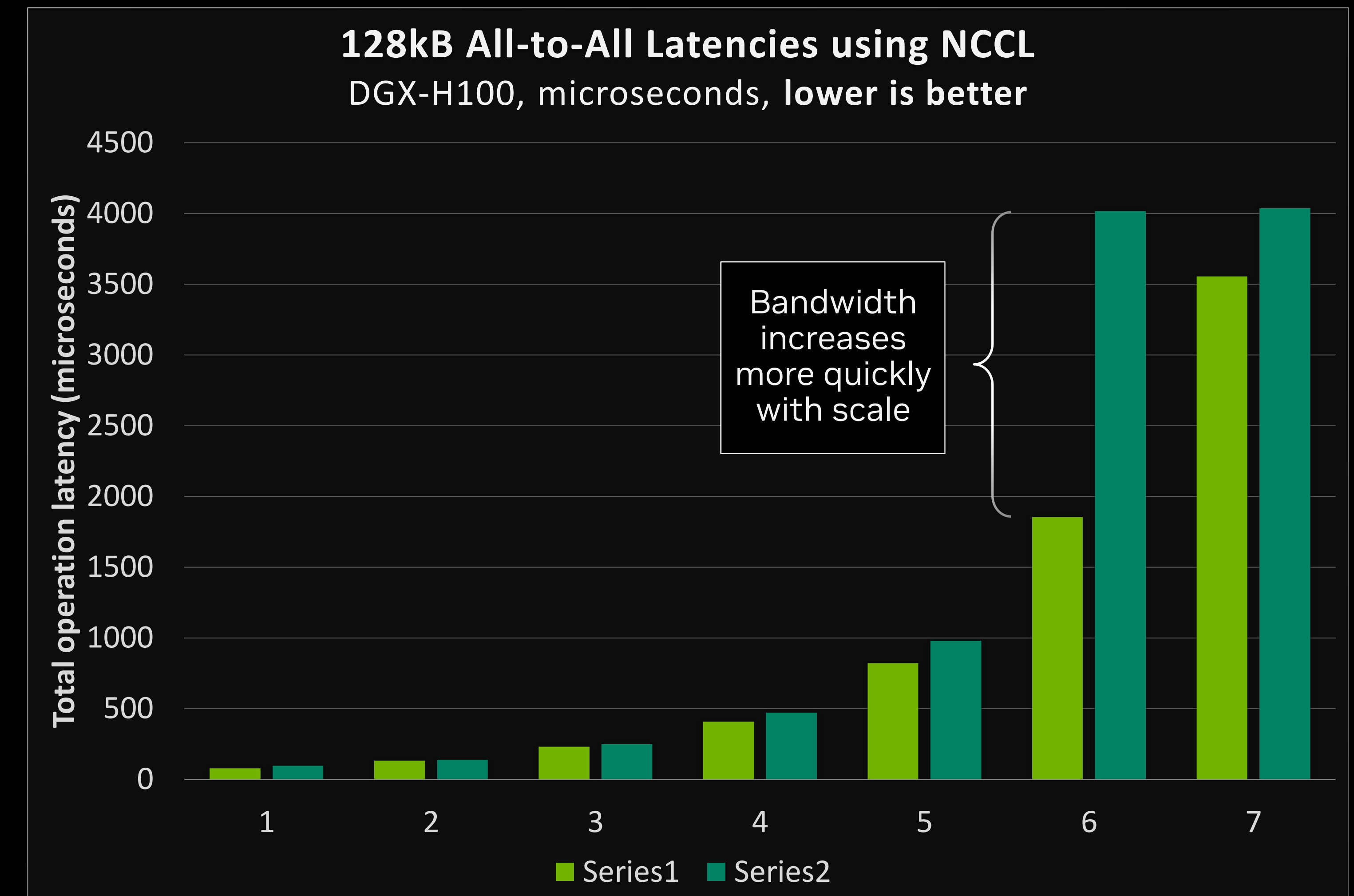
NCCL Sees Significant Latency Benefits From GPUDirect-KI

1-byte small messages, lower is better



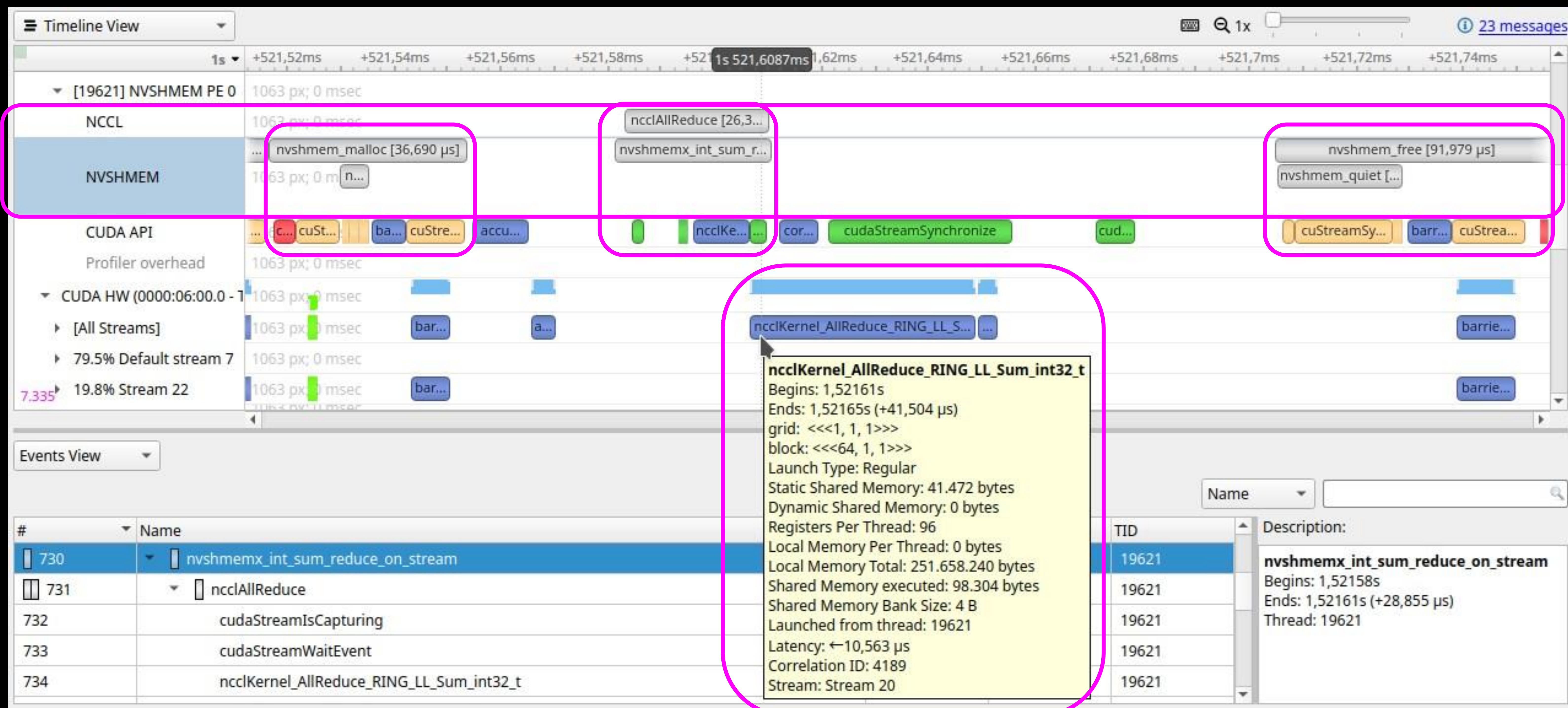
Smaller messages are **latency-limited**, so lower latencies mean faster communication

128kB large messages, lower is better



Larger messages are **bandwidth-limited**, so lower latencies indicate higher bandwidth achieved

NCCL & NVSHMEM Tools Trace



single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```


single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}
```

Runs on GPU

Runs on CPU

```
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```


single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```

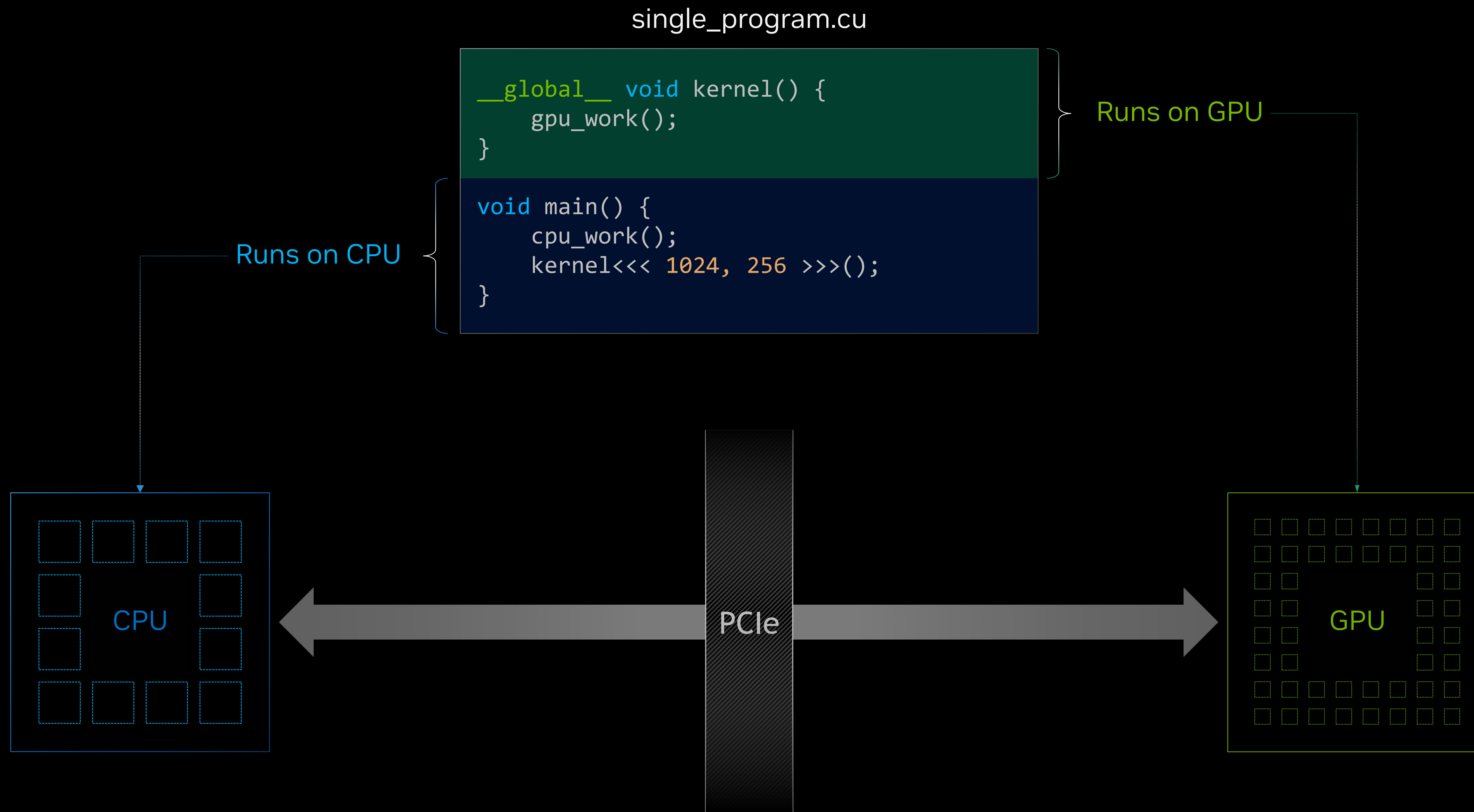
Runs on CPU

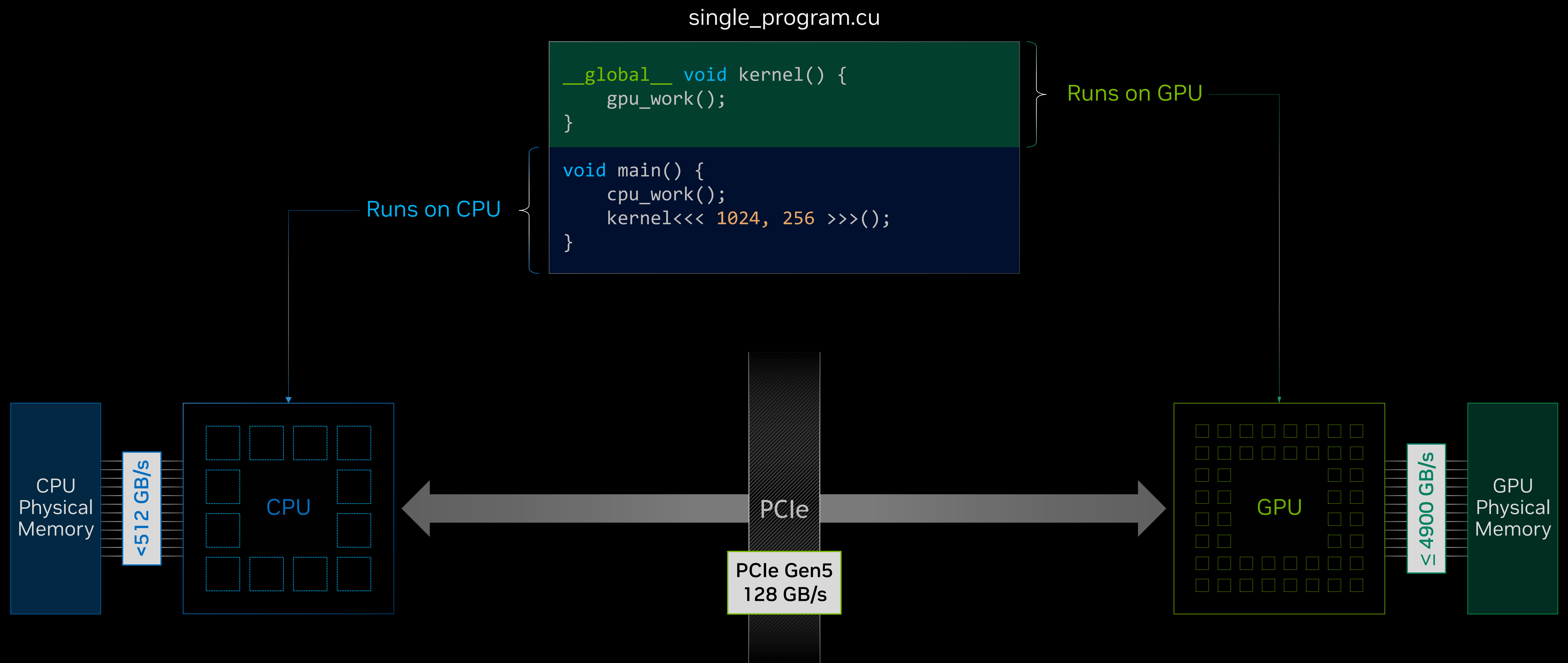
Runs on GPU

*Our key realization was that parallel computing
extends regular computing, not replaces it*

- NVIDIA CEO Jensen Huang

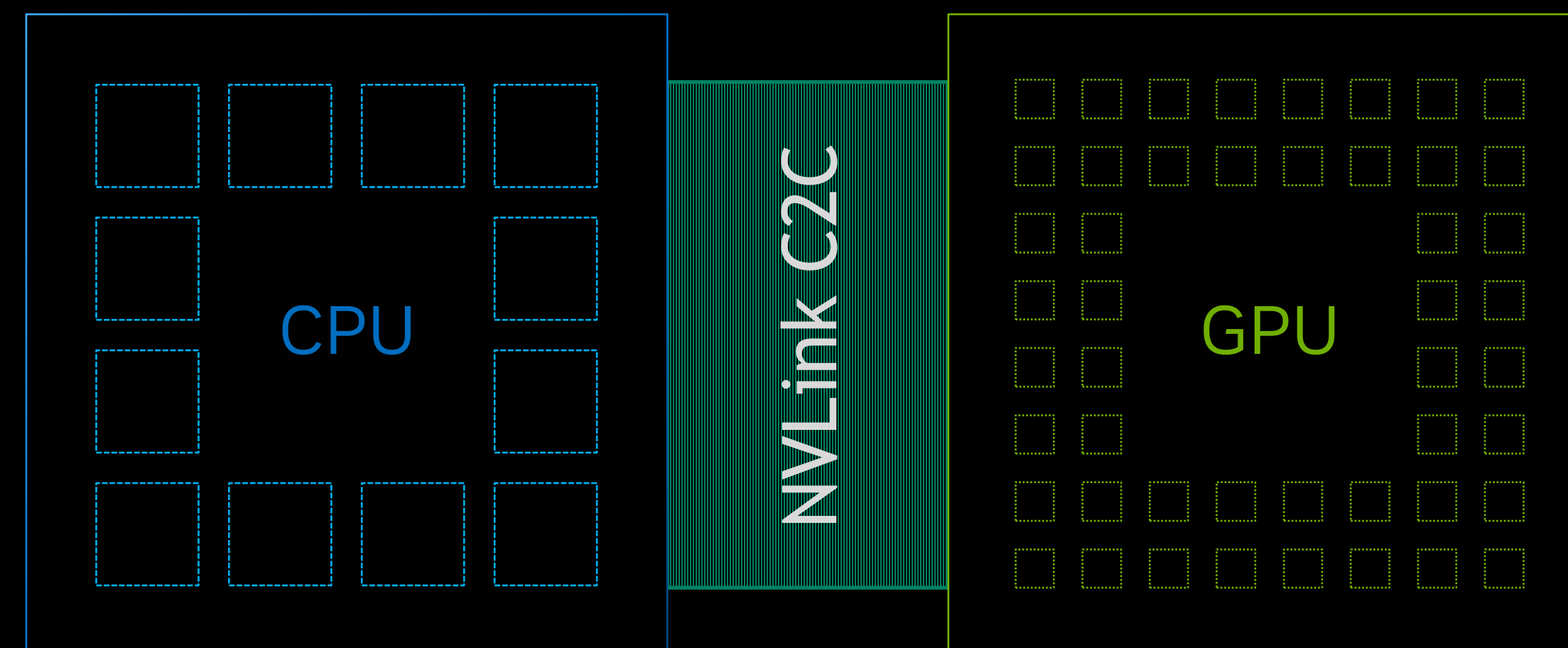






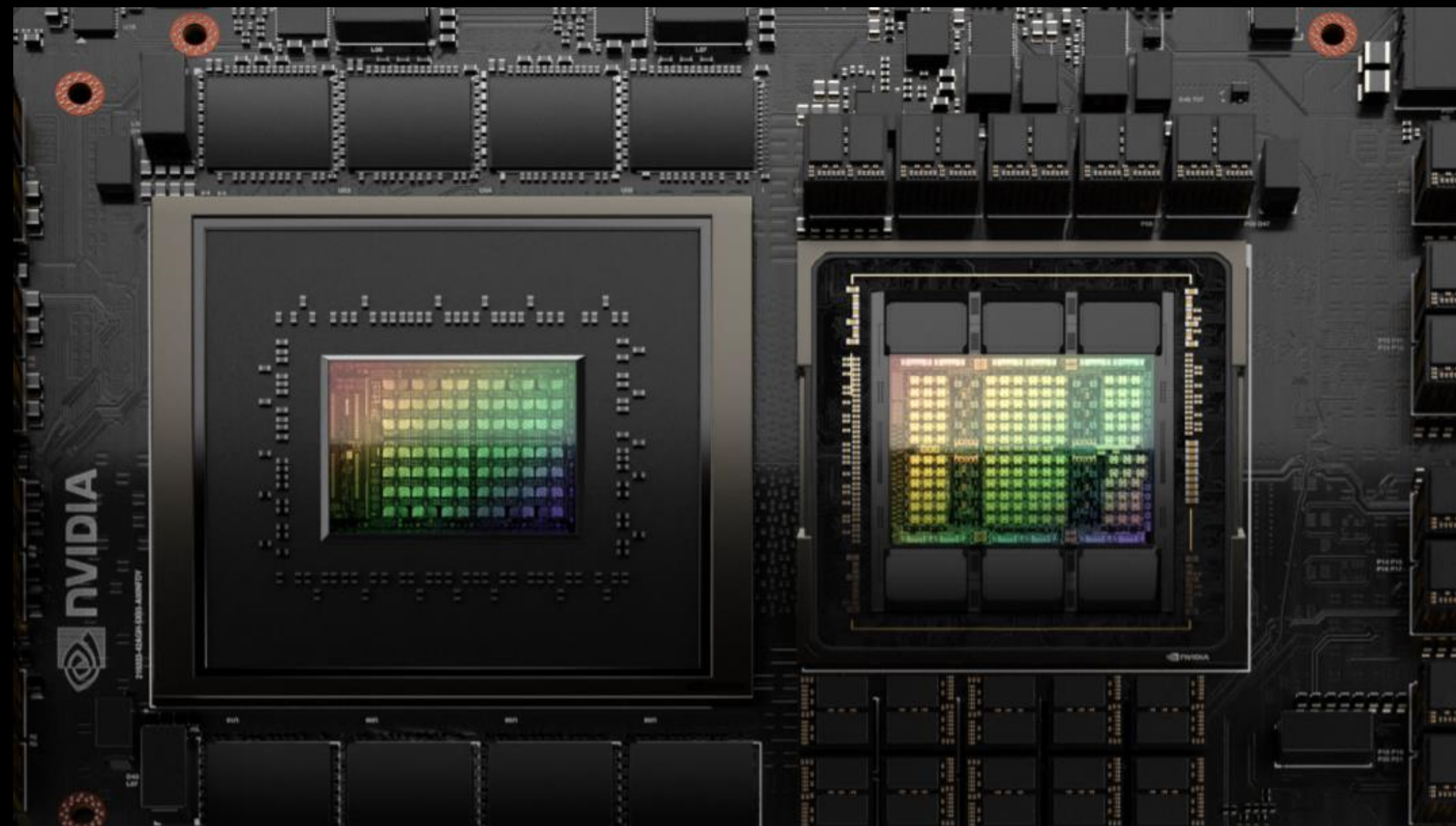
single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```



single_program.cu

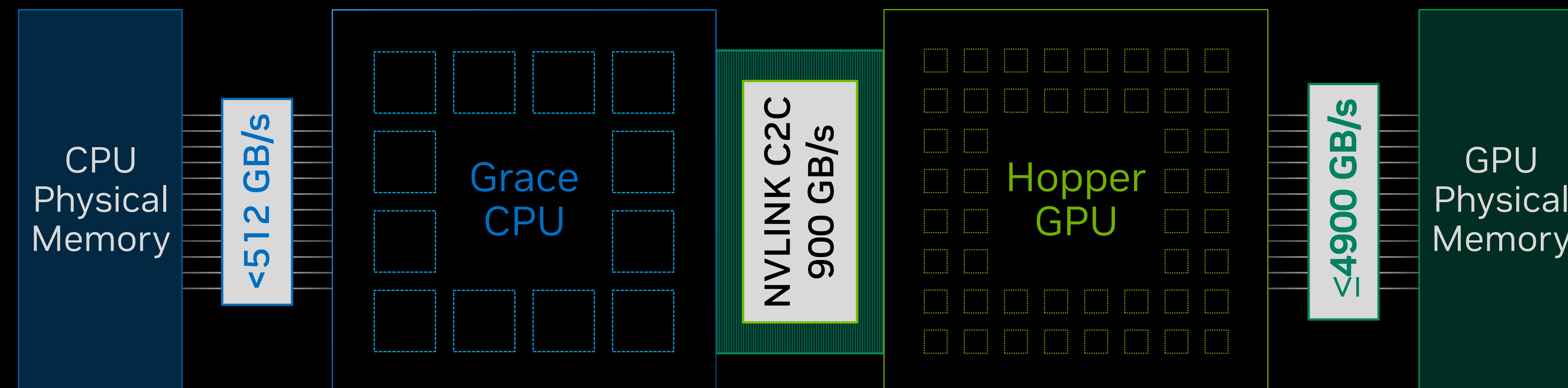
```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```



Grace-Hopper: A Unified Single Heterogeneous Processor

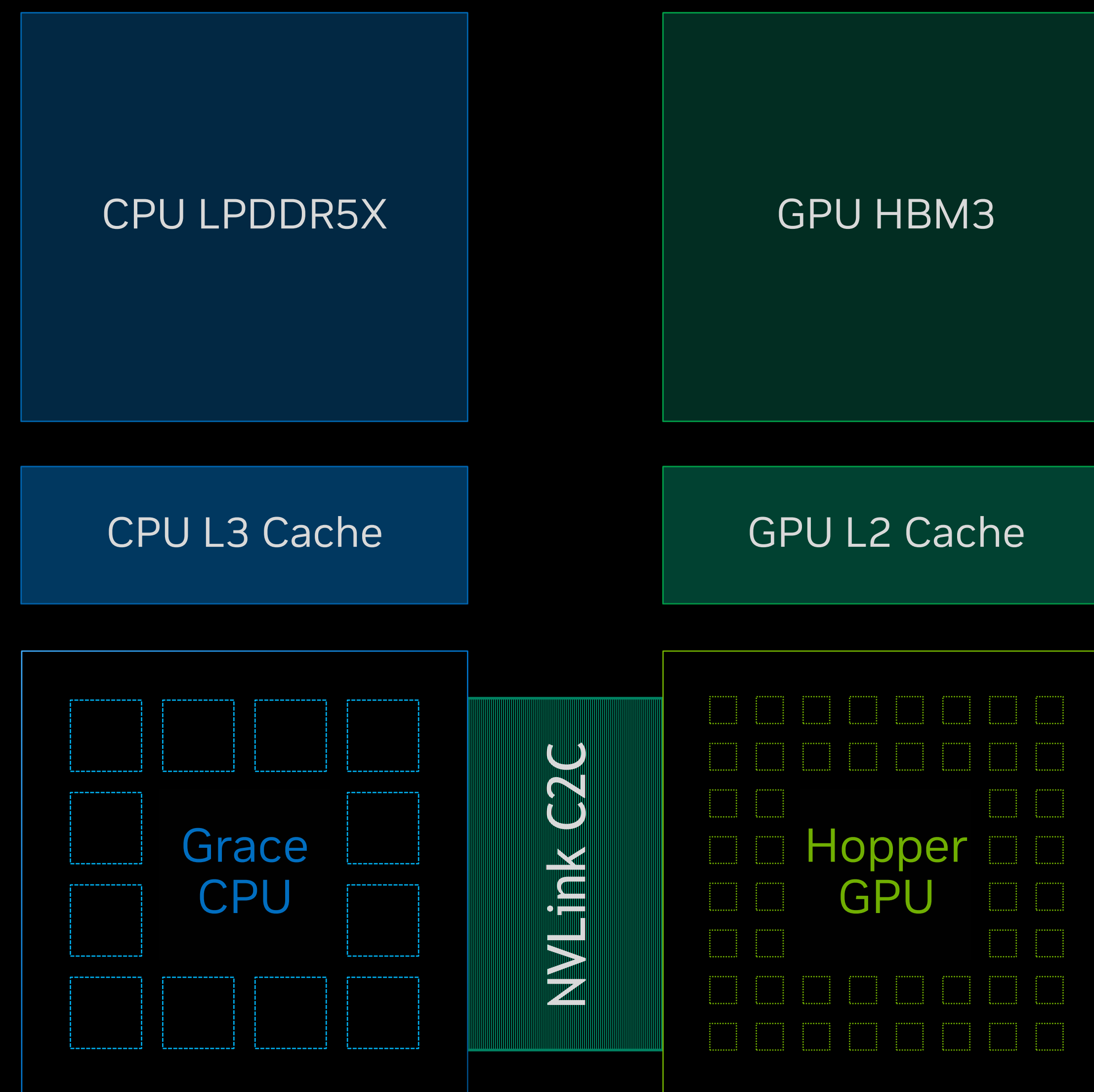
single_program.cu

```
__global__ void kernel() {  
    gpu_work();  
}  
  
void main() {  
    cpu_work();  
    kernel<<< 1024, 256 >>>();  
}
```

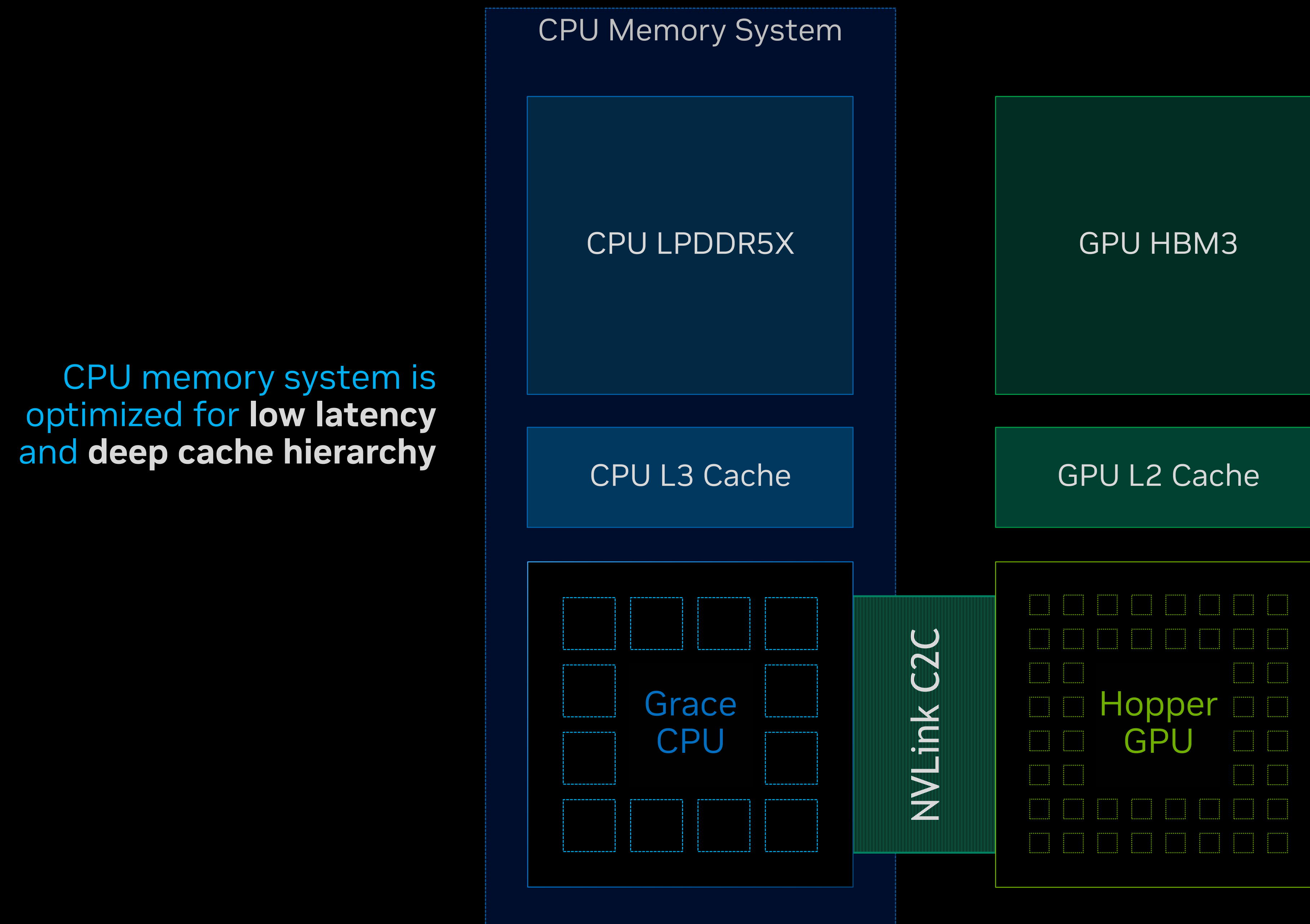


[A Deep Dive into the Latest HPC Software \[S61203\]](#)

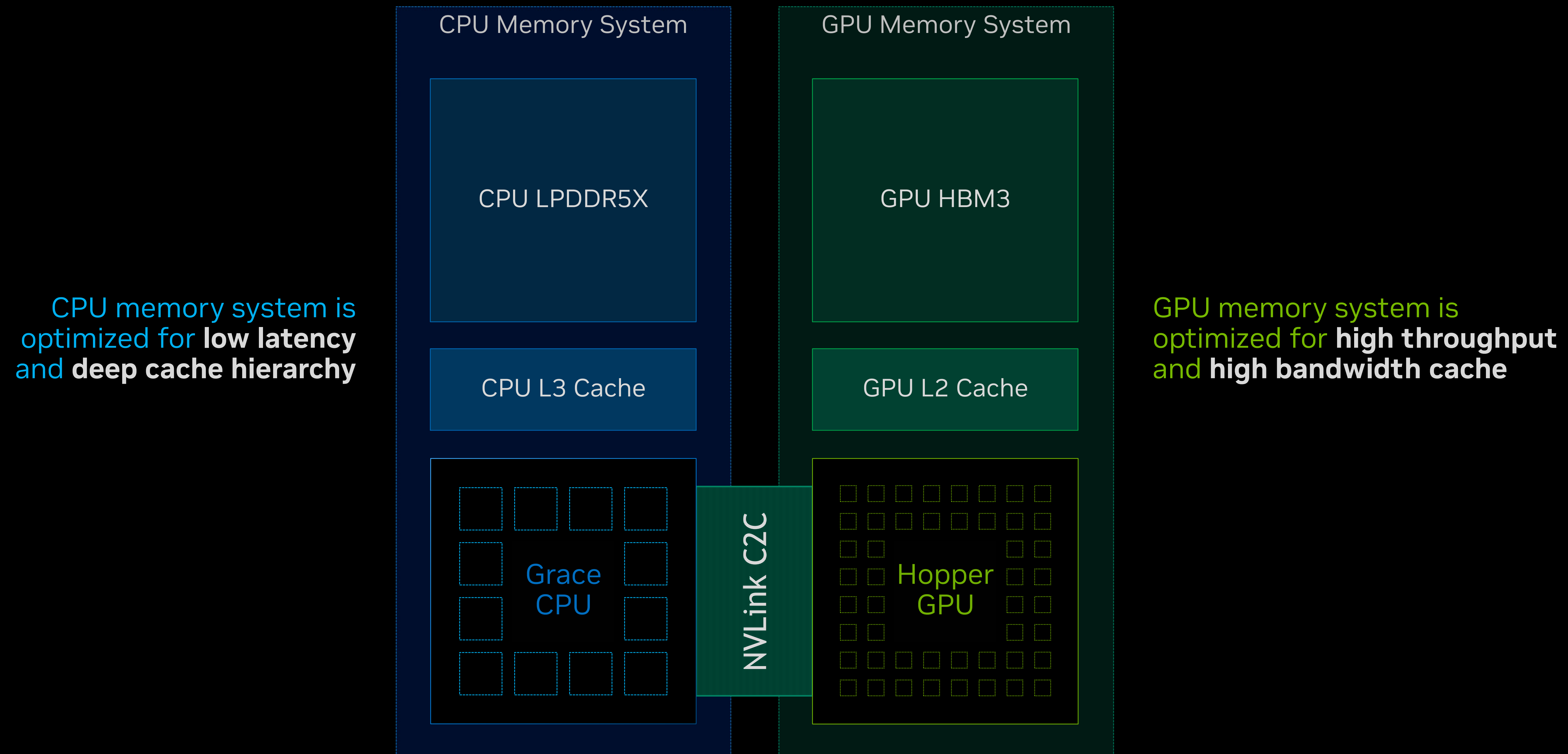
Two Memory Systems, Each Optimized For Its Processor



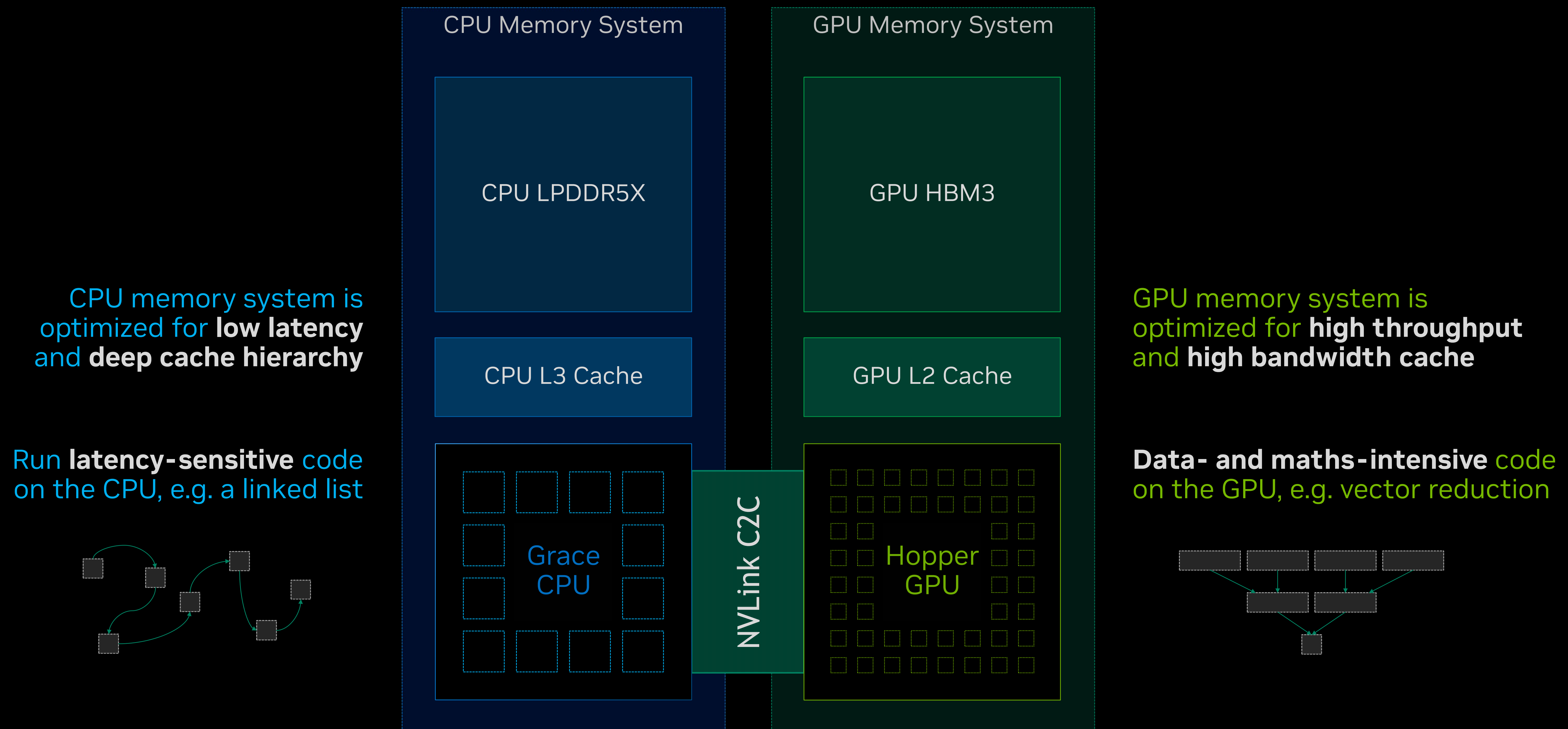
Two Memory Systems, Each Optimized For Its Processor



Two Memory Systems, Each Optimized For Its Processor

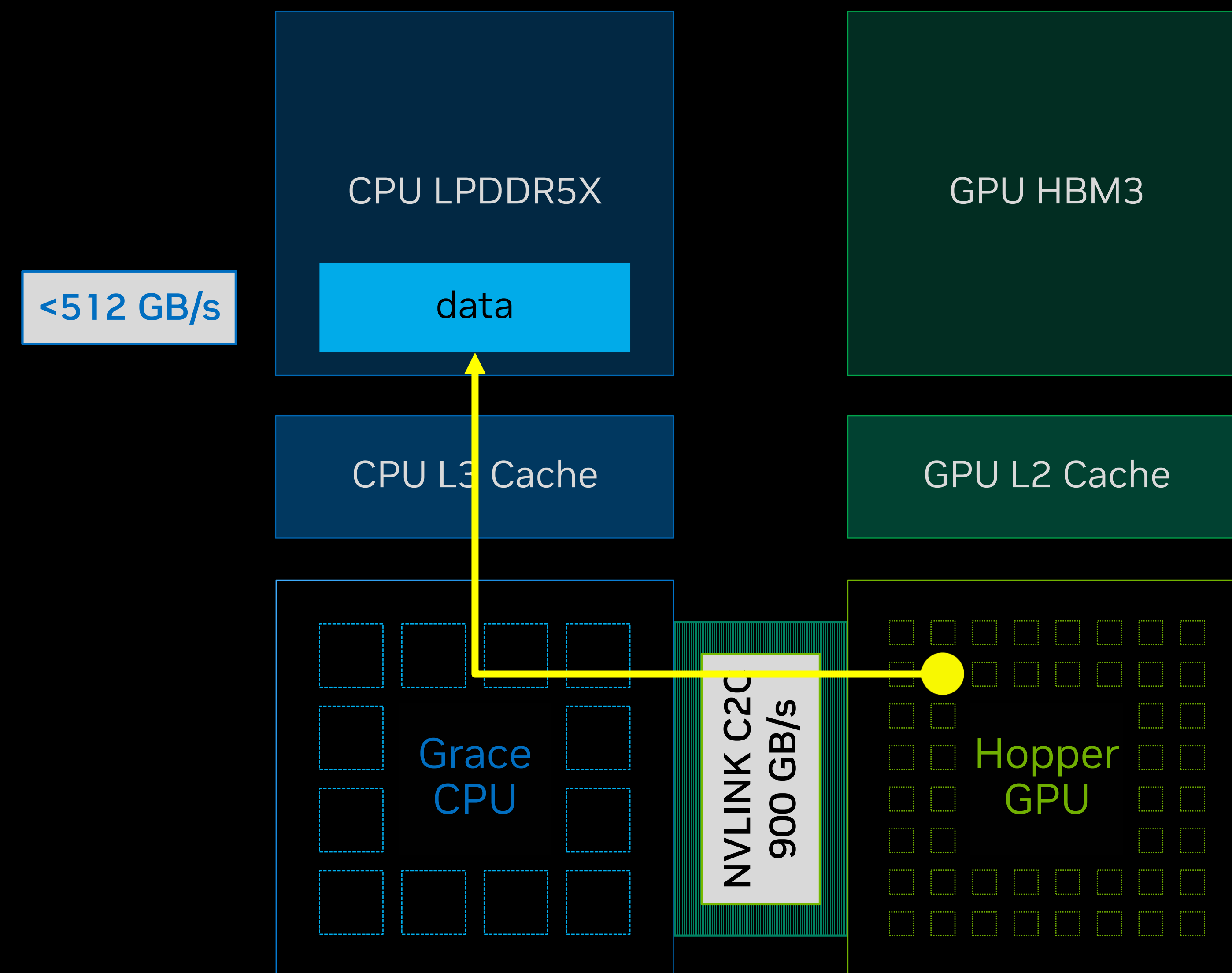


Two Memory Systems, Each Optimized For Its Processor



Two Memory Systems With One Address Space

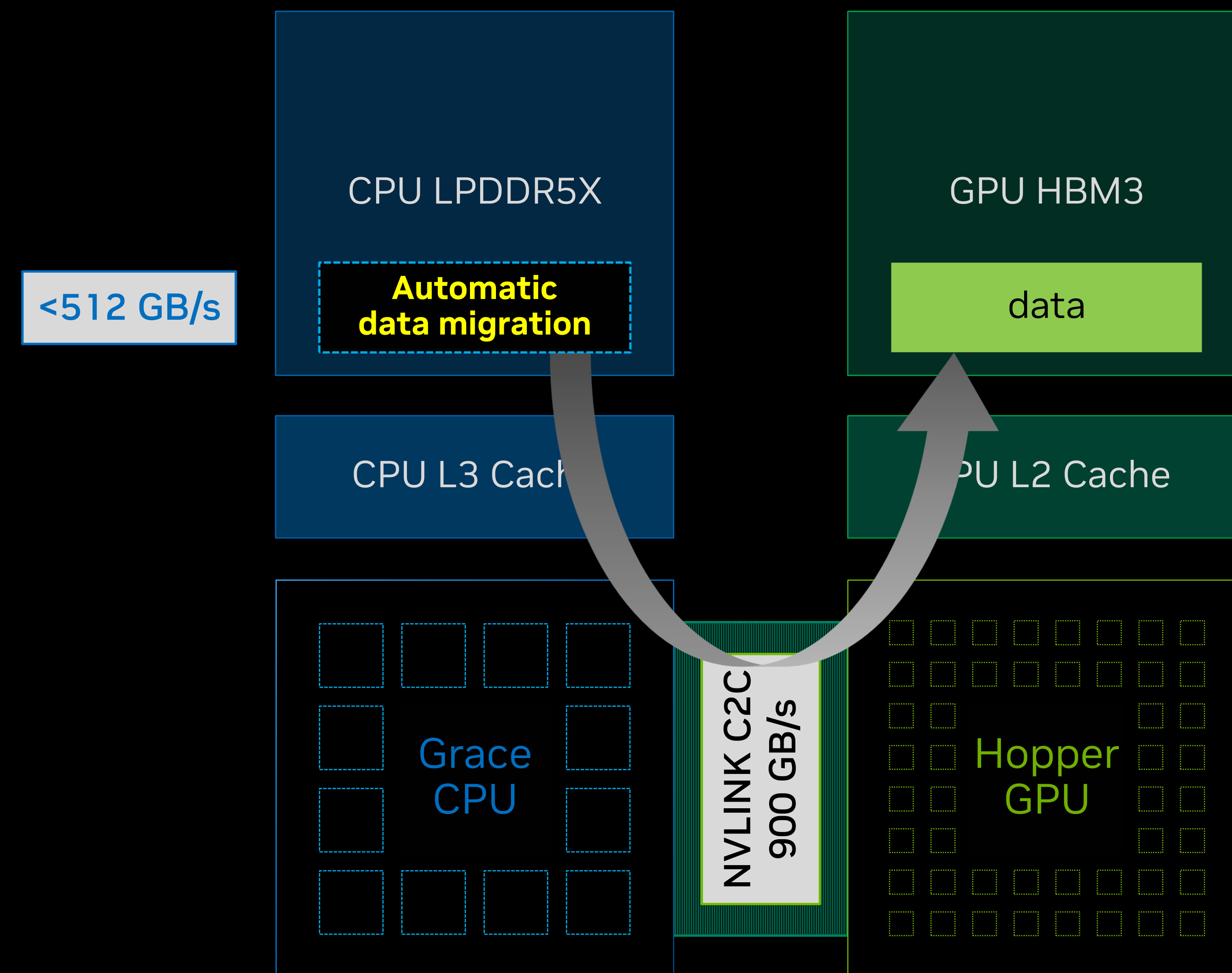
Both CPU and GPU can access the other's memory at any time



C2C connection is fast enough that GPU reads data at full LPDDR5X rate

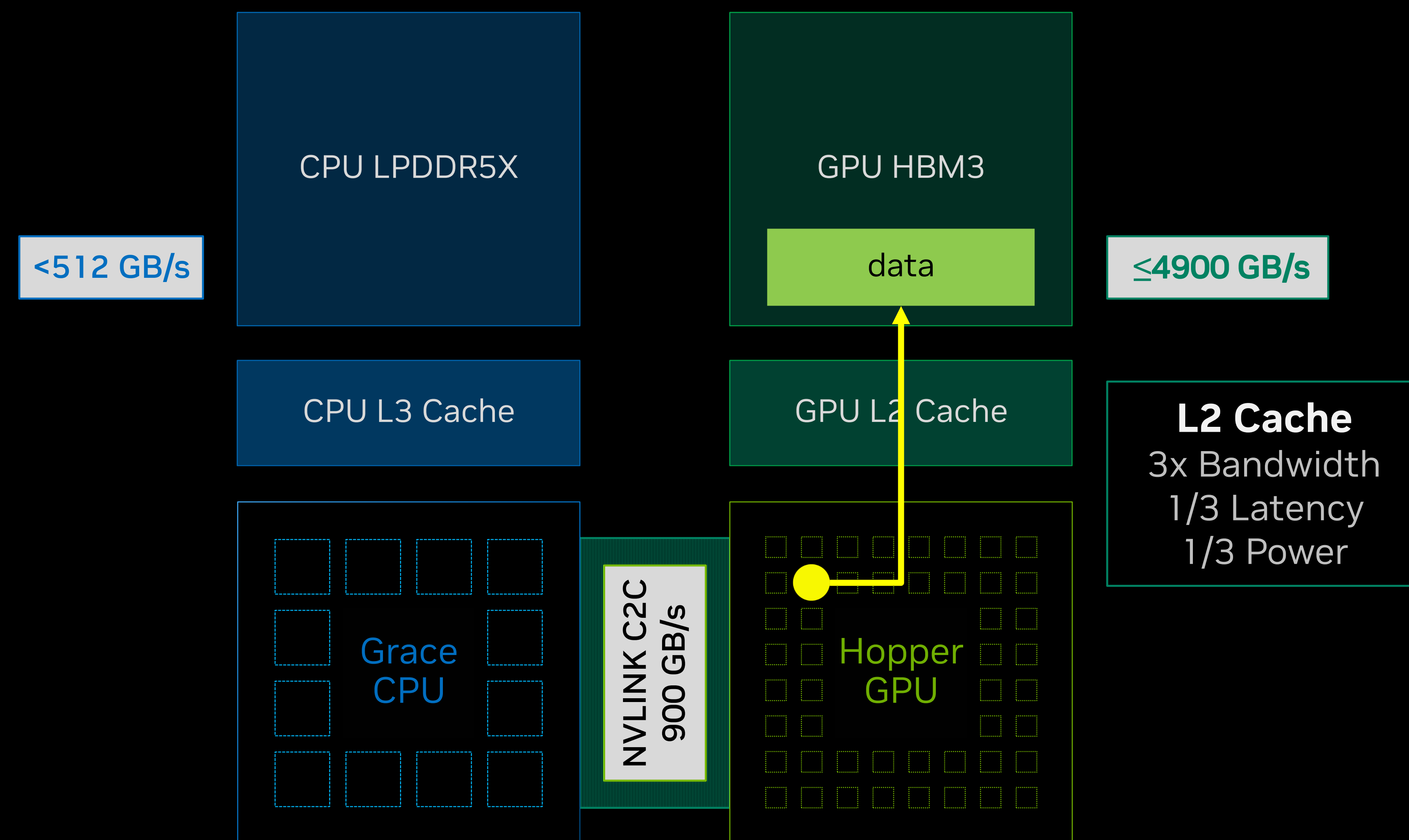
Two Memory Systems With One Address Space

The Unified Memory system can move data to the GPU automatically based on access patterns



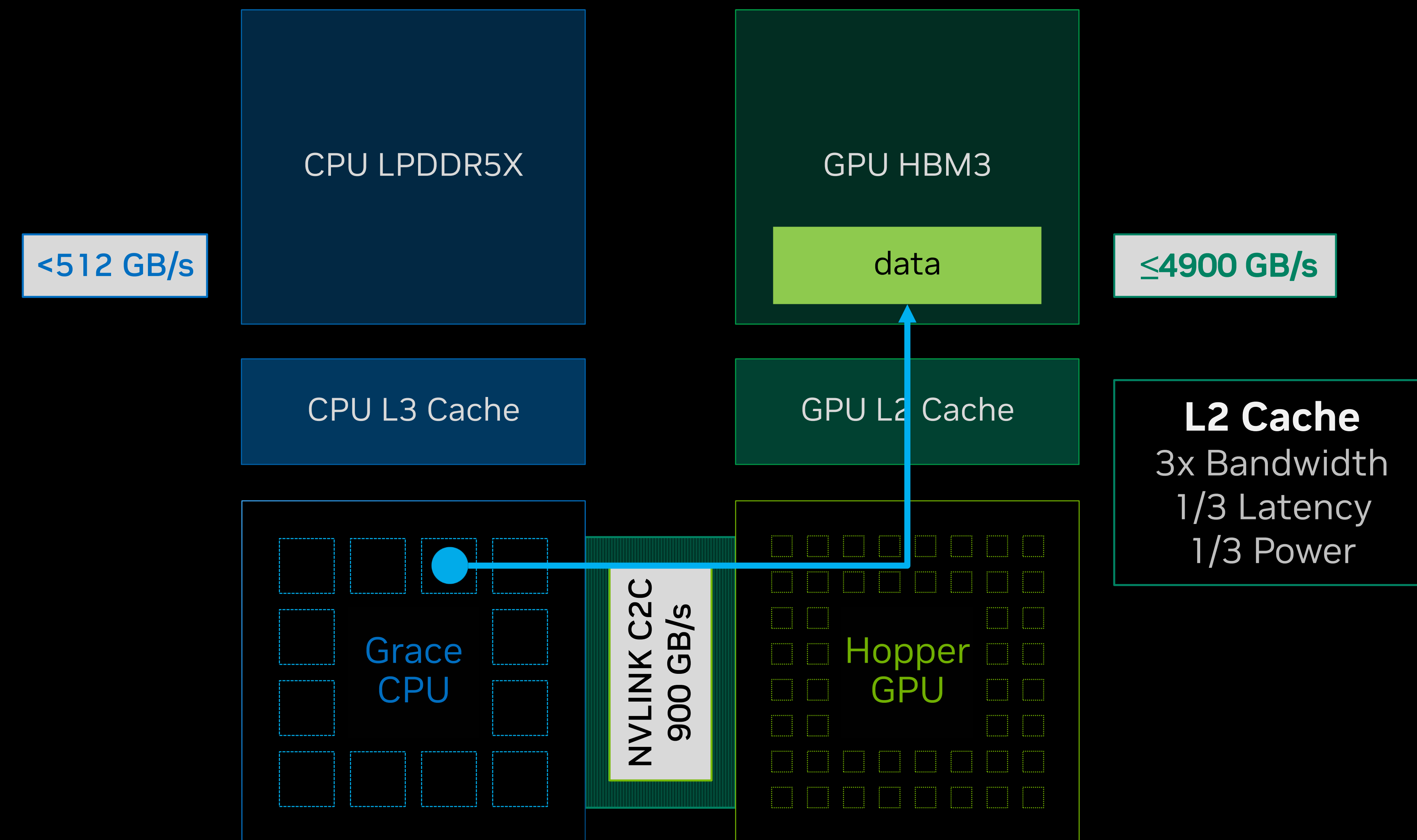
Data Migration Optimizes GPU Accesses on GPU Memory System

Data migration to the GPU can enable as much as 10x faster data access from the GPU



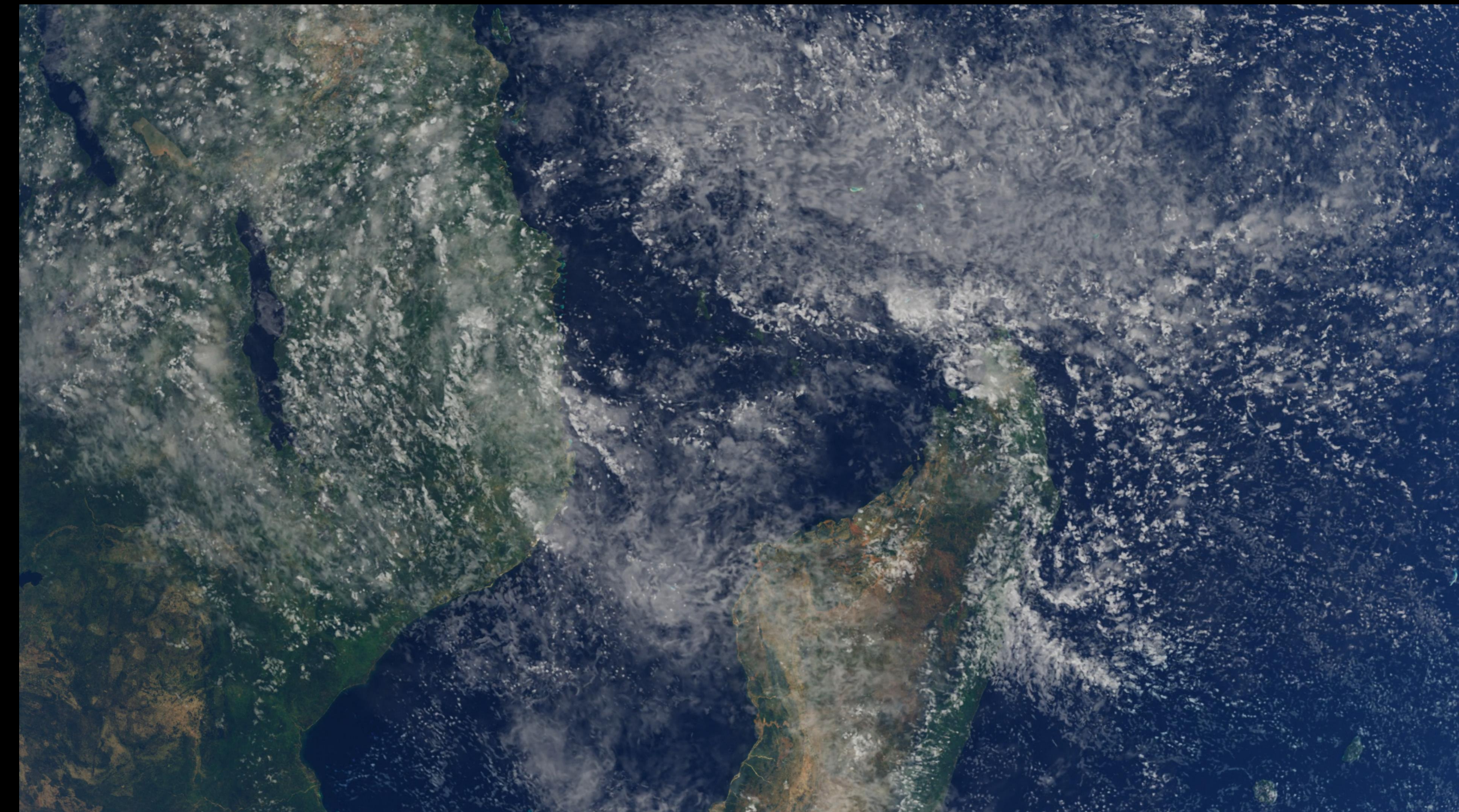
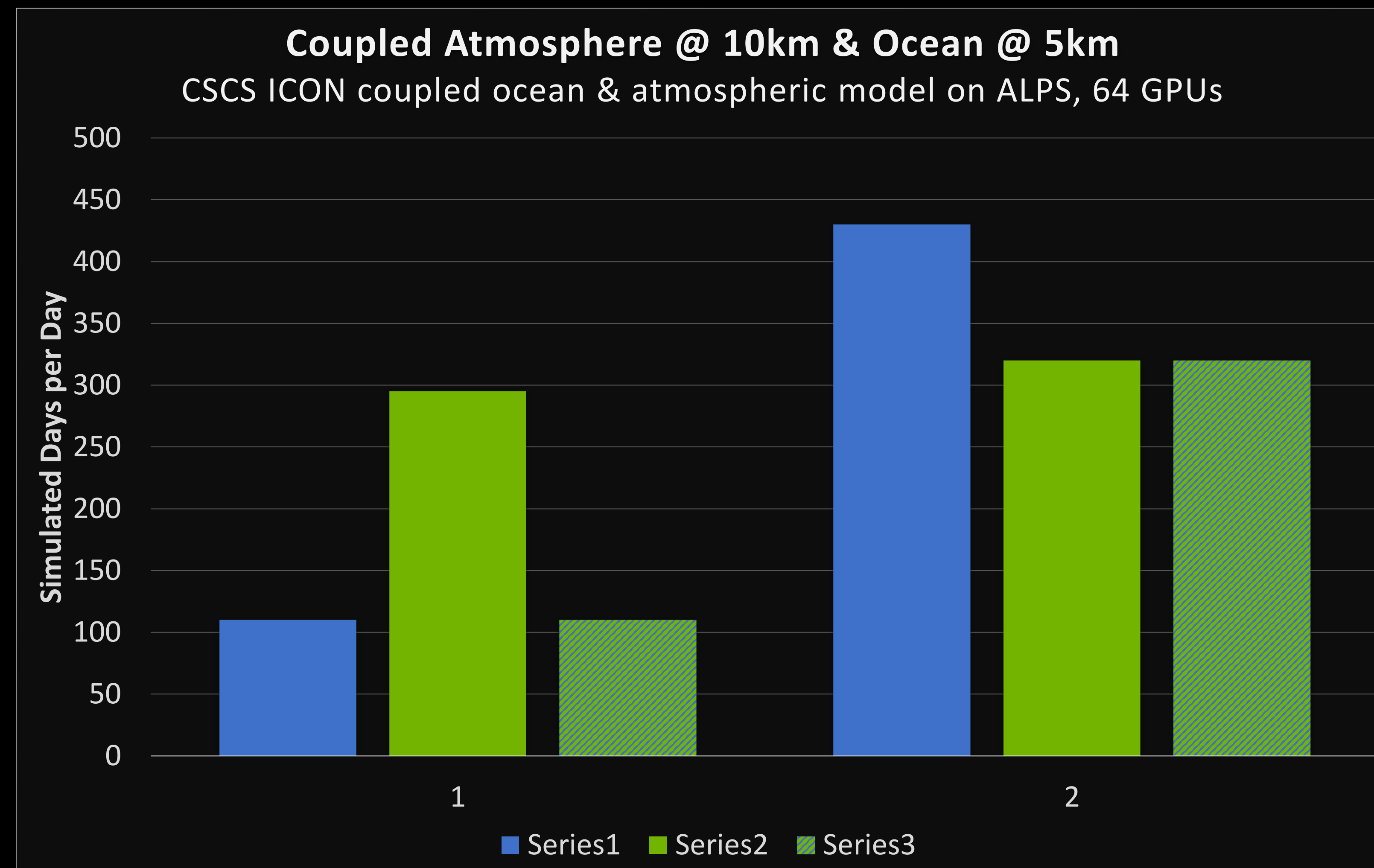
Migration Does Not Affect CPU Access To Data

The CPU can still access the data even when it is in GPU memory

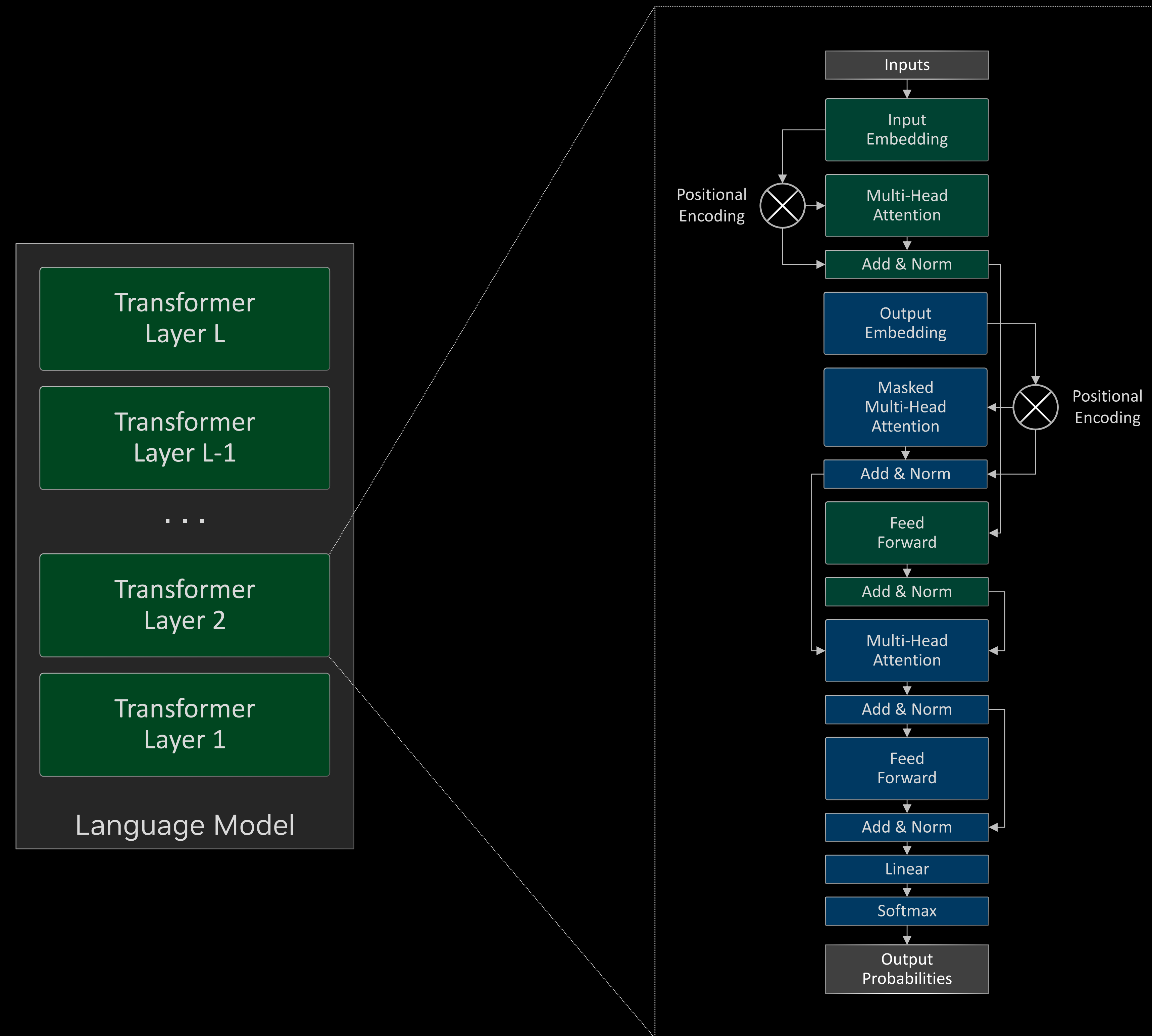


CPU sees increased latency and decreased bandwidth when reading from GPU memory

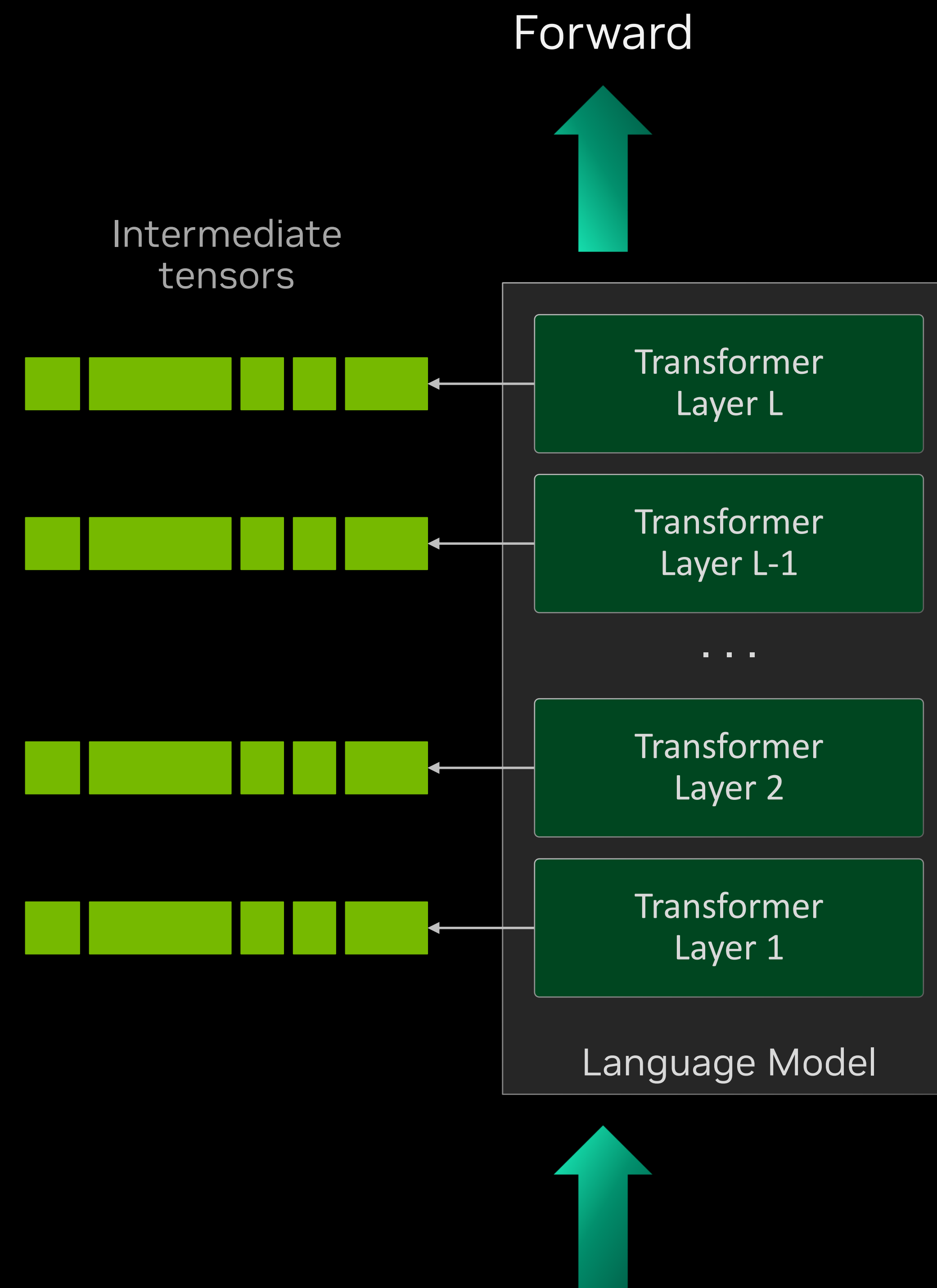
Enabling Unified CPU+GPU Execution of Strongly Couple Systems



Supervised Fine Tuning

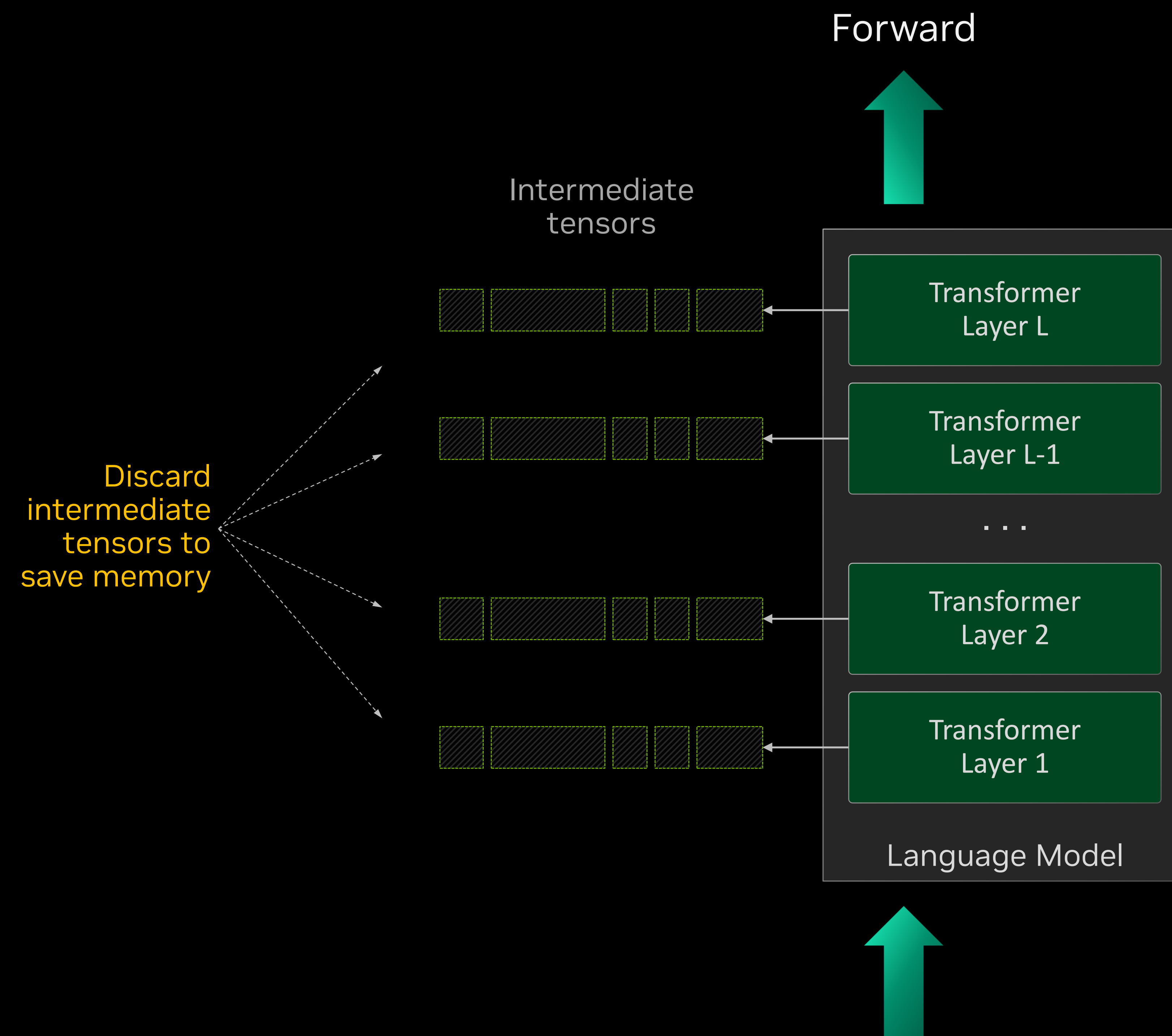


Supervised Fine Tuning



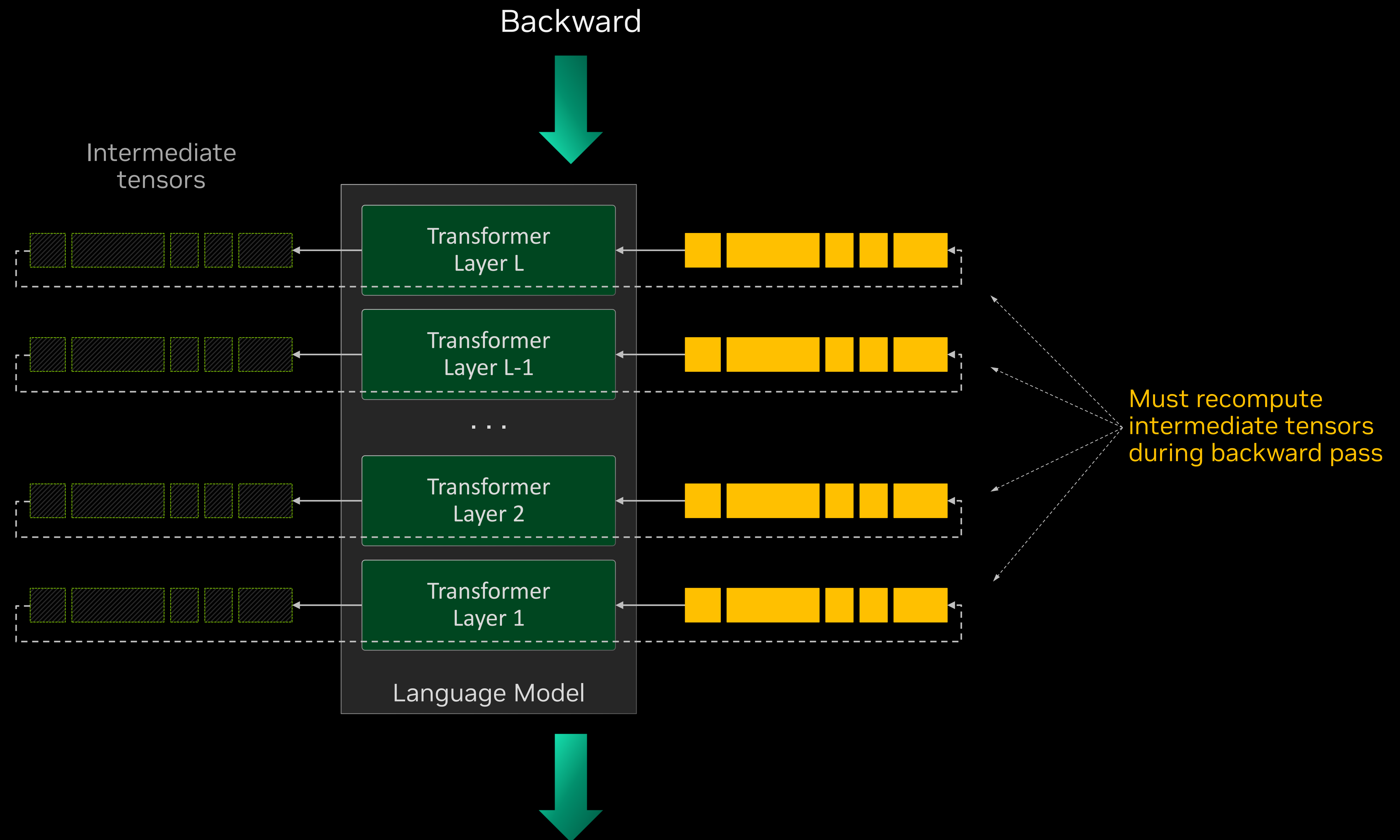
Supervised Fine Tuning

Recompute activations for more efficient memory use



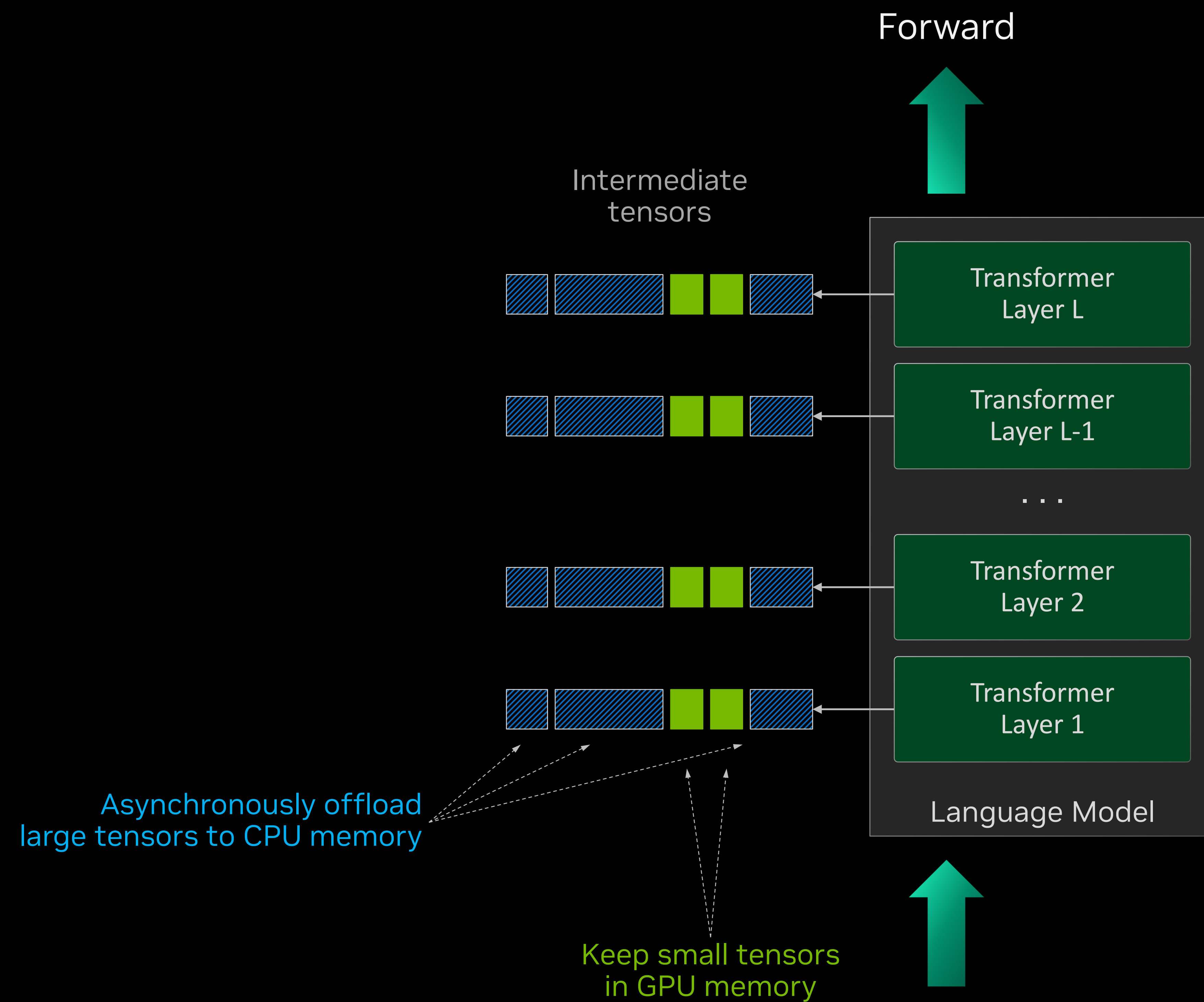
Supervised Fine Tuning

Recompute activations for more efficient memory use



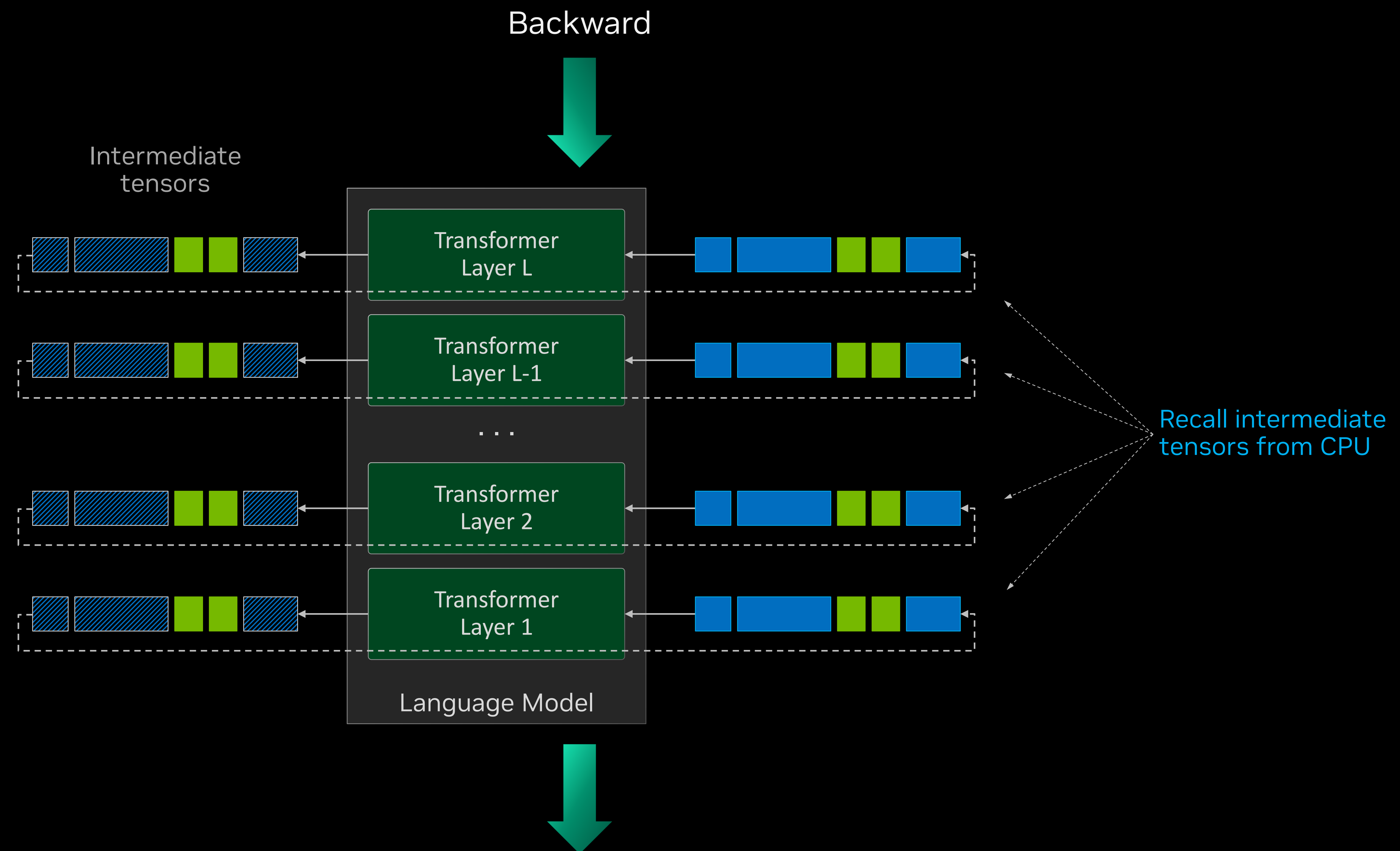
Supervised Fine Tuning

Asynchronous offloading to CPU memory



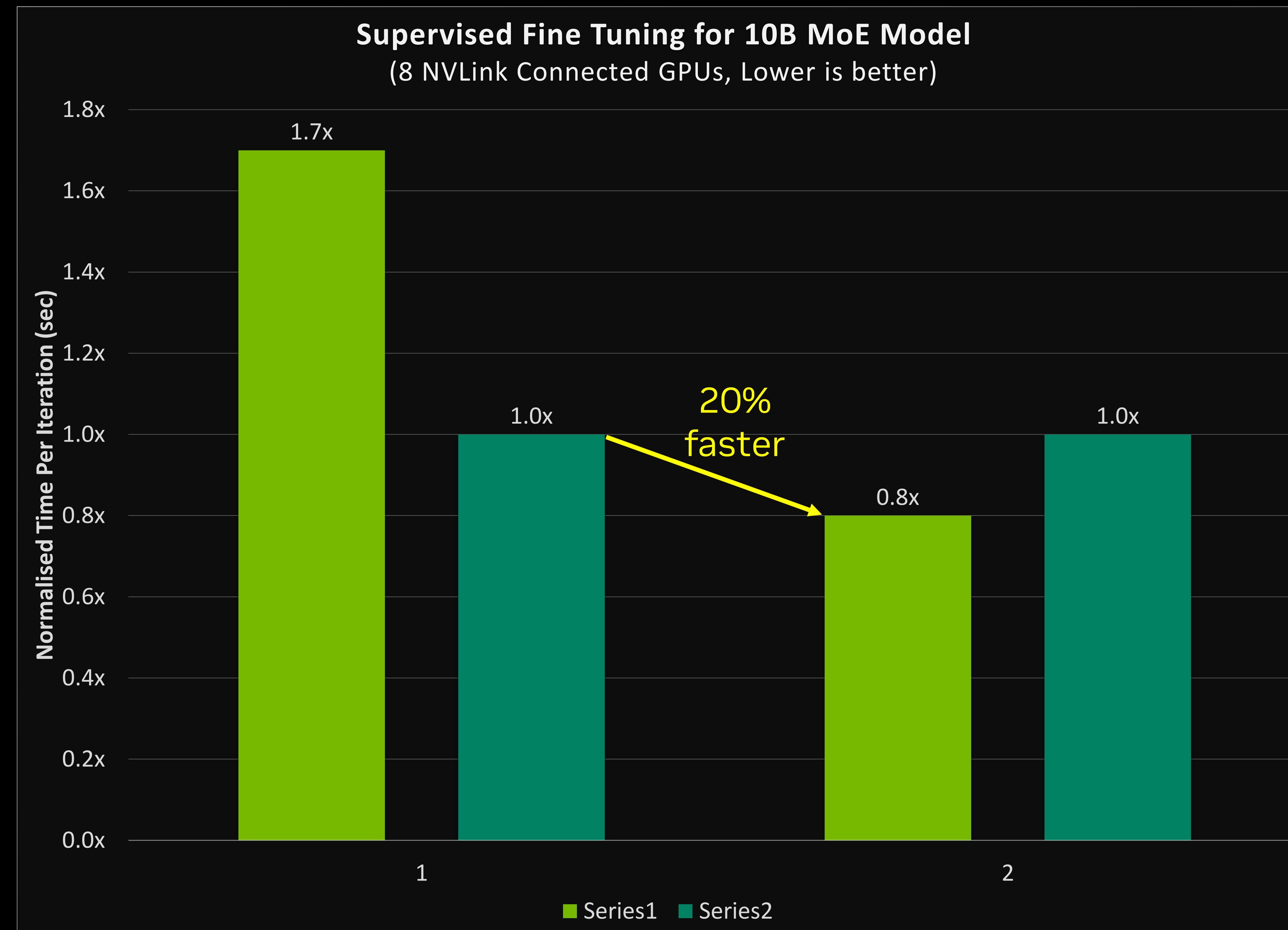
Supervised Fine Tuning

Asynchronous offloading to CPU memory

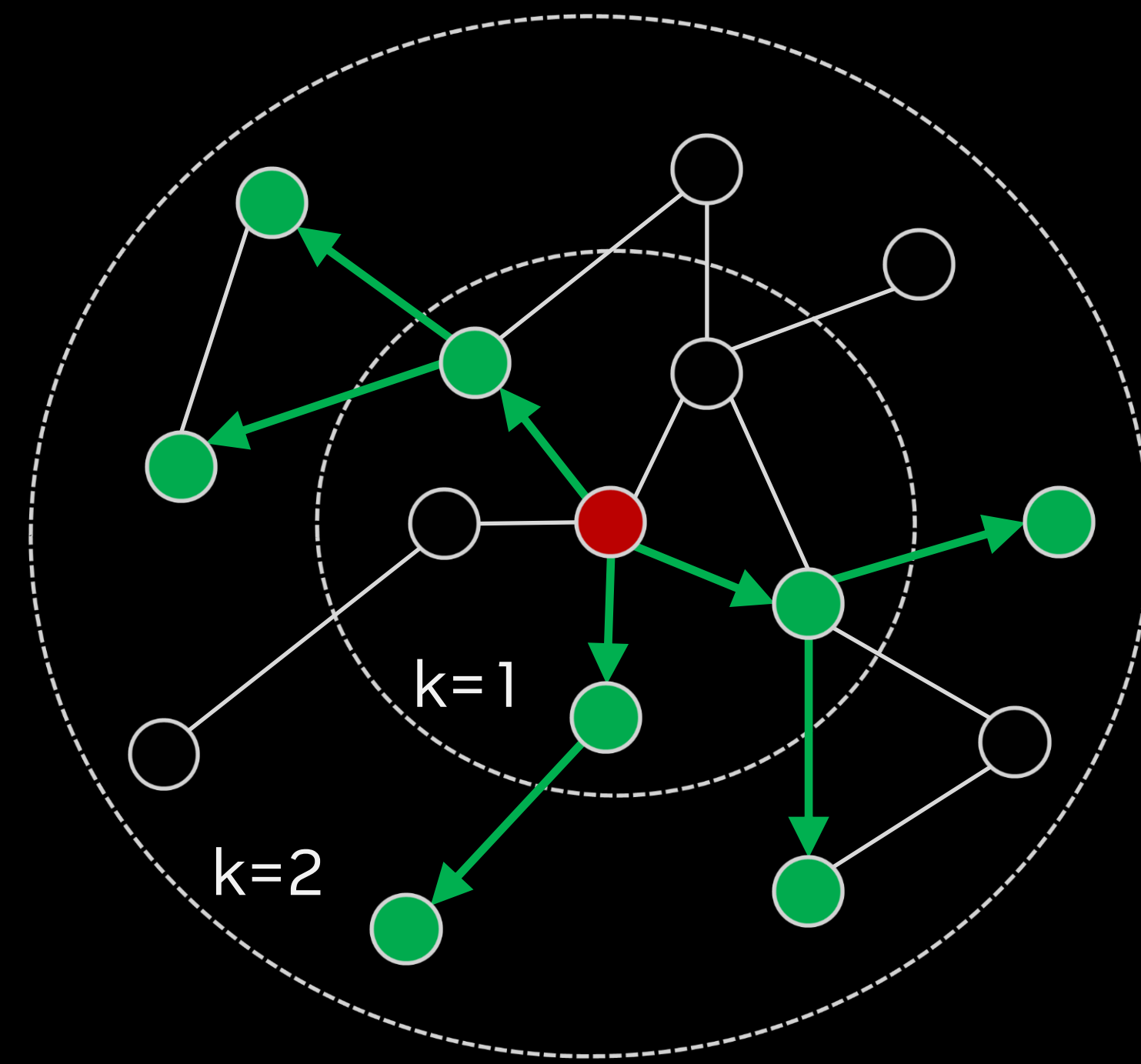


Supervised Fine Tuning

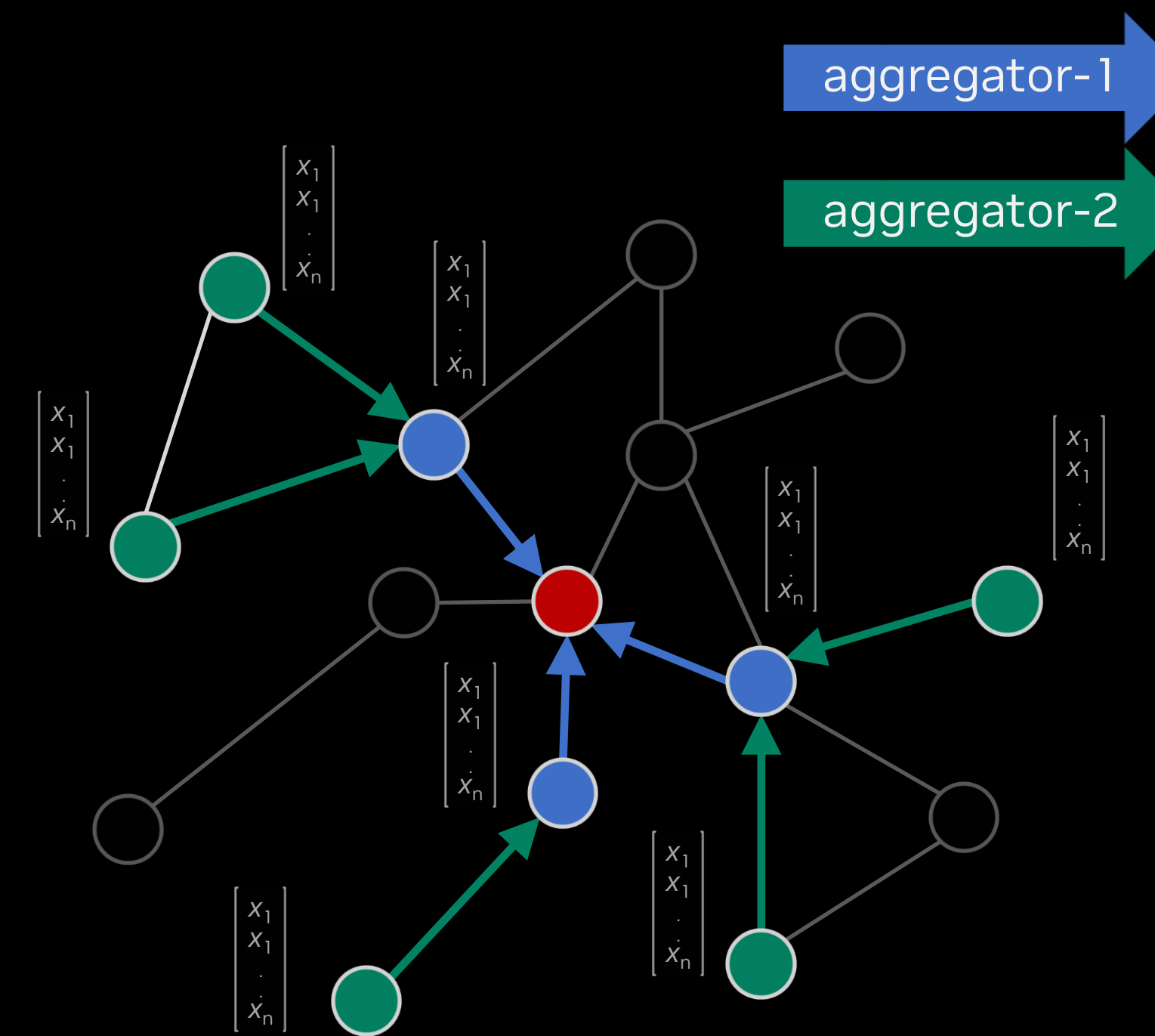
Asynchronous offloading to CPU memory in Megatron LM



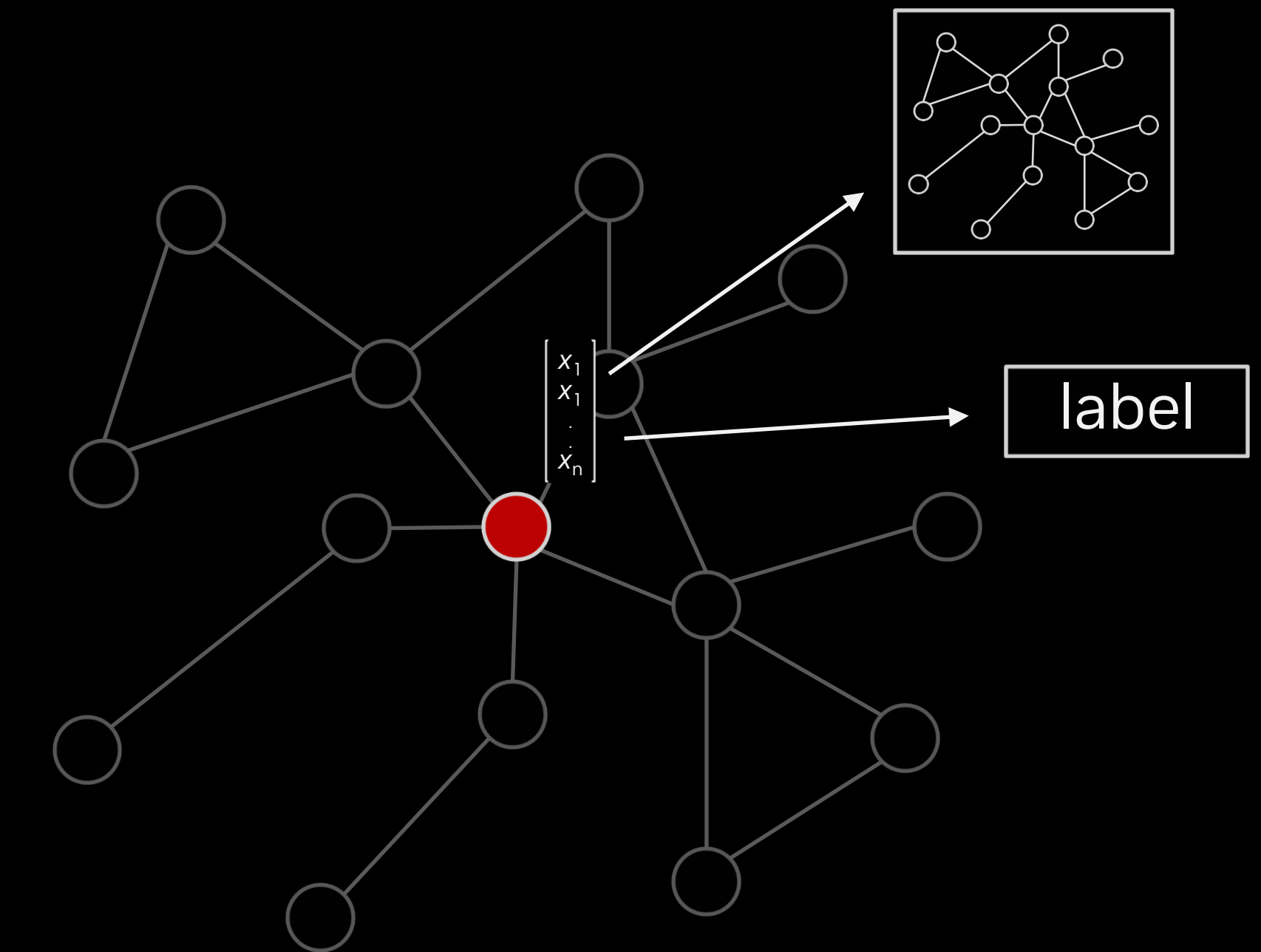
GraphSAGE Model



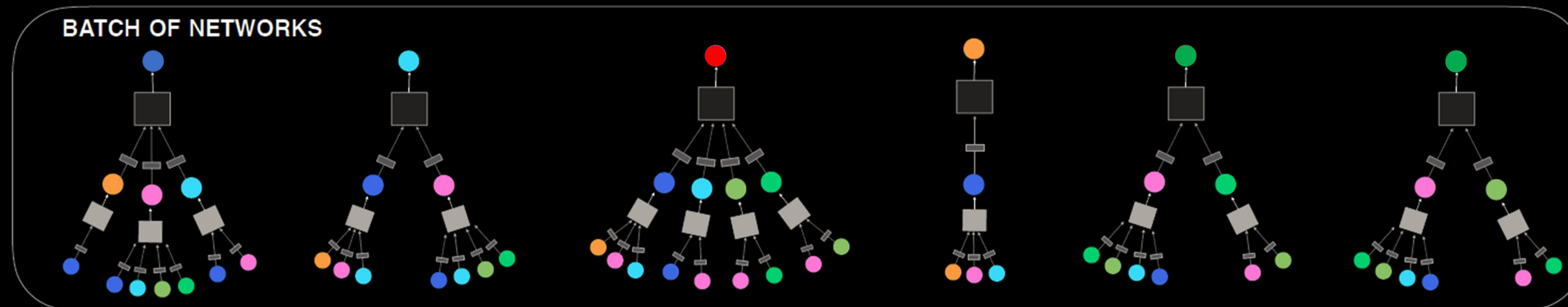
1. Sample neighbourhood



2. Aggregate feature information from neighbours



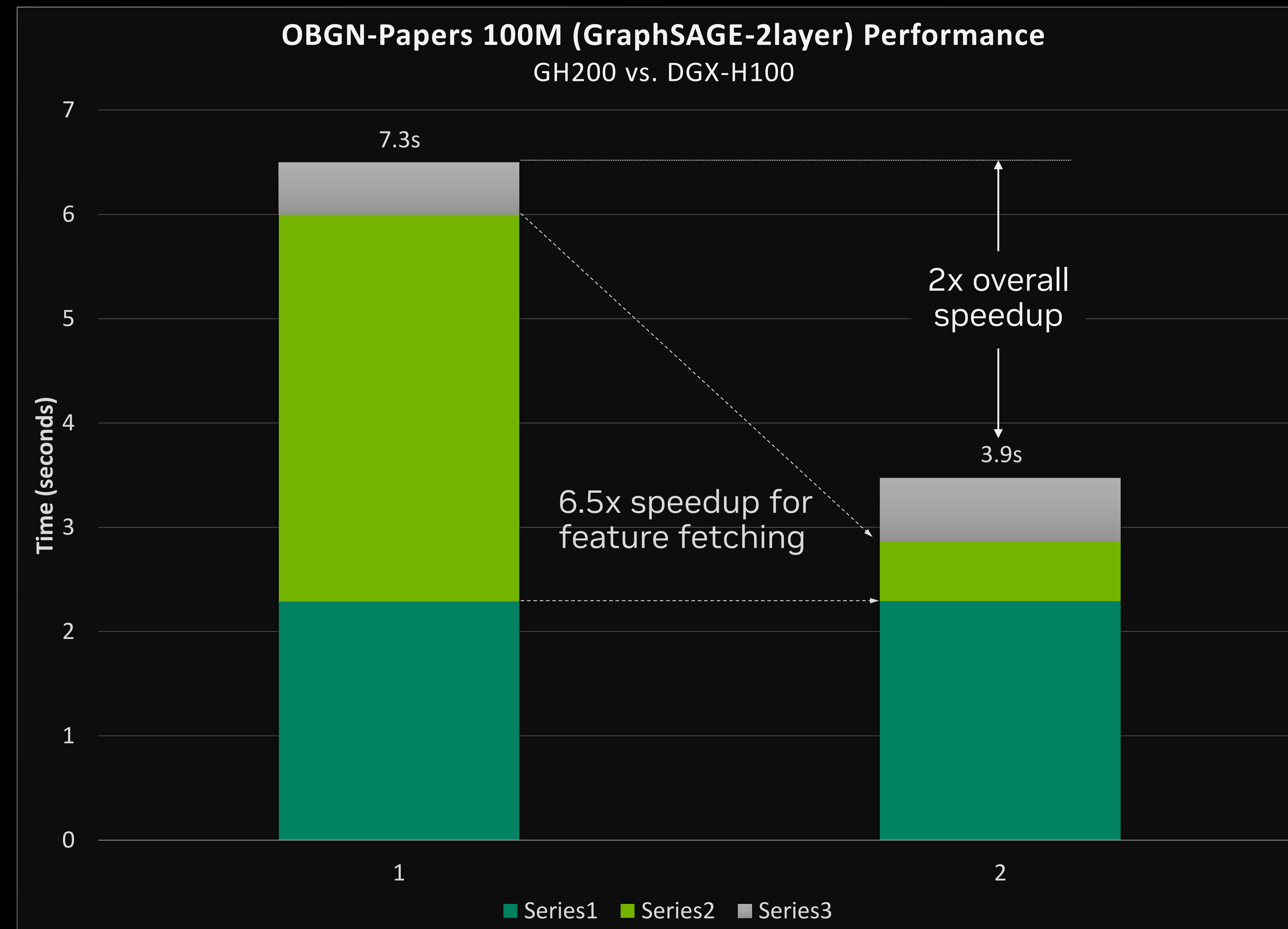
3. Predict graph context and label using aggregated information



Graph node type and connectivity form different minibatches in the model

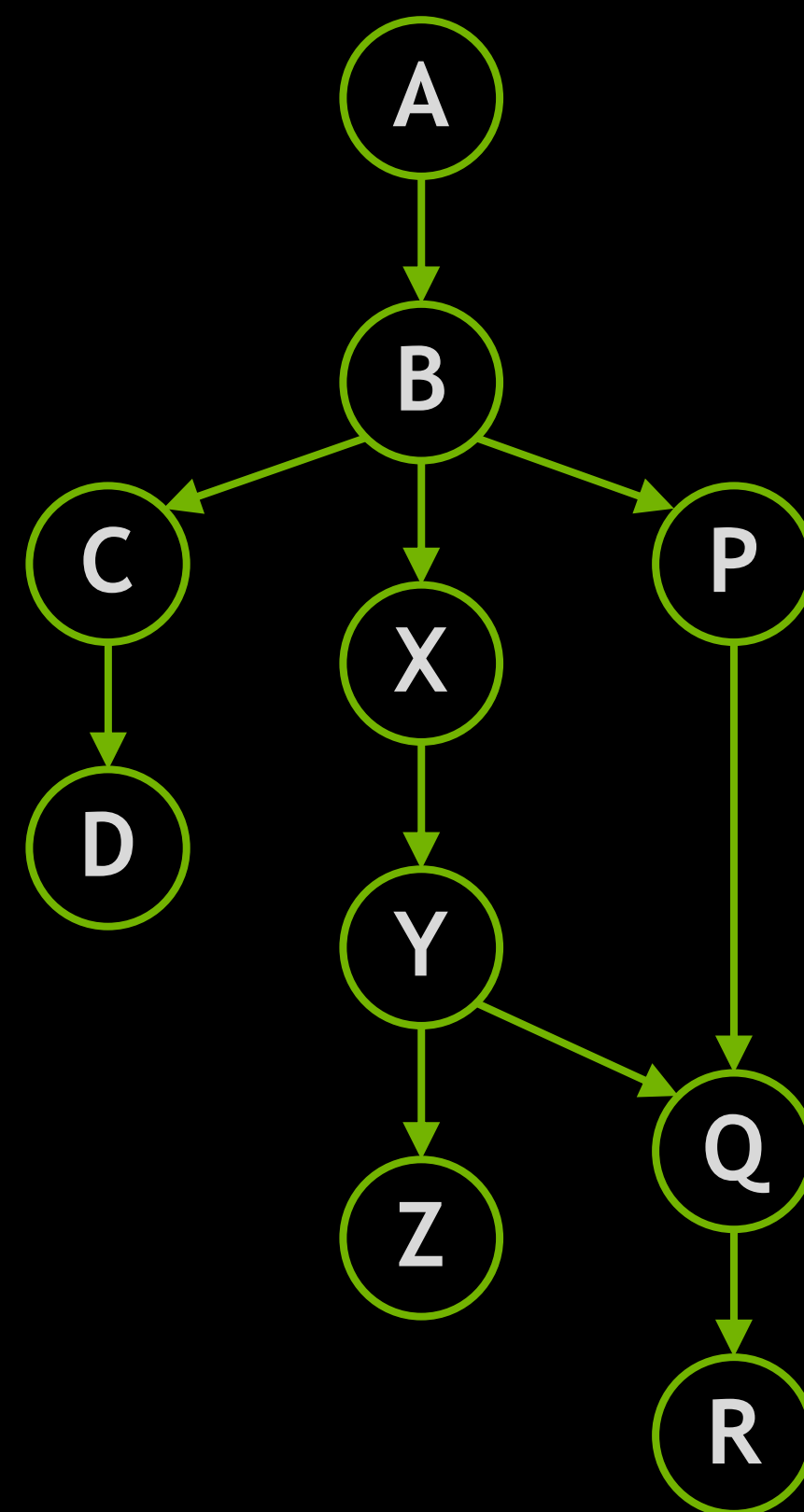
[Large-Scale Graph GNN Training Accelerated With cuGraph \[S61197\]](#)

Benefit For Feature Storage in GH200 Unified Memory

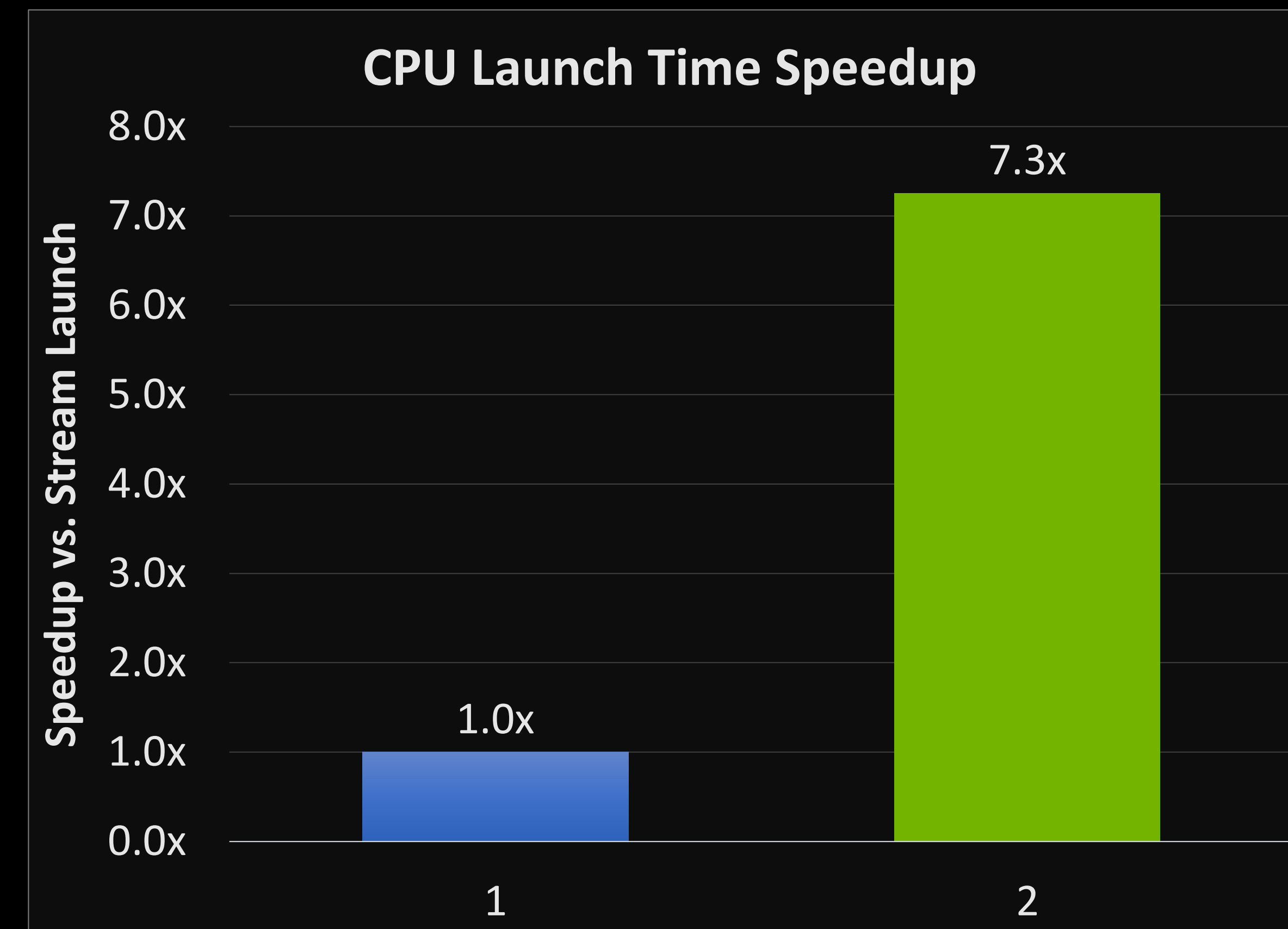


Task Graph Execution

If you can declare your work ahead of time, then use it many times, it's really worth it



Define once, launch
many times



Launch of a pre-built graph is **7-8x faster**
than launch of the same kernels into a stream

“Declarative” task-graph description of work

1. Declare whole workflow up-front
2. **One** launch call submits these **10 kernels at once**
(maximum graph size is effectively unlimited)

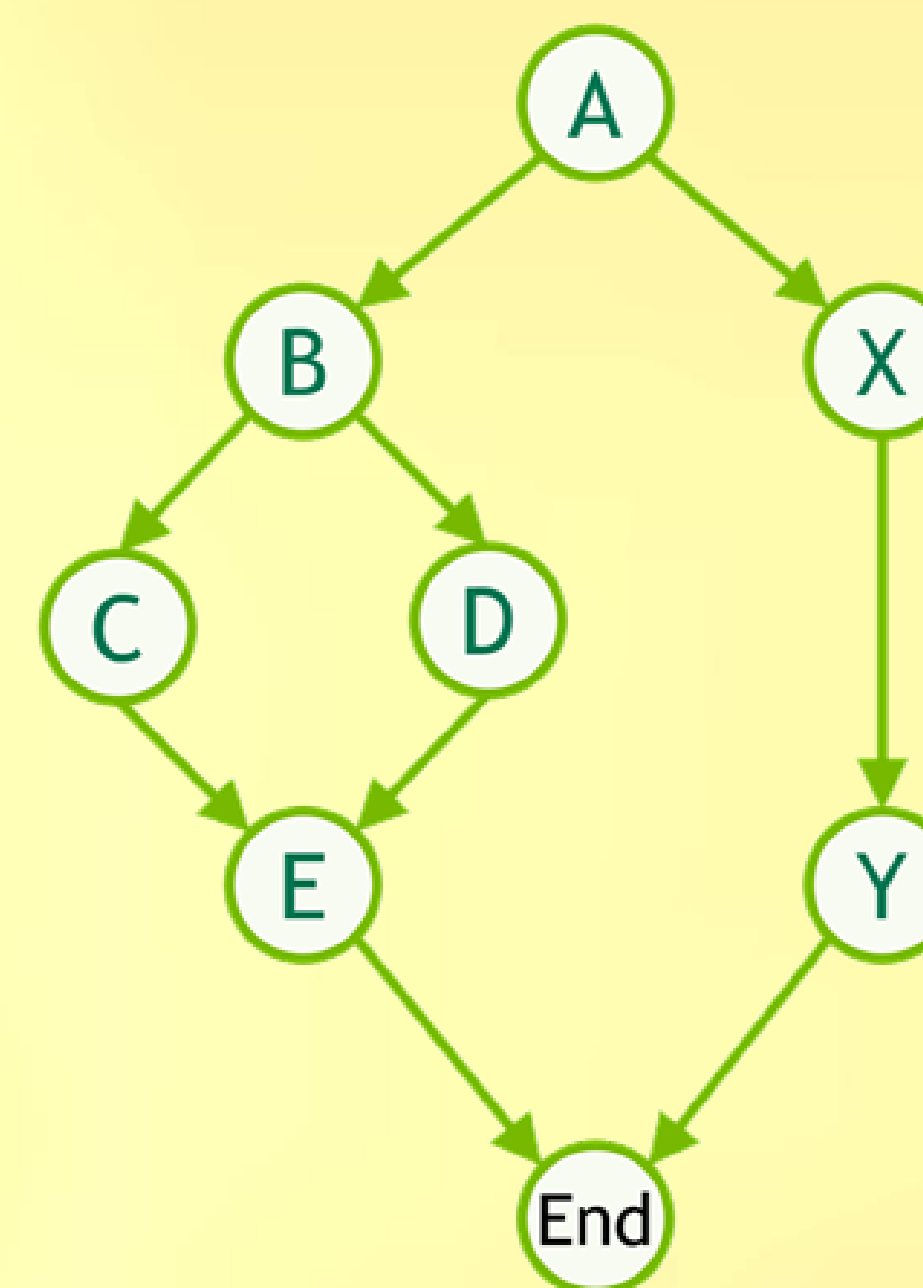
The Evolution of a 2018 Programming Model

WHAT IS A TASK GRAPH?

Sequence of operations, connected by dependencies.

Each node is a Task. Task types include:

- Execution Kernel or function (**host** or **device**)
- Control Conditional branch in graph (if/else)
- Resource Memory allocation / movement, etc.
- Sub-Graph Graphs are hierarchical



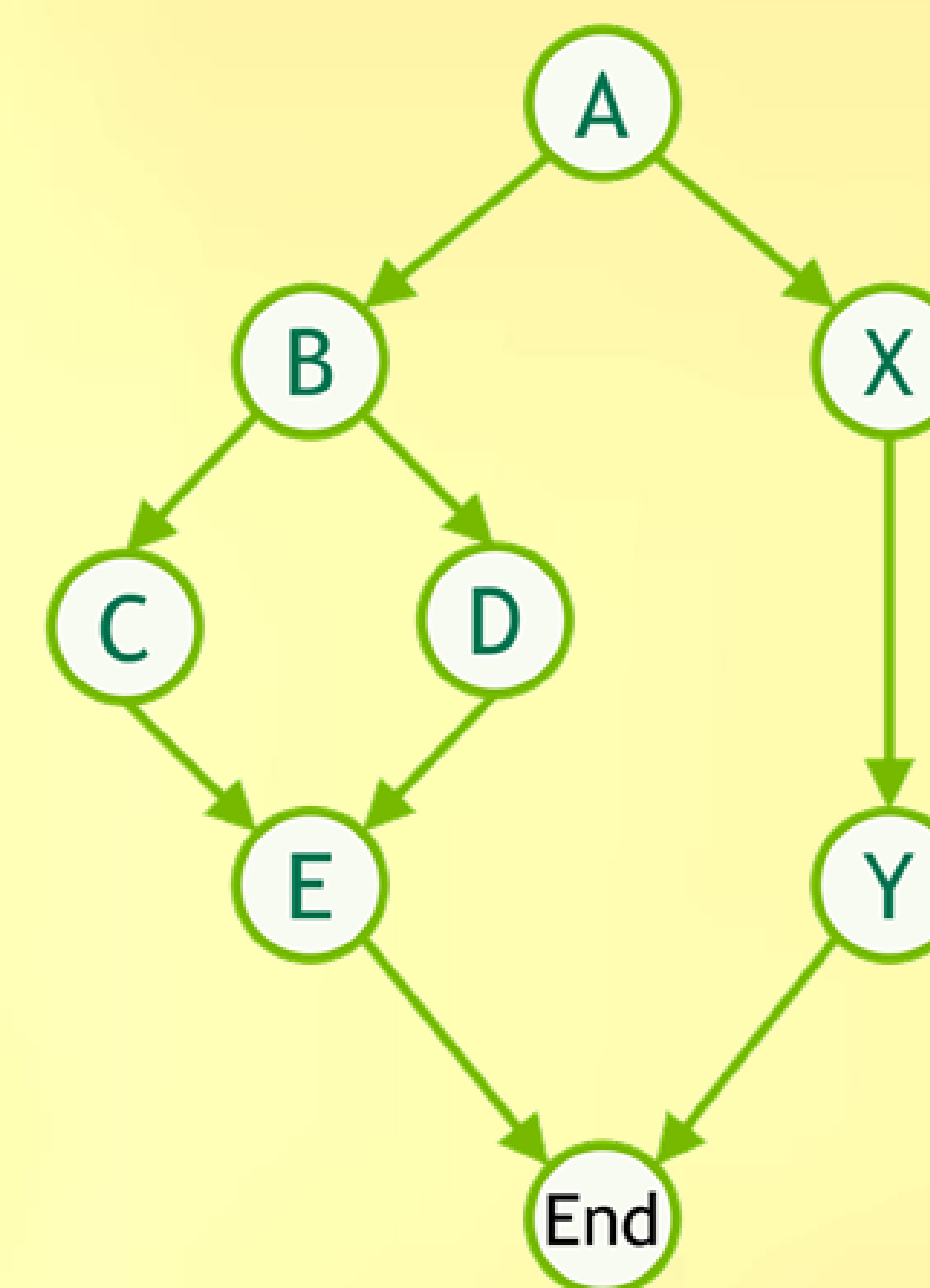
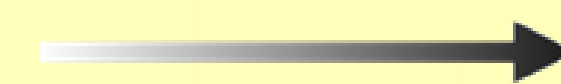
The Evolution of a 2018 Programming Model

TASK GRAPH PROPERTIES

Re-usable

Graphs can be generated once and executed repeatedly

```
for(int i=0; i<1000; i++) {  
    launch_graph( G );  
}
```



Dynamic Control Flow CUDA Graphs: Conditional Nodes

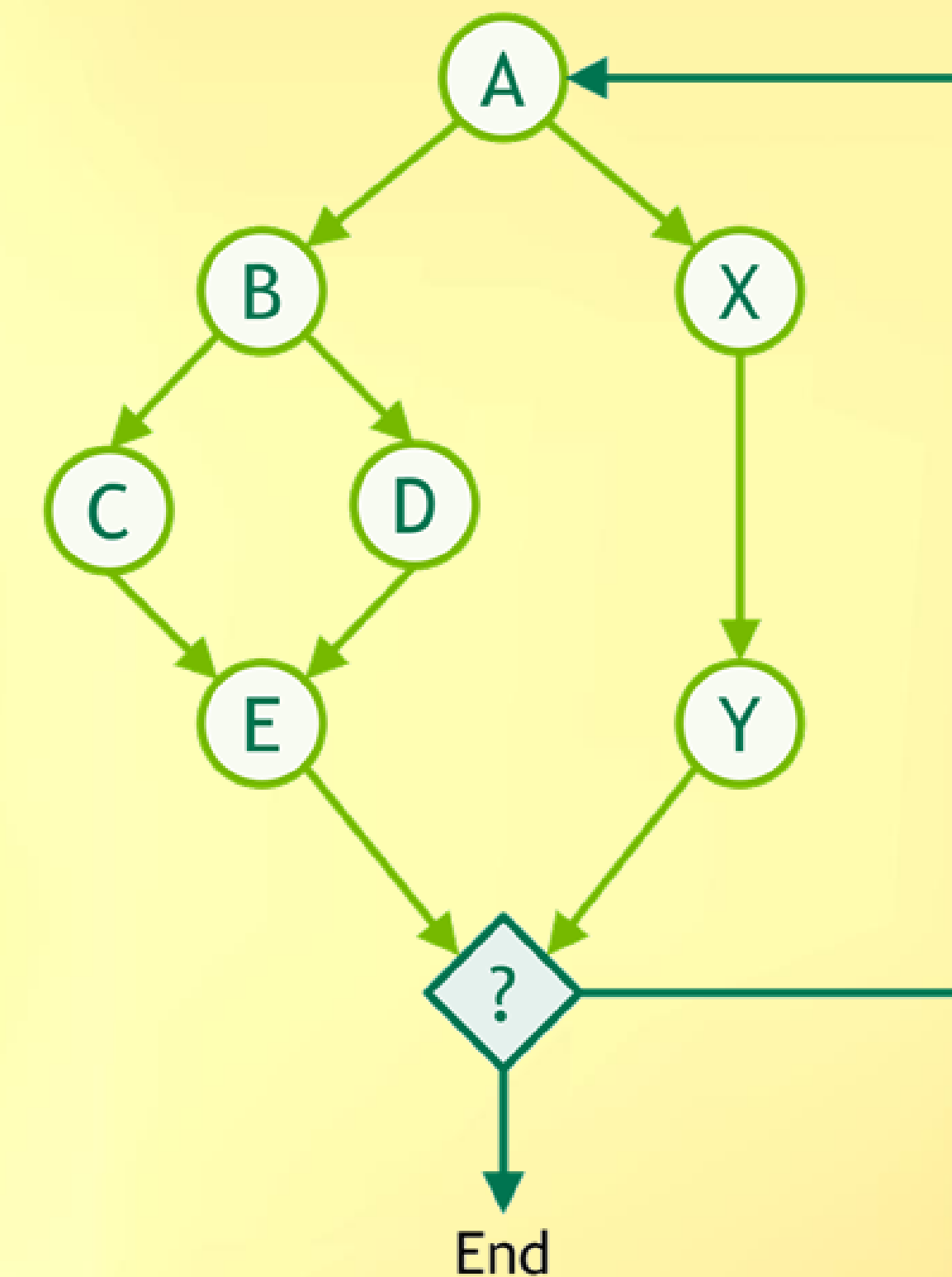
TASK GRAPH PROPERTIES

Re-usable

Graphs can be generated once and executed repeatedly

Cyclic

“Iterate until converged” is a common pattern
(e.g. Stochastic Gradient Descent)



NVIDIA CONFIDENTIAL - NDA REQUIRED

5 NVIDIA

Dynamic Control Flow CUDA Graphs: Conditional Nodes

TASK GRAPH PROPERTIES

Re-usable

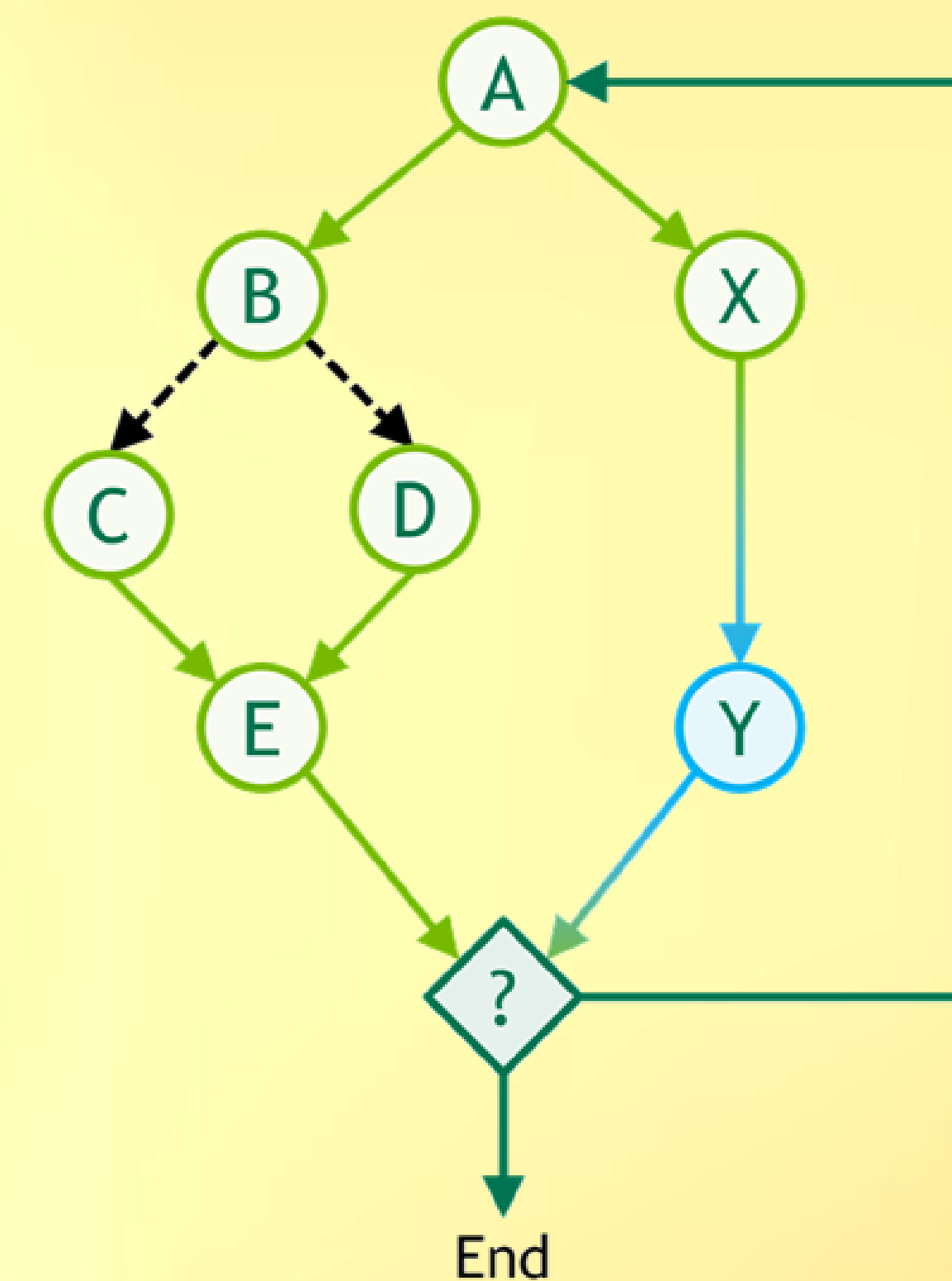
Graphs can be generated once and executed repeatedly

Cyclic

“Iterate until converged” is a common pattern
(e.g. Stochastic Gradient Descent)

Dynamic

Run-time, data-dependent execution
(e.g. B may be followed by either C or D)



Dynamic Control Flow CUDA Graphs: Conditional Nodes

TASK GRAPH PROPERTIES

Re-usable

Graphs can be generated once and executed repeatedly

Cyclic

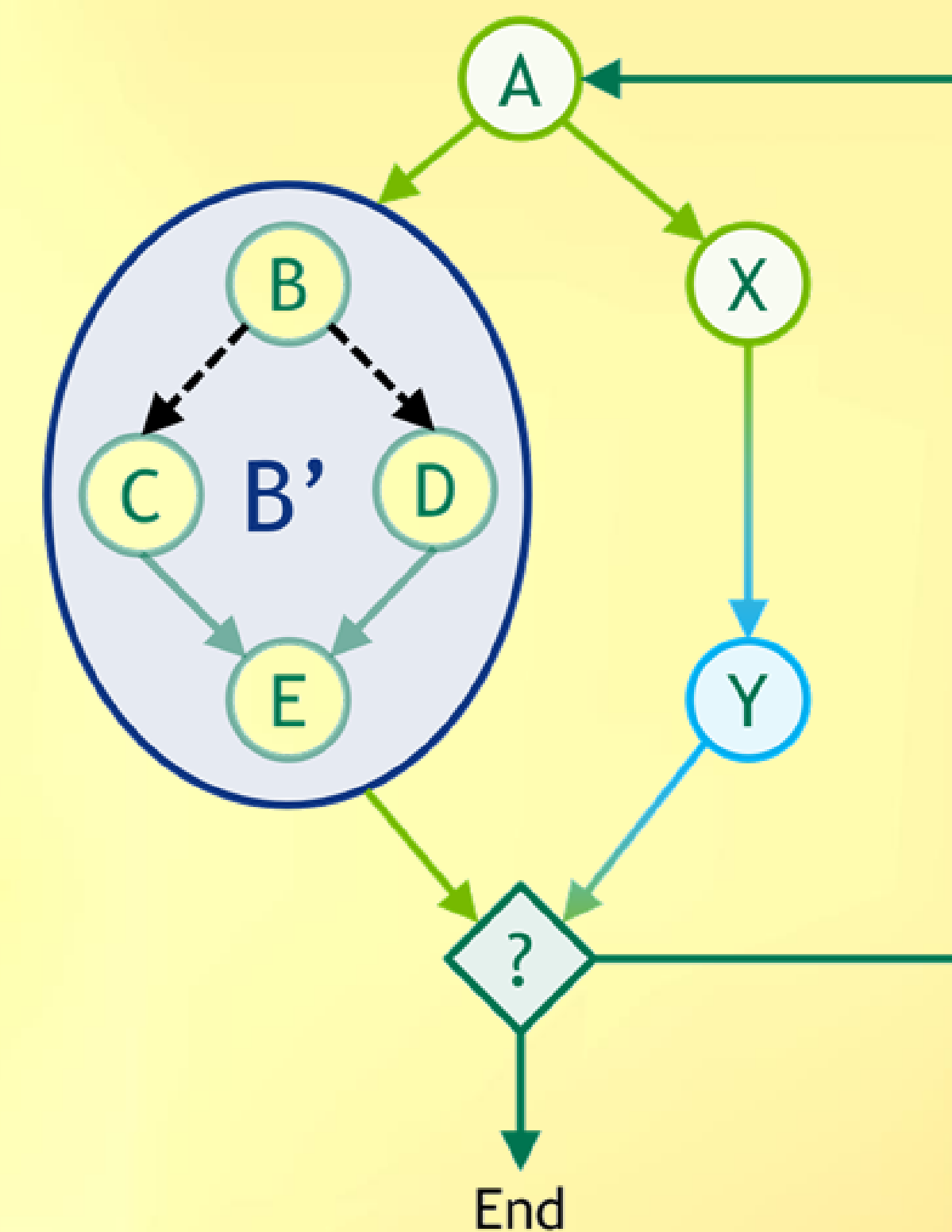
“Iterate until converged” is a common pattern
(e.g. Stochastic Gradient Descent)

Dynamic

Run-time, data-dependent execution
(e.g. B may be followed by either C or D)

Hierarchical

A task may be an entire sub-graph - a graph of graphs
(e.g. BCDE may be graph B')



NVIDIA CONFIDENTIAL - NDA REQUIRED

7 NVIDIA

Data-Dependent Execution

“Iterate until converged” is an almost universal pattern

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations

Data-Dependent Execution

“Iterate until converged” is an almost universal pattern

Main
loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations

Data-Dependent Execution

“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)      Loop  
                                     body  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations

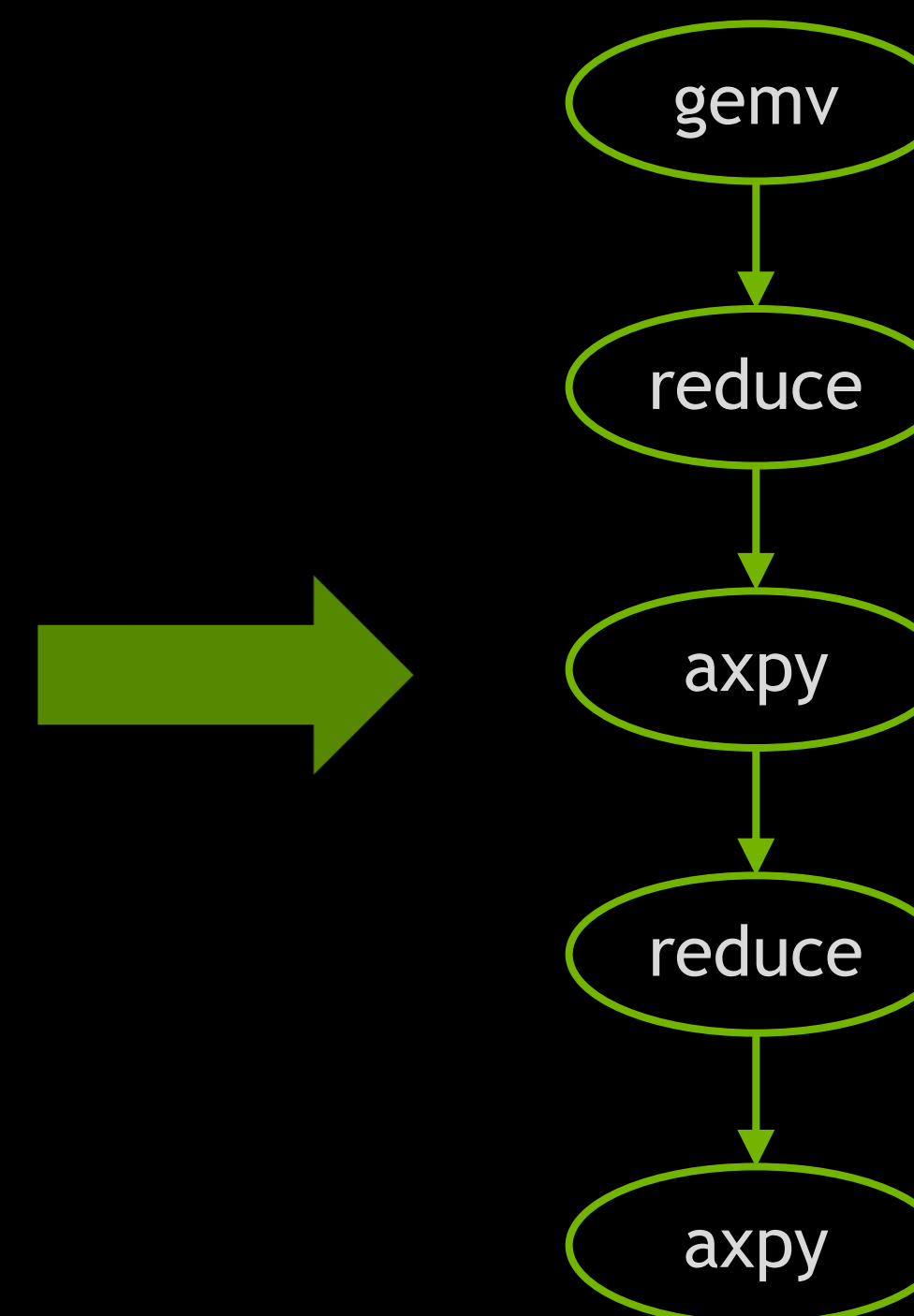
Data-Dependent Execution

“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)      Loop body  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



Convert loop body
into a task graph

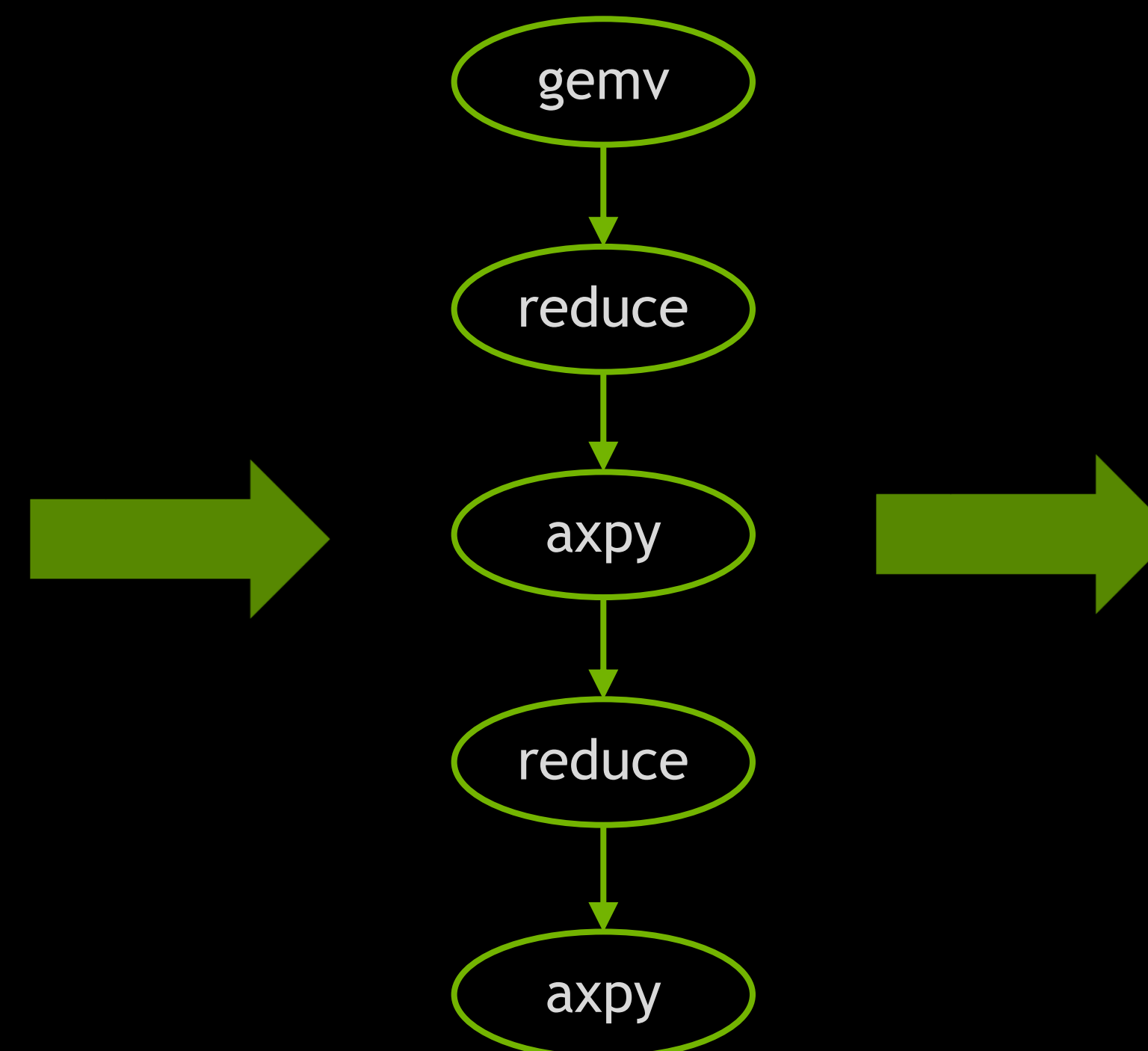
Data-Dependent Execution

“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)      Loop body  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



Convert loop body
into a task graph

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        launch_graph(A, x, r, p, rsold)  
  
        while(residual > 1e-8)  
    return x  
end
```

Task graph launch optimizes loop body execution

Data-Dependent Execution

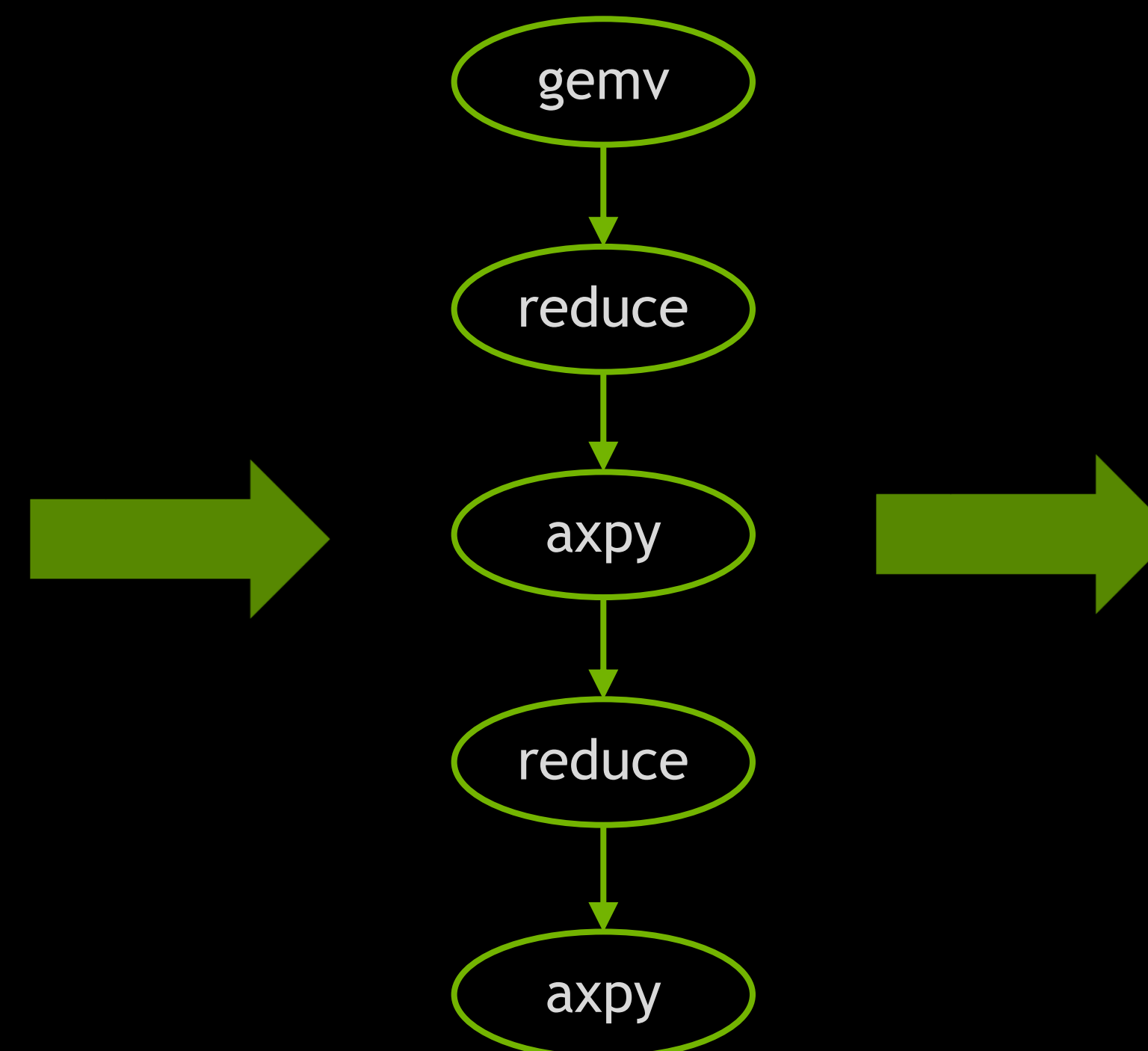
“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Loop body

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



Convert loop body
into a task graph

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        launch_graph(A, x, r, p, rsold)  
  
        while(residual > 1e-8)  
    return x  
end
```

Task graph launch optimizes loop body execution

Data-Dependent Execution

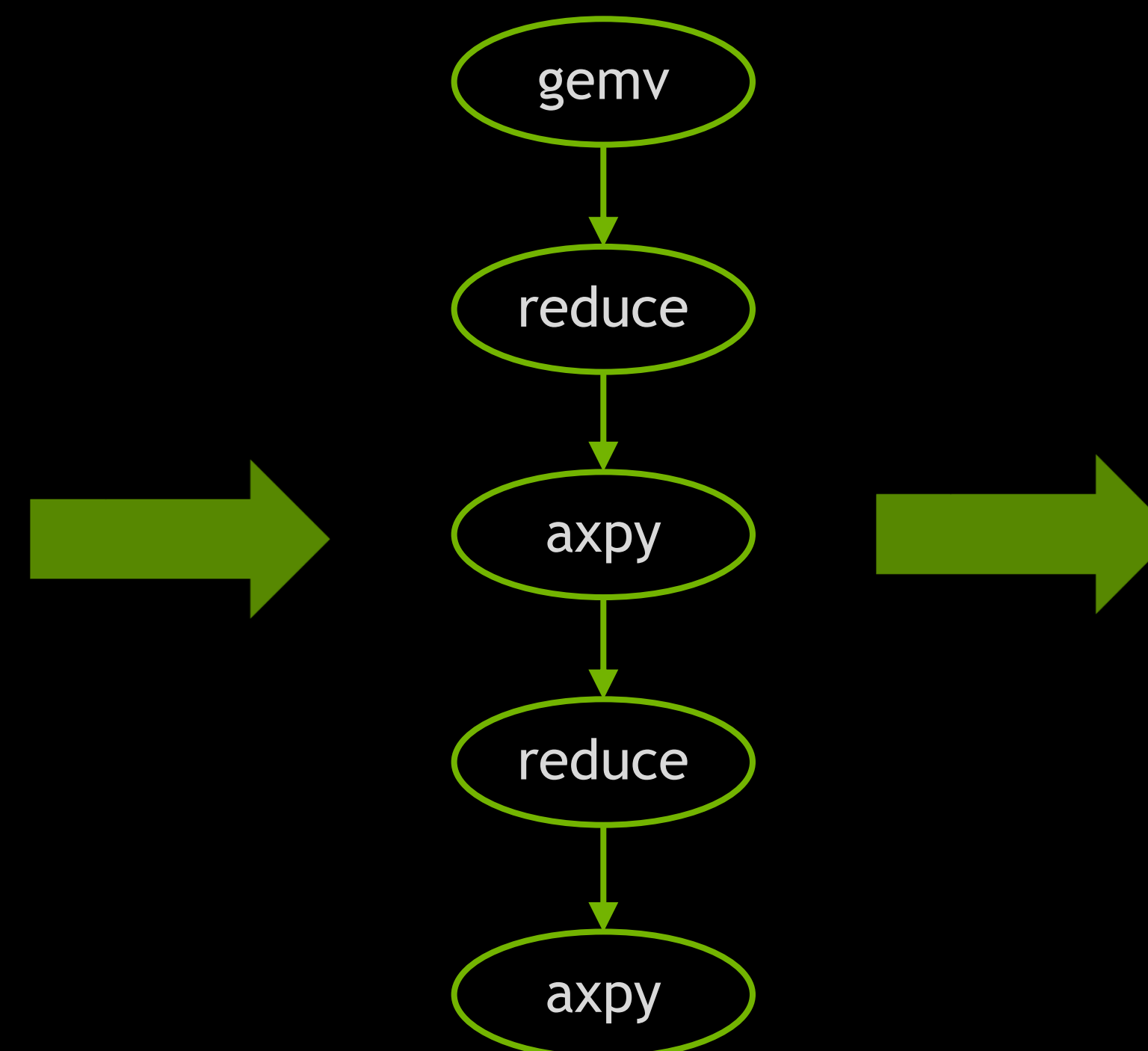
“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while (residual > 1e-8)  
  
    return x  
end
```

Loop body

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



Convert loop body
into a task graph

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        launch_graph(A, x, r, p, rsold)  
        synchronize()  
        residual = copy_from_gpu()  
    while (residual > 1e-8)  
  
    return x  
end
```

Task graph launch optimizes loop body execution
but then must return to CPU to evaluate loop again

Data-Dependent Execution **On The GPU**

“Iterate until converged” is an almost universal pattern

Main
loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations

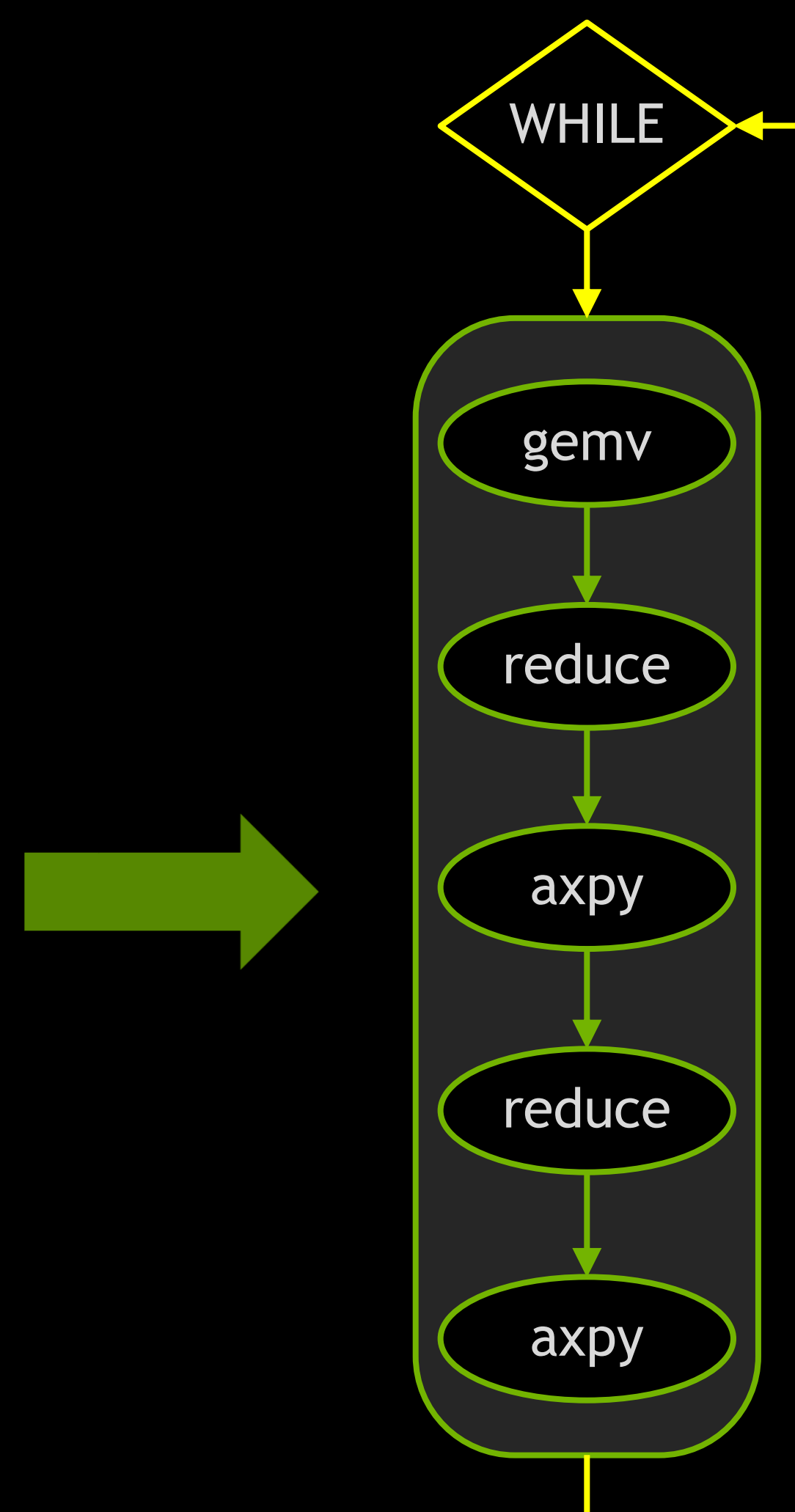
Data-Dependent Execution **On The GPU**

“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



Convert **entire loop** into
a task graph using
new “**conditional nodes**”

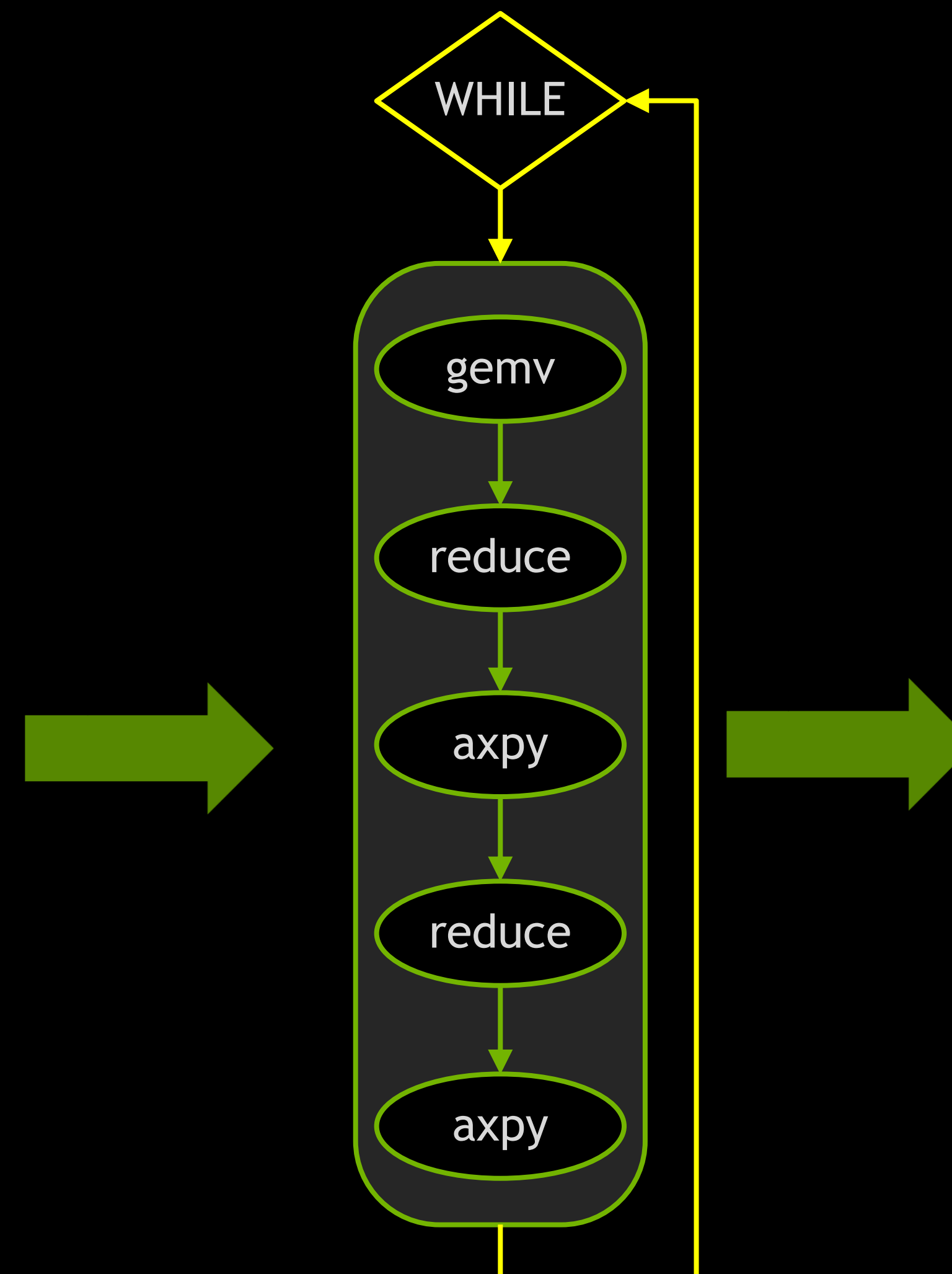
Data-Dependent Execution **On The GPU**

“Iterate until converged” is an almost universal pattern

Main loop

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    do  
        Ap = A * p  
        alpha = rsold / (Ap * transpose(p))  
        x = x + (alpha * p)  
        r = r - (alpha * Ap)  
        rsnew = r * transpose(r)  
  
        residual = sqrt(rsnew)  
        p = r + (rsnew / rsold) * p  
        rsold = rsnew  
    while(residual > 1e-8)  
  
    return x  
end
```

Pseudo-code of the conjugate gradient algorithm
for solving systems of linear equations



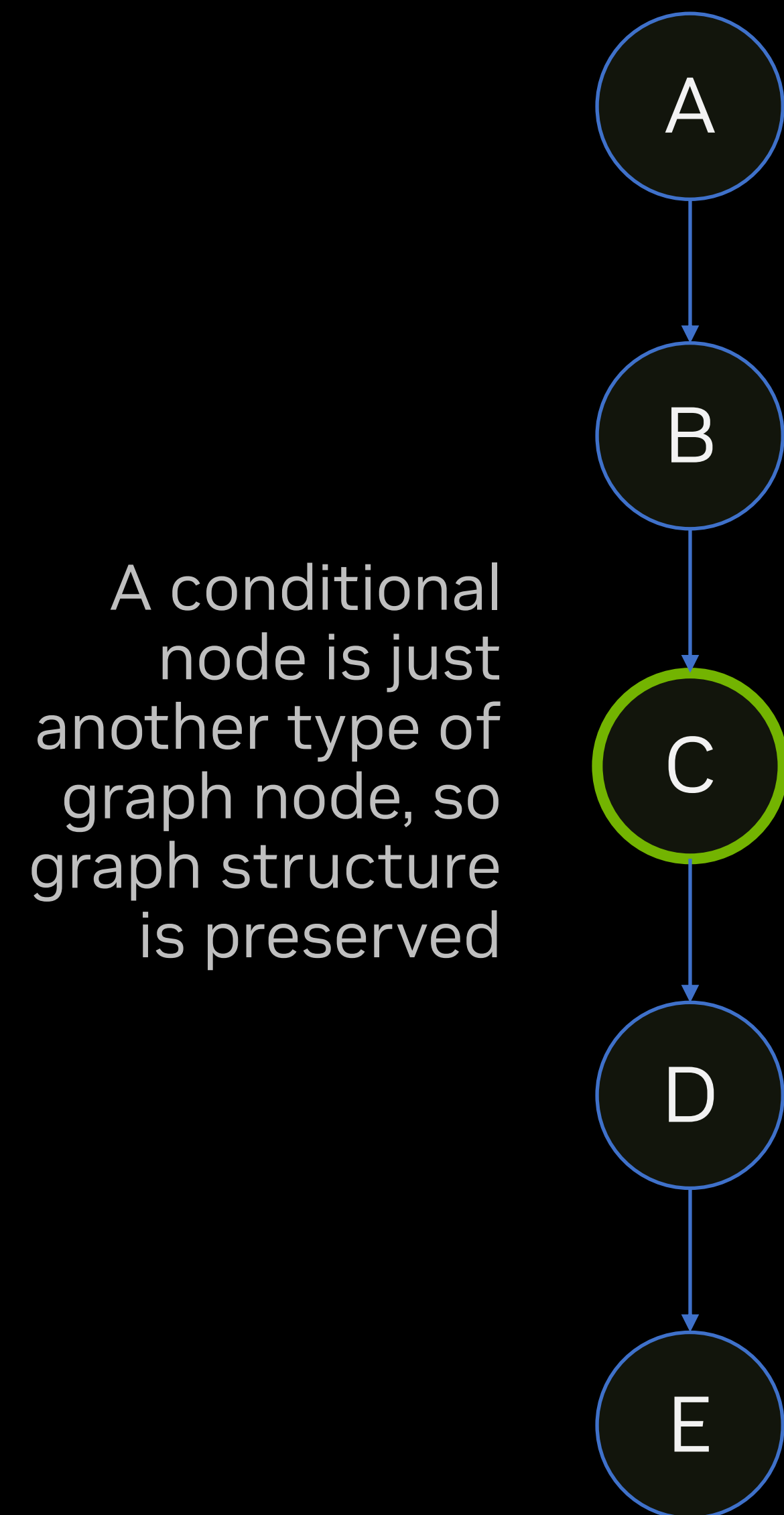
Convert **entire loop** into
a task graph using
new “**conditional nodes**”

```
function ConjugateGradient(A, b, x):  
    r = b - A * x  
    p = r  
    rsold = r * transpose(r)  
  
    launch_conditional_graph(A, x, r, p,  
                             rsold, 1e-8)  
  
    return x  
end
```

Entire CG solve runs to completion on GPU
using just one single graph launch

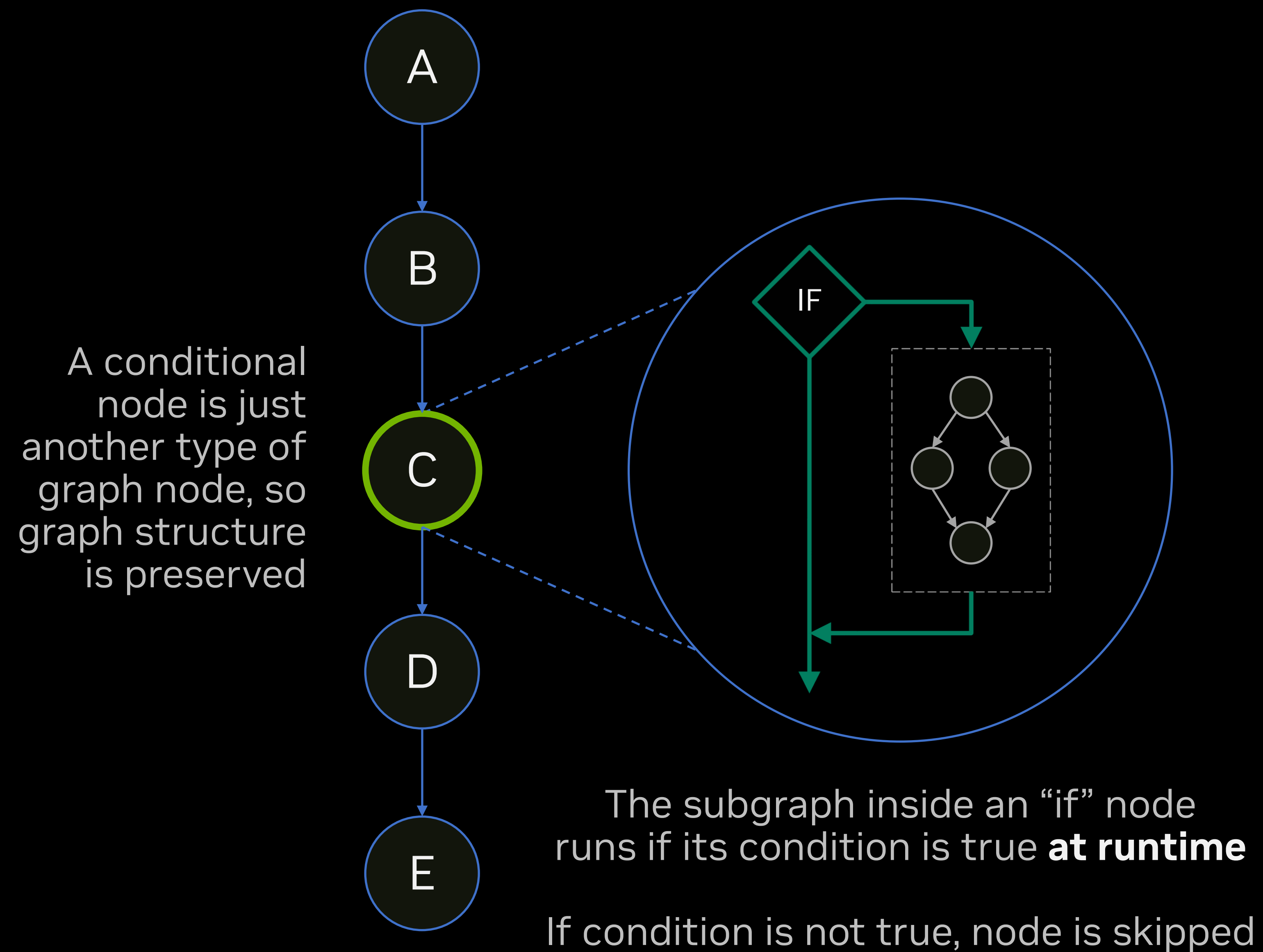
Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs `if()` or `while()` a condition is true



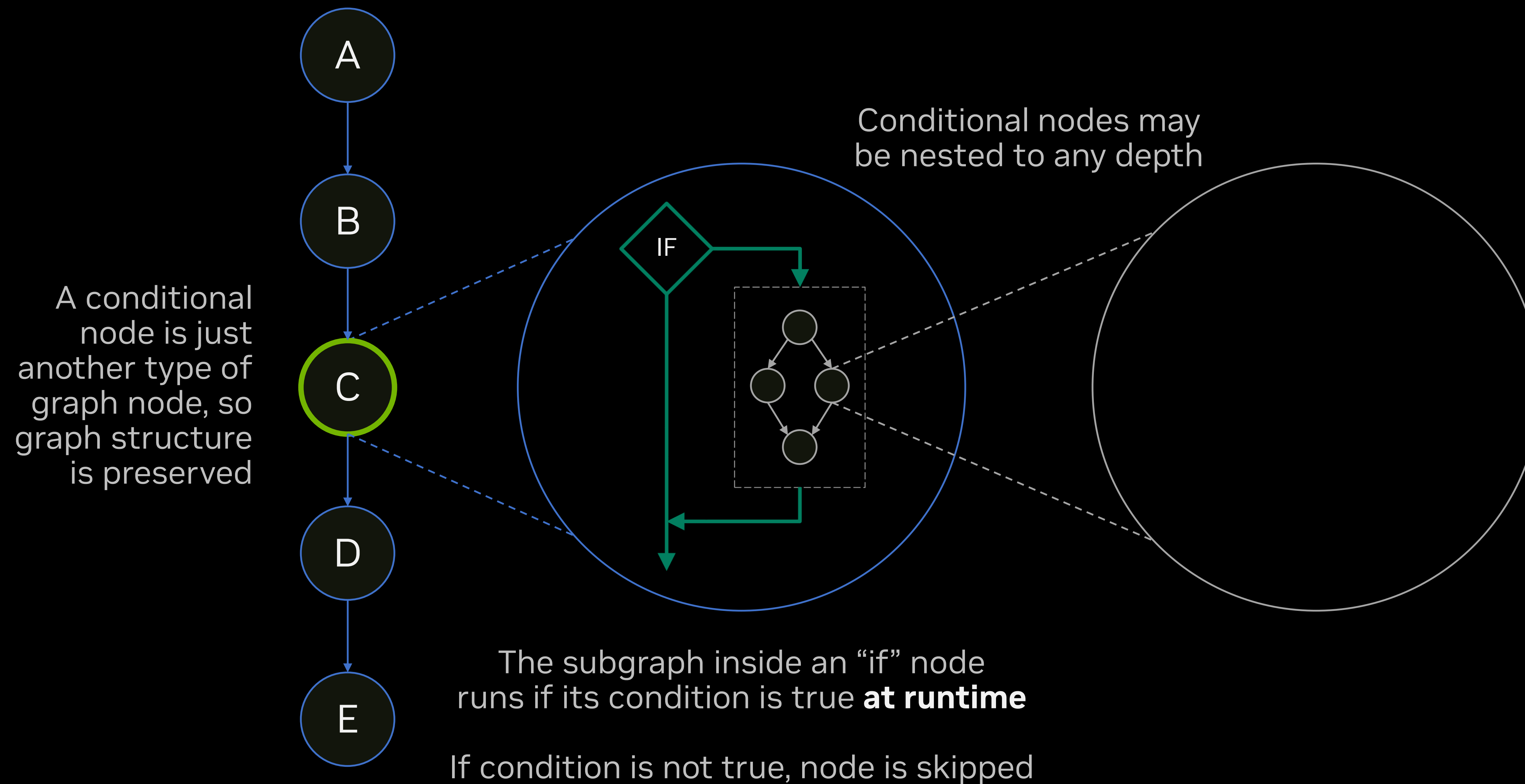
Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs if() or while() a condition is true



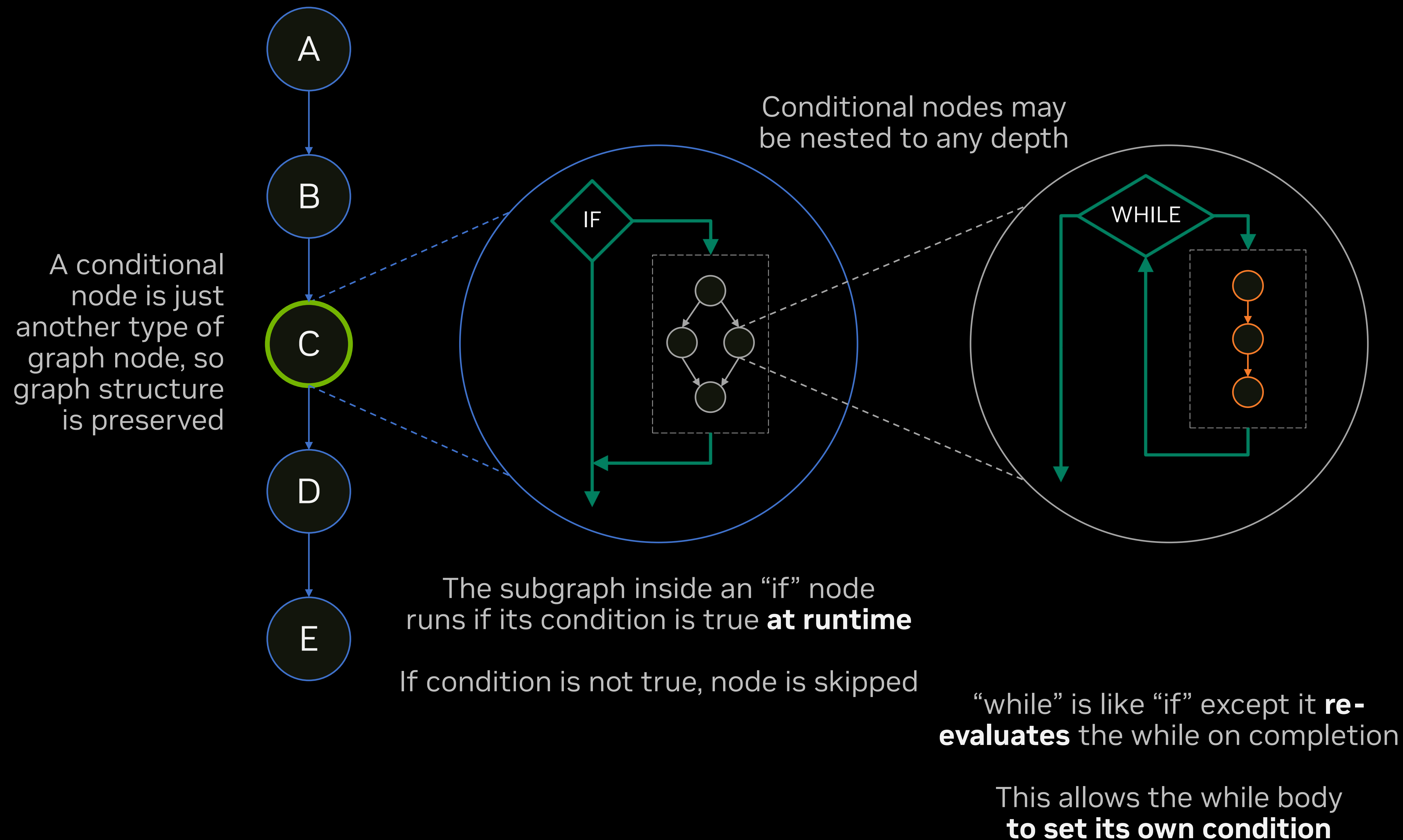
Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs if() or while() a condition is true



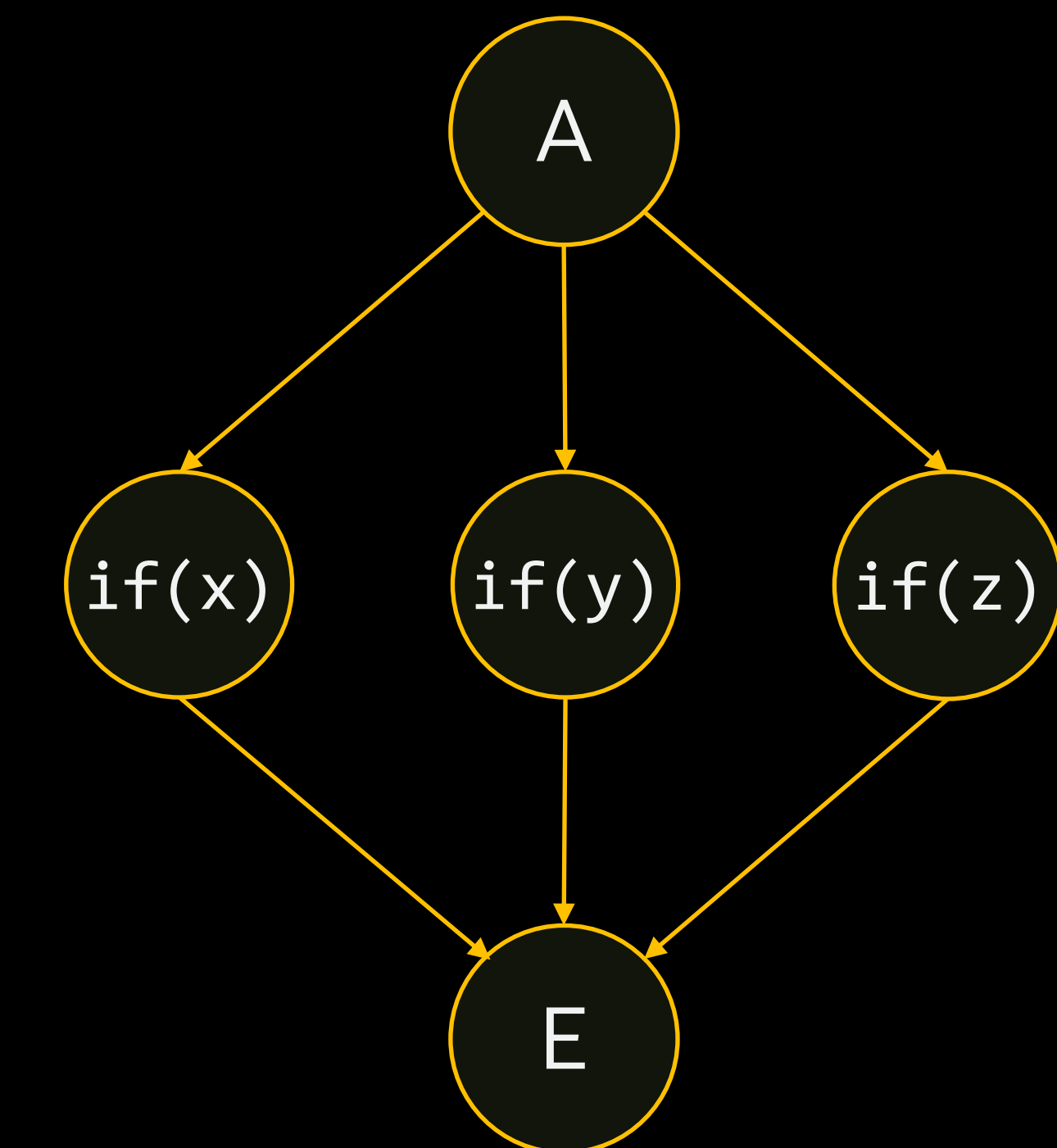
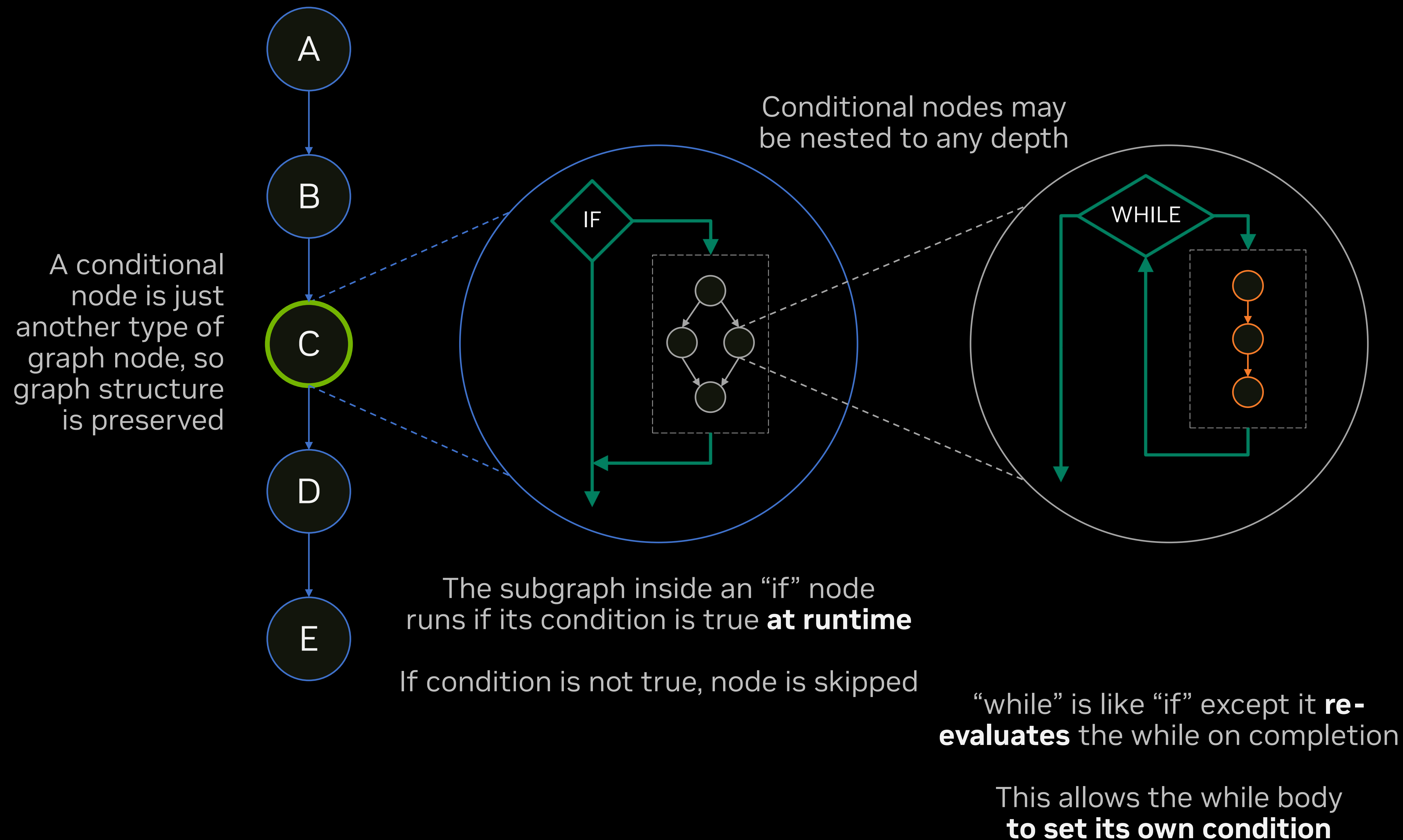
Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs if() or while() a condition is true



Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs `if()` or `while()` a condition is true



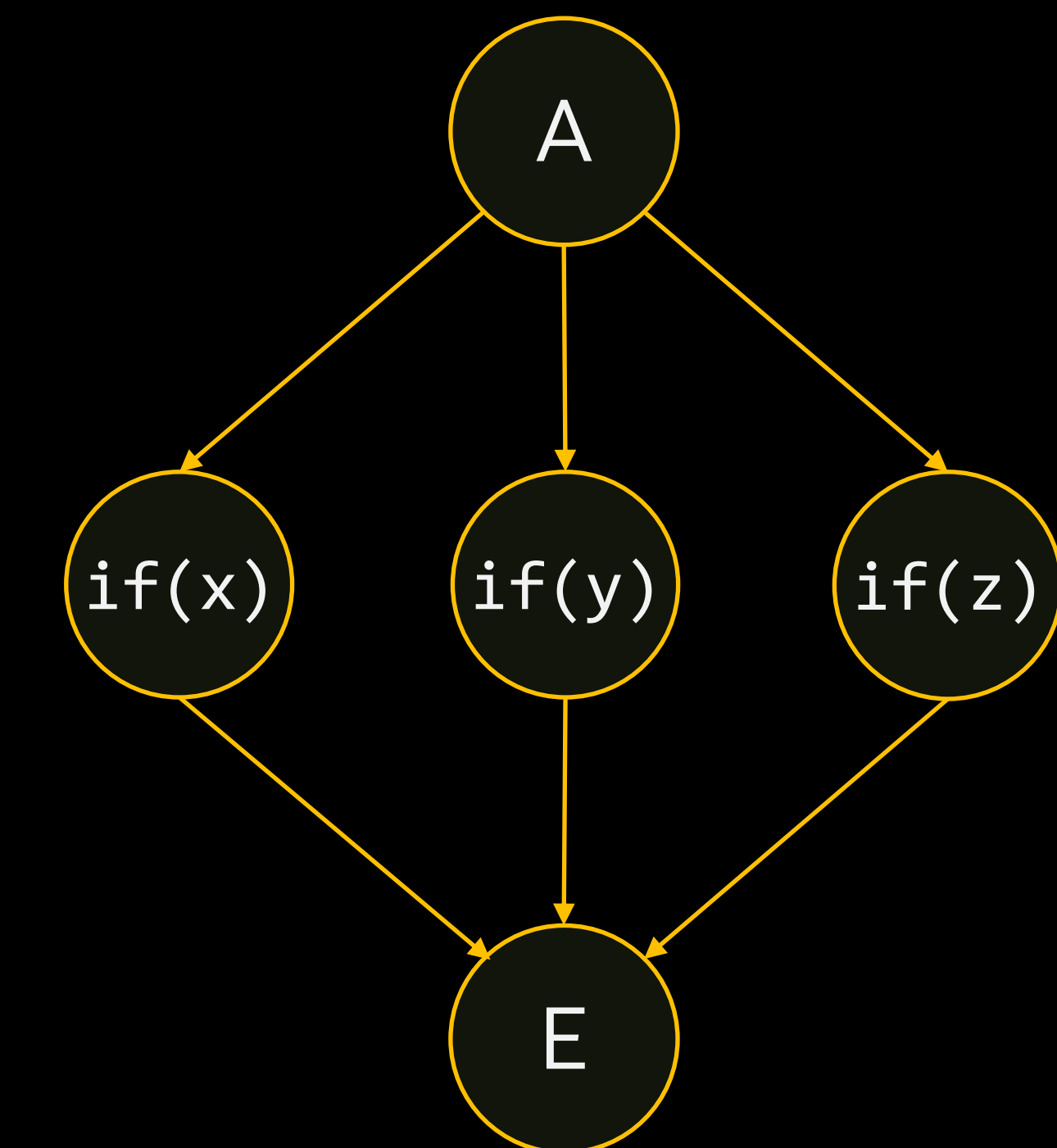
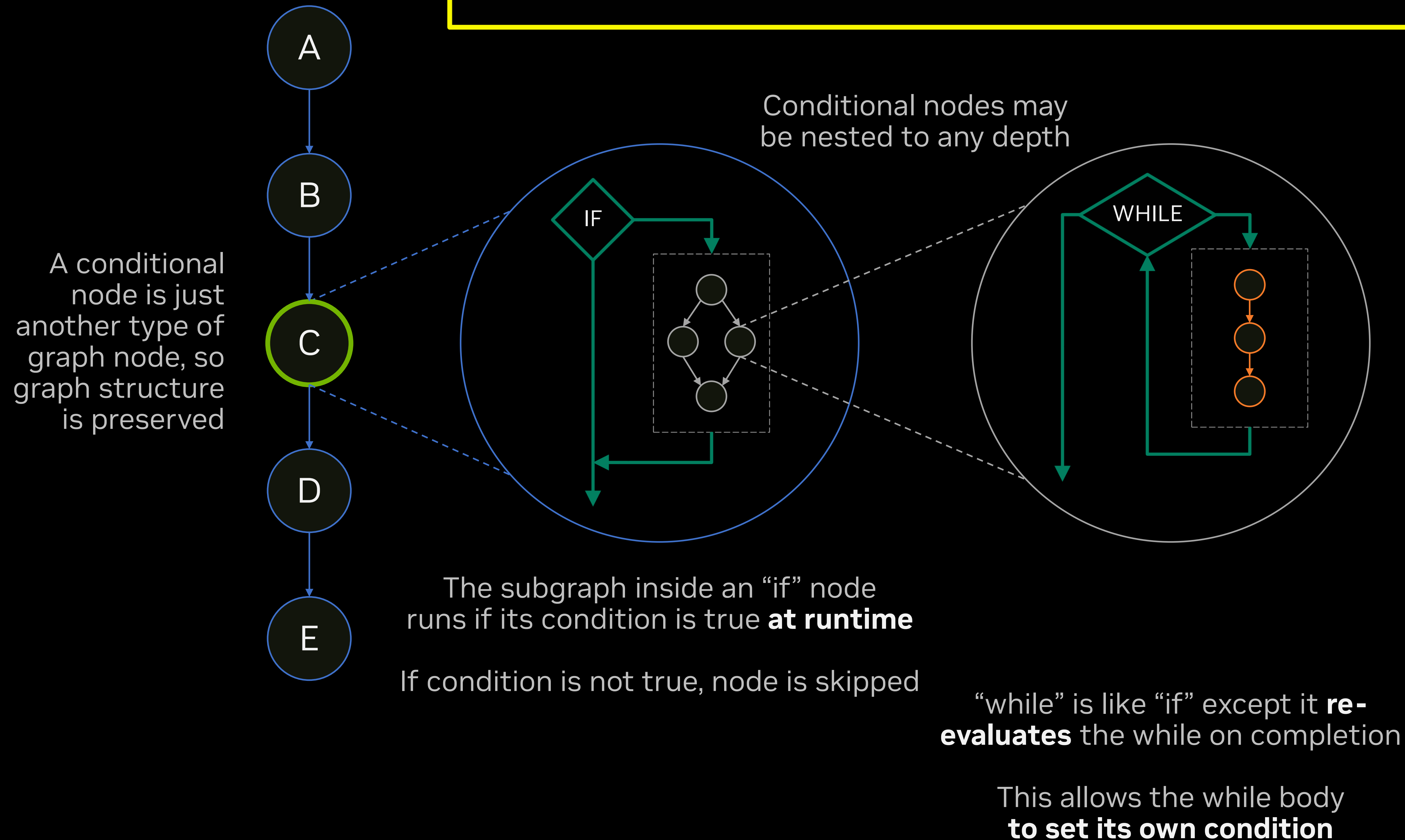
Conditional nodes are just graph nodes so multiple “if”s can function like “switch”

Not limited to a single true condition: **zero, one, some or all conditions may run**

Conditional Graph Nodes

A new type of graph node that contains a subgraph which runs `if()` or `while()` a condition is true

Conditional graph nodes available from CUDA 12.4



Conditional nodes are just graph nodes so multiple “if”s can function like “switch”

Not limited to a single true condition:
zero, one, some or all conditions may run

References

S62162 [Deep Dive into Math Libraries](#)

S61198 [CUTLASS: A Performant, Flexible, and Portable Way to Target Hopper Tensor Cores](#)

S63345 [Warp: Advancing Simulation AI with Differentiable GPU Computing in Python](#)

S62262 [Legate: A Productive Programming Framework for Composable, Scalable, Accelerated Libraries](#)

S62388 [Achieving Higher Performance From Your Data Center and Cloud Application](#)

S61339 [Multi GPU Programming Models for HPC and AI](#)

S62559 [Accelerating and Securing GPU Accesses to Large Datasets](#)

S62129 [Training Deep Learning Models at Scale: How NCCL Enables Best Performance on AI Data Center Networks](#)

S61598 [Scientific Computing With NVIDIA Grace and the Arm Software Ecosystem](#)

S61203 [A Deep Dive into the Latest HPC Software](#)

S62275 [Performance Optimization for Grace CPU Superchip](#)

S62157 [Early Science with Grace Hopper at Scale on Alps](#)

S61159 [Grace Hopper Superchip Architecture and Performance Optimizations for Deep Learning Applications](#)

S61197 [Large-Scale Graph GNN Training Accelerated With cuGraph](#)

CWE62240 [NVIDIA Math Libraries](#)

CWE62915 [Best Practices of GPU Code Development With CUDA NVCC Compiler](#)

CWE61532 [What's in Your CUDA Toolbox? CUDA Profiling, Optimization, and Debugging Tools](#)

CWE63651 [Legate Programming Framework for Composable, Scalable, Accelerated Libraries](#)

CWE61229 [Inter-GPU Communication Techniques and Libraries for HPC and AI](#)

